

xv6：一个简单的、类 Unix 教学操作系统

根据 MIT xv6 英文原版自动翻译整理

Russ Cox Frans Kaashoek Robert Morris 2020 年 8 月 31 日

目录

1 操作系统接口 9

1.1 进程与内存 10

1.2 I/O 与文件描述符 13

1.3 管道 15

1.4 文件系统 17

1.5 现实世界 19

1.6 练习 20

2 操作系统组织 21

2.1 抽象物理资源 22

2.2 用户模式、监督模式与系统调用 22

2.3 内核组织 23

2.4 代码：xv6 组织 24

2.5 进程概述 24

2.6 代码：启动 xv6 与第一个进程 27

2.7 现实世界 28

2.8 练习 28

3 页表 29

3.1 分页硬件 29

3.2 内核地址空间 31

3.3 代码：创建地址空间 33

3.4 物理内存分配 34

3.5 代码：物理内存分配器 34

3.6 进程地址空间 35

3.7 代码：sbrk 36

3.8 代码：exec 37

3.9 现实世界 38

3.10 练习 39

4 陷阱与系统调用 41

4.1 RISC-V 陷阱机制 42

4.2 来自用户空间的陷阱 43

4.3 代码：调用系统调用	44
4.4 代码：系统调用参数	45
4.5 来自内核空间的陷阱	46
4.6 缺页异常	46
4.7 现实世界	48
4.8 练习	48
5 中断与设备驱动程序	49
5.1 代码：控制台输入	49
5.2 代码：控制台输出	50
5.3 驱动程序中的并发	51
5.4 定时器中断	51
5.5 现实世界	52
5.6 练习	53
6 锁	55
6.1 竞争条件	56

6.2 代码：锁	58
6.3 代码：使用锁	60
6.4 死锁与锁排序	60
6.5 锁与中断处理程序	62
6.6 指令与内存排序	62
6.7 睡眠锁	63
6.8 现实世界	64
6.9 练习	64
7 调度	67
7.1 多路复用	67
7.2 代码：上下文切换	68
7.3 代码：调度	69
7.4 代码：mycpu 与 myproc	70
7.5 睡眠与唤醒	71
7.6 代码：睡眠与唤醒	74

7.7 代码：管道	75
7.8 代码：wait、exit 与 kill	76
7.9 现实世界	77
7.10 练习	79
8 文件系统 81	
8.1 概述	81
8.2 缓冲区缓存层	82
8.3 代码：缓冲区缓存	83
8.4 日志层	84
8.5 日志设计	85
8.6 代码：日志	86
8.7 代码：块分配器	87
8.8 inode 层	87
8.9 代码：inode	89
8.10 代码：inode 内容	90

8.11 代码：目录层	91
8.12 代码：路径名	92
8.13 文件描述符层	93
8.14 代码：系统调用	94
8.15 现实世界	95
8.16 练习	96
9 并发回顾	99
9.1 锁模式	99
9.2 类锁模式	100
9.3 完全不用锁	100
9.4 并行性	101
9.5 练习	102
10 总结	103

前言与致谢

本书是一份面向操作系统课程的草稿教材。它通过研究一个名为 xv6 的示例内核来讲解操作系统的主要概念。xv6 以 Dennis Ritchie 和 Ken Thompson 的 Unix Version 6 (v6) [14] 为原型。xv6 大体上遵循 v6 的结构和风格，但以 ANSI C [6] 为实现语言，面向多核 RISC-V [12] 架构。本书应与 xv6 源代码对照阅读，这一方式受到了 John Lions 所著《UNIX 第六版注解》[9] 的启发。有关 v6 与 xv6 的在线资源（包括若干使用 xv6 的实验作业），请参见 <https://pdos.csail.mit.edu/6.S081>。我们已在麻省理工学院的操作系统课程 6.828 与 6.S081 中使用本书。感谢这些课程的教师、助教与同学们，他们以直接或间接的方式为 xv6 做出了贡献。特别感谢 Adam Belay、Austin Clements 和 Nickolai Zeldovich。最后，感谢所有通过电子邮件向我们反馈文本错误或改进建议的朋友：Abutalib Aghayev、Sebastian Boehm、Anton Burtsev、Raphael Carvalho、Tej Chajed、Rasit Eskicioglu、Color Fuzzy、Giuseppe、Tao Guo、Naoki Hayama、Robert Hilderman、Wolfgang Keller、Austin Liew、Pavan Maddamsetti、Jacek Masiulaniec、Michael McConville、m3hm00d、miguelvieira、Mark Morrissey、Harry Pan、Askar Safin、Salman Shah、Adeodato Sim ó、Ruslan Savchenko、Pawel Szczurko、Warren Toomey、tyfkda、tzerbib、Xi Wang 以及 Zou Chang Wei。如果您发现错误或有改进建议，请发送电子邮件至 Frans Kaashoek 和 Robert Morris (kaashoek,rtm@csail.mit.edu)。

第一章：操作系统接口

操作系统的职责是在多个程序之间共享计算机，并提供一套比硬件本身更为丰富的服务。操作系统对底层硬件进行管理和抽象，使得例如文字处理程序无需关心所使用的磁盘硬件类型。操作系统在多个程序之间共享硬件，使它们能够（或看起来能够）同时运行。此外，操作系统为程序之间的交互提供受控的方式，使它们能够共享数据或协同工作。操作系统通过接口向用户程序提供服务。设计一个良好的接口是一件困难的事情。一方面，我们希望接口尽量简单而精简，因为这样更容易正确实现；另一方面，我们又可能倾向于为应用程序提供许多复杂的功能。解决这一矛盾的关键在于：设计依赖少数几种机制的接口，通过组合这些机制来提供强大的通用性。

本书以一个具体的操作系统为例来阐释操作系统概念。该操作系统名为 xv6，它提供了 Ken Thompson 和 Dennis Ritchie 的 Unix 操作系统 [14] 所引入的基本接口，并模仿了 Unix 的内部设计。Unix 提供了一个精简的接口，其机制能够良好地组合，呈现出惊人的通用性。这一接口大获成功，以至于现代操作系统——BSD、Linux、Mac OS X、Solaris，乃至在一定程度上的 Microsoft Windows——都具备类 Unix 的接口。理解 xv6 是理解上述系统及众多其他系统的良好起点。

如图 1.1 所示，xv6 采用传统的内核形式，内核是一个为运行中的程序提供服务的特殊程序。每个运行中的程序称为进程，其内存中包含指令、数据和栈。指令实现程序的计算逻辑，数据是计算所作用的变量，栈用于组织程序的过程调用。一台计算机通常有许多进程，但只有一个内核。

当进程需要调用内核服务时，它会调用系统调用——即操作系统接口中的某个调用。系统调用进入内核，内核执行相应服务后返回。因此，进程在用户空间和内核空间之间交替执行。内核利用 CPU ¹ 提供的硬件保护机制，确保每个

> ¹ 本文通常用 CPU（central processing unit，中央处理单元）一词来指代执行计算的硬件部件。其他文档（如 RISC-V 规范）也会用 processor、core 和 hart 等词代替 CPU。

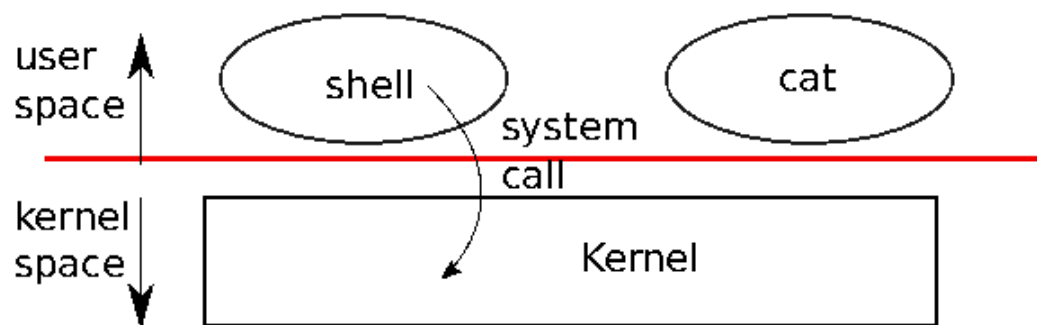


图 1.1：一个内核与两个用户进程。

在用户空间中执行的进程只能访问其自身的内存。内核以实现这些保护所需的硬件特权级别运行，而用户程序则在没有这些特权的情况下运行。当用户程序调用系统调用时，硬件会提升特权级别，并开始执行内核中预先安排好的函数。

内核所提供的系统调用集合，就是用户程序所看到的接口。xv6 内核提供了 Unix 内核传统上所提供的服务和系统调用的一个子集。图 1.2 列出了 xv6 的全部系统调用。

本章其余部分将概述 xv6 的服务——进程、内存、文件描述符、管道以及文件系统——并通过代码片段以及对 shell（Unix 的命令行用户界面）如何使用这些服务的讨论加以说明。shell 对系统调用的使用体现了这些接口设计的精妙之处。

shell 是一个普通程序，它从用户处读取命令并加以执行。shell 是用户程序而非内核的一部分，这一事实体现了系统调用接口的强大：shell 本身并无任何特殊之处。这也意味着 shell 易于替换；因此，现代 Unix 系统提供了多种可供选择的 shell，每种都有其自己的用户界面和脚本功能。xv6 的 shell 是 Unix Bourne shell 精髓的简单实现，其实现可在 (user/sh.c:1) 中找到。

1.1 进程与内存

一个 xv6 进程由用户空间内存（指令、数据和栈）以及内核私有的每进程状态组成。xv6 对进程进行时间共享：它在等待执行的进程集合之间透明地切换可用的 CPU。当一个进程未在运行时，xv6 会保存其 CPU 寄存器，并在下次运行该进程时将其恢复。内核为每个进程关联一个进程标识符，即 PID。进程可以使用 `fork` 系统调用创建新进程。`fork` 创建一个新进程，称为子进程，其内存内容与调用进程（称为父进程）完全相同。`fork` 在父进程和子进程中均会返回。在父进程中，`fork` 返回子进程的 PID；在子进程中，`fork` 返回零。例如，考虑以下用 C 语言编写的程序片段 [6]：

```
int pid = fork();
if(pid > 0){
    printf("parent: child=%d\n", pid);
```

System call	Description
<code>int fork()</code>	Create a process, return child's PID.
<code>int exit(int status)</code>	Terminate the current process; status reported to <code>wait()</code> . No return.
<code>int wait(int *status)</code>	Wait for a child to exit; exit status in <code>*status</code> ; returns child PID.
<code>int kill(int pid)</code>	Terminate process PID. Returns 0, or -1 for error.
<code>int getpid()</code>	Return the current process's PID.
<code>int sleep(int n)</code>	Pause for n clock ticks.
<code>int exec(char *file, char *argv[])</code>	Load a file and execute it with arguments; only returns if error.
<code>char *sbrk(int n)</code>	Grow process's memory by n bytes. Returns start of new memory.
<code>int open(char *file, int flags)</code>	Open a file; flags indicate read/write; returns an fd (file descriptor).
<code>int write(int fd, char *buf, int n)</code>	Write n bytes from buf to file descriptor fd; returns n.
<code>int read(int fd, char *buf, int n)</code>	Read n bytes into buf; returns number read; or 0 if end of file.
<code>int close(int fd)</code>	Release open file fd.
<code>int dup(int fd)</code>	Return a new file descriptor referring to the same file as fd.
<code>int pipe(int p[])</code>	Create a pipe, put read/write file descriptors in p[0] and p[1].
<code>int chdir(char *dir)</code>	Change the current directory.
<code>int mkdir(char *dir)</code>	Create a new directory.
<code>int mknod(char *file, int, int)</code>	Create a device file.
<code>int fstat(int fd, struct stat *st)</code>	Place info about an open file into *st.
<code>int stat(char *file, struct stat *st)</code>	Place info about a named file into *st.
<code>int link(char *file1, char *file2)</code>	Create another name (file2) for the file file1.
<code>int unlink(char *file)</code>	Remove a file.

图 1.2：xv6 系统调用。除非另有说明，这些调用在无错误时返回 0，出错时返回 -1。

```

    pid = wait((int *) 0);
    printf("child %d is done\n", pid);
} else if(pid == 0){
    printf("child: exiting\n");
    exit(0);
} else {
    printf("fork error\n");
}
}

```

`exit` 系统调用使调用进程停止执行，并释放诸如内存和打开文件等资源。`exit`

接受一个整数状态参数，按惯例 0 表示成功，1 表示失败。`wait`

系统调用返回当前进程中某个已退出（或被终止）的子进程的

PID，并将该子进程的退出状态复制到传递给 `wait` 的地址处；如果调用者的子进程均未退出，`wait` 会等待直到某个子进程退出。如果调用者没有子进程，`wait` 立即返回

-1。如果父进程不关心子进程的退出状态，可以向 `wait` 传入地址 0。在上面的示例中，输出行

```

parent: child=1234
child: exiting

```

的顺序可能不固定，取决于父进程还是子进程先执行到 `printf` 调用。子进程退出后，父进程的 `wait` 返回，导致父进程打印 `parent: child 1234 is done`。尽管子进程最初与父进程拥有相同的内存内容，但父进程和子进程使用各自独立的内存和寄存器执行：在一个进程中修改变量不会影响另一个进程。

例如，当 `wait` 的返回值被存入父进程中的变量 `pid` 时，并不会改变子进程中的变量 `pid`。子进程中 `pid` 的值仍然为零。

`exec` 系统调用将调用进程的内存替换为从文件系统中某个文件加载的新内存映像。该文件必须具有特定格式，其中指定了文件哪个部分存放指令、哪个部分是数据、从哪条指令开始执行等信息。xv6 使用 ELF 格式，第 3 章对此进行了更详细的讨论。当 `exec` 成功时，它不会返回到调用程序；而是从 ELF 头中声明的入口点开始执行从文件加载的指令。`exec` 接受两个参数：包含可执行文件的文件名，以及一个字符串参数数组。例如：

```
char *argv[3];

argv[0] = "echo";
argv[1] = "hello";
argv[2] = 0;
exec("/bin/echo", argv);
printf("exec error\n");
```

这段代码将调用程序替换为以参数列表 `echo hello` 运行的程序 `/bin/echo`。大多数程序会忽略参数数组的第一个元素，按惯例它是程序的名称。

xv6 的 shell 使用上述调用来代表用户运行程序。shell 的主体结构很简单；参见 `main (user/sh.c:145)`。主循环使用 `getcmd` 从用户读取一行输入，然后调用 `fork`，创建一个 shell 进程的副本。父进程调用 `wait`，而子进程则执行该命令。例如，如果用户向 shell 输入了 `"echo hello"`，则 `runcmd` 会以 `"echo hello"` 作为参数被调用。`runcmd (user/sh.c:58)` 执行实际的命令。对于 `"echo hello"`，它会调用 `exec (user/sh.c:78)`。如果 `exec` 成功，子进程将执行来自 `echo` 的指令，而不再执行 `runcmd` 中的指令。在某个时刻，`echo` 会调用 `exit`，这将导致父进程从 `main (user/sh.c:145)` 中的 `wait` 返回。

你可能会疑惑为什么 `fork` 和 `exec` 没有合并为一个调用；我们稍后会看到，shell 在实现 I/O 重定向时正是利用了这两者的分离。为了避免先创建一个重复进程、随即又立即替换它（通过 `exec`）的浪费，操作内核通过使用虚拟内存技术（如写时复制，参见第 4.6 节）来优化 `fork` 在此场景下的实现。

xv6 大多数情况下隐式地分配用户空间内存：`fork` 为子进程分配存放父进程内存副本所需的内存，`exec` 则分配足以容纳可执行文件的内存。如果进程在运行时需要更多内存（例如用于 `malloc`），可以调用 `sbrk(n)` 将其数据内存扩大 `n` 字节；`sbrk` 返回新内存的起始地址。

1.2 I/O 与文件描述符

文件描述符是一个小整数，代表内核管理的某个对象，进程可以从中读取数据或向其写入数据。进程可以通过打开文件、目录或设备，或通过创建管道，或通过复制已有的描述符来获得文件描述符。为简便起见

，我们通常将文件描述符所指向的对象称为"文件"；文件描述符接口抽象掉了文件、管道和设备之间的差异，使它们看起来都像字节流。我们将输入和输出统称为 I/O。在内核内部，xv6 内核使用文件描述符作为每个进程的表的索引，因此每个进程都拥有一个从零开始的私有文件描述符空间。按照惯例，进程从文件描述符 0（标准输入）读取数据，将输出写入文件描述符 1（标准输出），将错误消息写入文件描述符 2（标准错误）。正如我们将看到的，shell 利用这一惯例来实现 I/O 重定向和管道。shell 确保它始终有三个打开的文件描述符（user/sh.c:151），默认情况下这些文件描述符对应控制台。read 和 write 系统调用用于从文件描述符命名的已打开文件中读取字节或向其写入字节。调用 read(fd, buf, n) 从文件描述符 fd 中最多读取 n 个字节，将它们复制到 buf 中，并返回实际读取的字节数。每个指向文件的文件描述符都有一个与之关联的偏移量。read 从当前文件偏移处读取数据，然后将偏移量向前推进已读取的字节数：后续的 read 将返回第一次 read 所返回字节之后的内容。当没有更多字节可读时，read 返回零以表示文件结束。调用 write(fd, buf, n) 将 buf 中的 n 个字节写入文件描述符 fd，并返回实际写入的字节数。只有发生错误时，写入的字节数才会少于 n。与 read 类似，write 在当前文件偏移处写入数据，然后将偏移量向前推进已写入的字节数：每次 write 都从上一次结束的地方继续。下面这段程序片段（构成了程序 cat 的核心）将数据从标准输入复制到标准输出。若发生错误，则向标准错误写入一条消息。

```
char buf[512];
int n;

for(;;){
    n = read(0, buf, sizeof buf);
    if(n == 0)
        break;
    if(n < 0){
        fprintf(2, "read error\n");
        exit(1);
    }

    if(write(1, buf, n) != n){
        fprintf(2, "write error\n");
        exit(1);
    }
}
```

这段代码片段中需要注意的重要一点是，cat 并不知道它是在从文件、控制台还是管道中读取数据。类似地，cat 也不知道它是在向控制台、文件还是其他对象输出。文件描述符的使用，以及文件描述符 0 为输入、文件描述符 1 为输出的惯例，使得 cat 的实现得以简洁。close 系统调用释放一个文件描述符，使其可被后续的 open、pipe 或 dup 系统调用（见下文）重新使用。新分配的文件描述符始终是当前进程中编号最小的未使用描述符。文件描述符与 fork 的交互使得 I/O 重定向易于实现。fork 在复制父进程内存的同时也复制其文件描述符表，因此子进程开始时拥有与父进程完全相同的已打开文件。系统调用 exec 替换调用进程的内存，但保留其文件表。这一行为使得 shell 可以通过 fork、在子进程中重新打开所选的文件描述符，然后调用 exec 运行新程序来实现 I/O

重定向。下面是 shell 执行命令 `cat < input.txt` 时所运行代码的简化版本：

```
char *argv[2];

argv[0] = "cat";
argv[1] = 0;
if(fork() == 0) {
    close(0);
    open("input.txt", O_RDONLY);
    exec("cat", argv);
}
```

子进程关闭文件描述符 0 之后，`open` 必然会将该文件描述符分配给新打开的 `input.txt`：0 将是最小的可用文件描述符。此后 `cat` 执行时，文件描述符 0（标准输入）指向 `input.txt`。由于这一操作序列仅修改子进程的描述符，父进程的文件描述符不受影响。xv6 shell 中 I/O 重定向的代码正是以这种方式工作的（`user/sh.c:82`）。回想一下，在代码执行到这一点时，shell 已经 `fork` 了子 shell，`runcmd` 将调用 `exec` 来加载新程序。`open` 的第二个参数由一组标志位组成，用于控制 `open` 的行为。这些标志的可能取值定义在文件控制（`fcntl`）头文件（`kernel/fcntl.h:1-5`）中：`O_RDONLY`、`O_WRONLY`、`O_RDWR`、`O_CREATE` 和 `O_TRUNC`，分别指示 `open` 以只读、只写、读写方式打开文件，若文件不存在则创建文件，以及将文件截断为零长度。现在应该清楚为什么 `fork` 和 `exec` 作为独立的调用是有益的：在这两个调用之间，shell 有机会重定向子进程的 I/O，而不影响主 shell 的 I/O 配置。也可以设想一个假想的合并系统调用 `forkexec`，但使用这样的调用进行 I/O 重定向的选项显得很笨拙。shell 可以在调用 `forkexec` 之前修改自身的 I/O 配置（然后再撤销这些修改）；或者 `forkexec` 可以将 I/O 重定向的指令作为参数接收；或者（最不理想的方案）每个类似 `cat` 的程序都需要自行处理 I/O 重定向。尽管 `fork` 复制了文件描述符表，但每个底层文件的偏移量在父子进程之间是共享的。考虑下面这个例子：

```
if(fork() == 0) {
    write(1, "hello ", 6);
    exit(0);
} else {
    wait(0);
    write(1, "world\n", 6);
}
```

在这段代码片段执行完毕后，附加到文件描述符 1 的文件将包含数据 `hello world`。父进程中的 `write`（由于 `wait`，只在子进程完成后才运行）从子进程 `write` 结束的地方继续写入。这一行为有助于从 shell 命令序列中产生顺序输出，例如 `(echo hello; echo world) >output.txt`。`dup` 系统调用复制一个已有的文件描述符，返回一个新的、指向同一底层 I/O 对象的描述符。两个文件描述符共享同一偏移量，就像 `fork` 所复制的文件描述符一样。下面是将 `hello world` 写入文件的另一种方式：

```
fd = dup(1);
write(1, "hello ", 6);
write(fd, "world\n", 6);
```


如果两个文件描述符是通过一系列 `fork` 和 `dup` 调用从同一个原始文件描述符派生而来的，它们就共享同一偏移量。否则，即使两个文件描述符是通过同一文件的 `open` 调用得到的，它们也不共享偏移量。`dup` 使 shell 能够实现如下命令：`ls existing-file non-existing-file > tmp1 2>&1`。其中 `2>&1` 告诉 shell 为该命令提供一个文件描述符 2，它是描述符 1 的副本。已存在文件的文件名和不存在文件的错误消息都将出现在文件 `tmp1` 中。xv6 的 shell 不支持对错误文件描述符的 I/O 重定向，但现在你已经知道如何实现它了。文件描述符是一种强大的抽象，因为它们隐藏了所连接对象的细节：向文件描述符 1 写入数据的进程，可能是在写入一个文件、一个类似控制台的设备，或者一个管道。

1.3 管道

管道是内核中的一个小缓冲区，以一对文件描述符的形式暴露给进程，一个用于读取，一个用于写入。向管道一端写入数据后，该数据便可从管道另一端读取。管道为进程间通信提供了一种方式。以下示例代码将程序 `wc` 的标准输入连接到管道的读取端并运行该程序。

```
int p[2];
char *argv[2];

argv[0] = "wc";
argv[1] = 0;

pipe(p);
if(fork() == 0) {
    close(0);
    dup(p[0]);
    close(p[0]);
    close(p[1]);
    exec("/bin/wc", argv);
} else {
    close(p[0]);
    write(p[1], "hello world\n", 12);
    close(p[1]);
}
```

程序调用 `pipe`，创建一个新管道，并将读写文件描述符记录在数组 `p` 中。执行 `fork` 后，父进程和子进程都拥有指向该管道的文件描述符。子进程调用 `close` 和 `dup`，使文件描述符 0 指向管道的读取端，关闭 `p` 中的文件描述符，然后调用 `exec` 运行 `wc`。当 `wc` 从其标准输入读取时，实际上是从管道读取。父进程关闭管道的读取端，向管道写入数据，然后关闭写入端。如果没有数据可用，对管道的 `read` 操作将会阻塞，直到有数据被写入，或者所有指向写入端的文件描述符都被关闭；在后一种情况下，`read` 将返回 0，就像到达了数据文件的末尾一样。`read` 会阻塞直到不可能有新数据到来，这一特性正是上述代码中子进程在执行 `wc` 之前必须关闭管道写入端的重要原因：如果 `wc` 的某个文件描述符指向管道的写入端，`wc` 将永远无法看到文件末尾（end-of-file）。xv6 的 shell 以类似上述代码的方式实现管道，例如 `grep fork`

sh.c | wc

-l (user/sh.c:100)。子进程创建一个管道，将管道左端与右端连接起来。然后它对管道左端调用 fork 和 runcmd，对管道右端也调用 fork 和 runcmd，并等待两者都完成。管道的右端可能本身就包含一个管道的命令（例如 a | b | c），该命令本身会再 fork 出两个新的子进程（一个用于 b，一个用于 c）。因此，shell 可能会创建一棵进程树。这棵树的叶节点是各个命令，内部节点是等待左右子进程完成的进程。从原理上讲，可以让内部节点运行管道的左端，但正确实现这一点会使实现变得复杂。考虑仅做如下修改：

修改 sh.c，使其不为 p->left 调用 fork，而是在内部进

程中直接运行 runcmd(p->left)。那么，例如 echo hi | wc 将不会产生任何输出，因为当 echo hi 在 runcmd 中退出时，内部进程也随之退出，永远不会调用 fork 来运行管道的右端。这种错误行为可以通过在 runcmd 中不为内部进程调用 exit 来修复，但这个修复会使代码变得复杂：此时 runcmd 需要知道自己是否是内部进程。不为

runcmd(p->right) 调用 fork 同样会带来复杂性。例如，仅做该修改后，

sleep 10 | echo hi 将会立即打印 "hi"，而不是等待 10 秒后再打印，因为 echo 会立即运行并退出，而不等待 sleep 完成。由于 sh.c 的目标是尽可能简单，它并不尝试避免创建内部进程。管道看起来可能并不比临时文件更强大：管道 echo hello world | wc 完全可以不使用管道来实现，如 echo hello world >/tmp/xyz; wc </tmp/xyz。在这种情况下，管道相比临时文件至少有四个优势。第一，管道会自动清理自身；而使用文件重定向时，shell 必须在完成后小心地删除 /tmp/xyz。第二，管道可以传递任意长度的数据流，而文件重定向需要磁盘上有足够的空闲空间来存储所有数据。第三，管道允许管道各阶段并行执行，而文件方式要求第一个程序完成后第二个程序才能开始。第四，如果要实现进程间通信，管道的阻塞式读写比文件的非阻塞语义更加高效。

1.4 文件系统

xv6 文件系统提供数据文件和目录。数据文件包含未经解释的字节数组，目录则包含对数据文件和其他目录的命名引用。目录构成一棵树，起点是一个名为根目录的特殊目录。像 /a/b/c 这样的路径，指的是根目录 / 下目录 a 中目录 b 里名为 c 的文件或目录。不以 / 开头的路径会相对于调用进程的当前目录进行解析，当前目录可以通过 chdir 系统调用来更改。下面两段代码打开的是同一个文件（假设涉及的所有目录均已存在）：

```
chdir("/a");
chdir("b");
open("c", O_RDONLY);

open("/a/b/c", O_RDONLY);
```


第一段代码将进程的当前目录更改为 `/a/b`；第二段代码既不引用也不更改进程的当前目录。系统调用中有专门用于创建新文件和目录的接口：`mkdir` 创建新目录，带有 `O_CREATE` 标志的 `open` 创建新数据文件，`mknod` 创建新设备文件。下面的示例展示了这三者的用法：

```
mkdir("/dir");
fd = open("/dir/file", O_CREATE|O_WRONLY);
close(fd);

mknod("/console", 1, 1);
```

`mknod` 创建一个指向某个设备的特殊文件。设备文件关联有主设备号和次设备号（即 `mknod` 的两个参数），它们唯一标识一个内核设备。当进程后续打开该设备文件时，内核会将 `read` 和 `write` 系统调用转发至内核设备的实现，而不是传递给文件系统。文件的名称与文件本身是相互独立的；同一个底层文件（称为 `inode`）可以拥有多个名称，这些名称称为链接。每个链接对应目录中的一条条目标，该条目包含文件名以及对 `inode` 的引用。`inode` 保存着文件的元数据，包括其类型（文件、目录或设备）、长度、文件内容在磁盘上的位置，以及指向该文件的链接数。`fstat` 系统调用可从文件描述符所引用的 `inode` 中获取信息，并将其填入 `struct stat`，该结构体定义于 `stat.h`（`kernel/stat.h`）中，如下所示：

```
#define T_DIR          1      // Directory
#define T_FILE         2      // File
#define T_DEVICE       3      // Device

struct stat {
    int dev;           // File system's disk device
    uint ino;          // Inode number
    short type;        // Type of file
    short nlink;       // Number of links to file
    uint64 size;       // Size of file in bytes
};
```

`link` 系统调用为与已有文件相同的 `inode` 创建另一个文件系统名称。下面这段代码创建了一个同时名为 `a` 和 `b` 的新文件：

```
open("a", O_CREATE|O_WRONLY);
link("a", "b");
```

对 `a` 进行读写与对 `b` 进行读写效果相同。每个 `inode` 由唯一的 `inode` 编号标识。在执行上述代码序列之后，可以通过检查 `fstat` 的结果来确认 `a` 和 `b` 引用的是相同的底层内容：两者将返回相同的 `inode` 编号（`ino`），且 `nlink` 计数将被设置为 2。`unlink` 系统调用从文件系统中移除一个名称。只有当文件的链接计数为零且没有文件描述符引用它时，文件的 `inode` 和存储其内容的磁盘空间才会被释放。因此，在上述代码序列末尾添加：

```
unlink("a");
```

之后，该 `inode` 和文件内容仍可通过 `b` 访问。此外，

```
fd = open("/tmp/xyz", O_CREATE|O_RDWR);
```

```
unlink("/tmp/xyz");
```

这是一种惯用写法，用于创建一个没有名称的临时 inode，当进程关闭 fd 或退出时，该 inode 会被自动清理。

Unix 提供了可从 shell 以用户级程序形式调用的文件工具，例如 `mkdir`、`ln` 和 `rm`。这种设计允许任何人通过添加新的用户级程序来扩展命令行界面。事后来看，这个方案显而易见，但与 Unix 同时代设计的其他系统往往将此类命令内置于 shell 中（并将 shell 内置于内核中）。有一个例外是 `cd`，它被内置于 shell 中（`user/sh.c:160`）。`cd` 必须更改 shell 自身的当前工作目录。如果 `cd` 作为普通命令运行，那么 shell 会 fork 一个子进程，子进程运行 `cd`，`cd` 会更改子进程的工作目录，而父进程（即 shell）的工作目录则不会改变。

1.5 现实世界

Unix 将"标准"文件描述符、管道以及便捷的 shell 语法结合在一起，用于对上述概念进行操作，这是编写通用可复用程序方面的重大进步。这一理念催生了"软件工具"文化，正是这种文化造就了 Unix 的强大与广泛流行，而 shell 也成为了第一种所谓的"脚本语言"。Unix 系统调用接口在 BSD、Linux 和 Mac OS X 等系统中延续至今。Unix

系统调用接口已通过可移植操作系统接口（POSIX）标准进行了规范化。Xv6 并不符合 POSIX 标准：它缺少许多系统调用（包括 `lseek` 等基本系统调用），且它所提供的许多系统调用与标准有所不同。我们为 xv6 设定的主要目标是简洁与清晰，同时提供一个简单的类 UNIX 系统调用接口。已有若干人通过添加更多系统调用和一个简单的 C 语言库对 xv6 进行了扩展，以便运行基本的 Unix 程序。然而，现代内核所提供的系统调用以及内核服务的种类远比 xv6 丰富。例如，它们支持网络、窗口系统、用户级线程、各类设备的驱动程序等。现代内核持续快速演进，提供了许多超越 POSIX 的特性。

Unix 通过统一的文件名与文件描述符接口，将对多种资源类型（文件、目录和设备）的访问整合在一起。这一理念可以进一步推广到更多类型的资源；Plan 9 [13] 就是一个很好的例子，它将"资源即文件"的概念应用于网络、图形等领域。然而，大多数源自 Unix 的操作系统并未沿用这一路线。

文件系统与文件描述符一直是强大的抽象机制。尽管如此，操作系统接口还存在其他模型。Unix 的前身 Multics 以一种使文件存储看起来像内存的方式对其进行了抽象，形成了一种截然不同的接口风格。Multics 设计的复杂性直接影响了 Unix 的设计者，促使他们致力于构建更为简单的系统。

Xv6 没有提供用户的概念，也不提供用户间的相互保护；以 Unix 的术语来说，所有 xv6 进程都以 root 身份运行。本书探讨 xv6 如何实现其类 Unix 接口，但其中的思想与概念的适用范围远不止 Unix 本身。任何操作系统都必须将多个进程多路复用到硬件上，实现进程间的相互隔离，并提供受控的进程间通信机制。学习 xv6 之后，你应当能够审视其他更为复杂的操作系统，并在其中发现 xv6 所蕴含的核心概念。

1.6 练习

1. 编写一个程序，使用 UNIX 系统调用，通过一对管道（每个方向各一个）在两个进程之间"乒乓"传递一个字节。测量该程序的性能，以每秒交换次数为单位。

第二章：操作系统组织结构

操作系统的一个关键需求是同时支持多个活动。例如，使用第一章所描述的系统调用接口，一个进程可以通过 `fork`

启动新的进程。操作系统必须在这些进程之间分时共享计算机的资源。例如，即使进程数量多于硬件 CPU 的数量，操作系统也必须确保所有进程都有机会执行。操作系统还必须安排进程之间的隔离。也就是说，如果某个进程存在缺陷并发生故障，它不应该影响那些不依赖于该问题进程的其他进程。然而，完全隔离又过于严格，因为进程之间应当可以有意地进行交互；管道就是一个例子。因此，操作系统必须满足三个需求：多路复用、隔离和交互。本章概述了操作系统如何组织以实现这三个需求。实现这些需求的方式有很多，但本书重点介绍以宏内核为核心的主流设计，这也是许多 Unix

操作系统所采用的方式。本章还概述了 xv6 进程——它是 xv6 中隔离的基本单元——以及 xv6 启动时第一个进程的创建过程。Xv6 运行在多核¹ RISC-V

微处理器上，其许多底层功能（例如进程实现）都是针对 RISC-V 的。RISC-V 是一款 64 位 CPU，xv6 使用“LP64”C 语言编写，这意味着 C 语言中的 `long` (L) 和指针 (P) 均为 64 位，而 `int` 为 32 位。本书假设读者已在某种体系结构上进行过一定的机器级编程，并将在相关内容出现时介绍 RISC-V 特有的概念。RISC-V 的一个有用参考资料是《The RISC-V Reader: An Open Architecture Atlas》[12]。用户级 ISA [2] 和特权架构 [1] 是官方规范。一台完整计算机中的 CPU 被各种支持硬件所围绕，其中大部分以 I/O 接口的形式存在。Xv6 是为 qemu 的 `-machine virt` 选项所模拟的支持硬件而编写的，这包括 RAM、一个包含启动代码的 ROM、一个连接用户键盘/屏幕的串行接口，以及一个用于存储的磁盘。

¹ 本书中“多核”是指多个共享内存但并行执行的 CPU，每个 CPU 拥有自己独立的寄存器组。本书有时也使用“多处理器”作为多核的同义词，尽管多处理器更具体地可以指代拥有多个独立处理器芯片的计算机。

2.1 抽象物理资源

当人们初次接触操作系统时，可能会问的第一个问题是：为什么需要它？也就是说，可以将图 1.2 中的系统调用实现为一个库，由应用程序与之链接。在这种方案下，每个应用程序甚至可以拥有根据自身需求定制的库。应用程序可以直接与硬件资源交互，并以最适合该应用程序的方式使用这些资源（例如，以实现高性能或可预测的性能）。某些用于嵌入式设备或实时系统的操作系统就采用这种方式组织。这种库方式的缺点在于，如果有多个应用程序同时运行，这些应用程序必须表现良好。例如，每个应用程序必须定期放弃 CPU，以便其他应用程序能够运行。如果所有应用程序相互信任且没有缺陷，这种协作式分时方案或许可行。但更常见的情况是应用程序之间互不信任，且存在缺陷，因此人们往往希望比协作式方案更强的隔离性。为了实现强隔离，禁止应用程序直接访问敏感硬件资源，转而将资源抽象为服务，是一种有效的方法。例如，Unix 应用程序仅通过文件系统的 `open`、`read`、`write` 和 `close` 系统调用与存储设备交互，而不是直接读写磁盘。这为应用程序提供了路径名的便利性，同时允许操作系统（作为接口的实现者）管理磁盘。即使隔离性不是问题，有意进行交互（或只是希望互不干扰）的程序也很可能发现，文件系统比直接使用磁盘是一种更方便的抽象。类似地，Unix 在进程之间透明地切换硬件 CPU，在必要时保存和恢复寄存器状态，因此应用程序无需感知分时机制。这种透明性使得操作系统即使在某些应用程序陷入

无限循环时也能共享 CPU。再举一个例子，Unix 进程使用 `exec` 来构建其内存映像，而不是直接与物理内存交互。这允许操作系统决定将进程放置在内存中的哪个位置；如果内存紧张，操作系统甚至可以将进程的部分数据存储在磁盘上。`exec` 还为用户提供了使用文件系统存储可执行程序映像的便利。Unix 进程之间的许多交互形式都通过文件描述符进行。文件描述符不仅抽象了许多细节（例如，管道或文件中的数据存储在哪儿），而且其定义方式也简化了交互。例如，如果管道中的某个应用程序失败，内核会为管道中的下一个进程生成文件结束信号。图 1.2

中的系统调用接口经过精心设计，既为程序员提供了便利，又具备实现强隔离的可能性。Unix 接口并非抽象资源的唯一方式，但它已被证明是一种非常好的方式。

2.2 用户模式、监督者模式与系统调用

强隔离需要在应用程序与操作系统之间设置一道严格的边界。如果应用程序发生错误，我们不希望操作系统崩溃，也不希望其他应用程序受到影响。相反，操作系统应当能够清理出错的应用程序，并继续运行其他应用程序。为了实现强隔离，操作系统必须做到：应用程序无法修改（甚至无法读取）操作系统的数据结构和指令，且应用程序无法访问其他进程的内存。

CPU 为强隔离提供了硬件支持。例如，RISC-V 提供了三种 CPU 可执行指令的模式：机器模式（machine mode）、监督者模式（supervisor mode）和用户模式（user mode）。在机器模式下执行的指令拥有完全特权；CPU

启动时处于机器模式。机器模式主要用于计算机的配置。Xv6

在机器模式下执行几行代码后，随即切换至监督者模式。在监督者模式下，CPU 被允许执行特权指令，例如：启用和禁用中断、读写保存页表地址的寄存器等。如果处于用户模式的应用程序试图执行特权指令，CPU 不会执行该指令，而是切换到监督者模式，以便监督者模式下的代码能够终止该应用程序，因为它做了不应该做的事情。

第 1 章的 Figure 1.1

展示了这一组织结构。应用程序只能执行用户模式指令（例如加法运算等），称其运行在用户空间（user space）；而监督者模式下的软件还可以执行特权指令，称其运行在内核空间（kernel space）。运行在内核空间（即监督者模式）中的软件称为内核（kernel）。

若应用程序希望调用内核函数（例如 xv6 中的 `read` 系统调用），则必须切换到内核。CPU

提供了一条特殊指令，可将 CPU

从用户模式切换至监督者模式，并在内核指定的入口点进入内核。（RISC-V 为此提供了 `ecall` 指令。）一旦 CPU 切换至监督者模式，内核便可验证系统调用的参数，判断应用程序是否被允许执行所请求的操作，进而拒绝或执行该操作。

至关重要的是，内核必须控制切换至监督者模式的入口点。如果应用程序能够自行决定内核入口点，恶意应用程序便可以，例如，在跳过参数验证的位置进入内核。

2.3 内核组织

一个关键的设计问题是：操作系统的哪些部分应该运行在监管者模式下。一种可能的方案是将整个操作系统置于内核中，使所有系统调用的实现都运行在监管者模式下。这种组织方式称为宏内核。在这种组织方式中，整个操作系统以完整的硬件特权运行。这种组织方式较为便利，因为操作系统设计者无需决定哪些部分不需要完整的硬件特权。此外，操作系统各部分之间的协作也更加容易。例如，操作系统可能拥有一个缓冲区缓存，可以同时被文件系统和虚拟内存系统共享。宏内核组织方式的缺点在于，操作系统各部分之间的接口往往较为复杂（正如我们在本书后续内容中将看到的那样），因此操作系统开发者很容易犯错。在宏内核中，错误是致命的，因为监管者模式下的错误往往会导致内核崩溃。一旦内核崩溃，

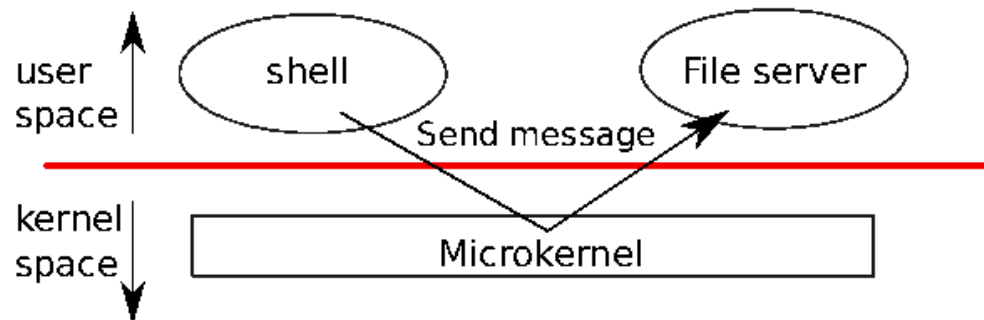


图 2.1：带有文件系统服务器的微内核

计算机将停止工作，所有应用程序也随之失败。计算机必须重新启动才能恢复运行。为了降低内核中出错的风险，操作系统设计者可以尽量减少运行在监管者模式下的操作系统代码量，将操作系统的大部分功能放在用户模式下执行。这种内核组织方式称为微内核。图 2.1 展示了这种微内核设计。在图中，文件系统作为用户级进程运行。以进程形式运行的操作系统服务称为服务器。为了允许应用程序与文件服务器交互，内核提供了一种进程间通信机制，用于在用户模式进程之间传递消息。例如，如果像 shell 这样的应用程序想要读写文件，它会向文件服务器发送一条消息并等待响应。在微内核中，内核接口由少量低层次函数组成，用于启动应用程序、发送消息、访问设备硬件等。这种组织方式使内核相对简单，因为操作系统的大部分功能驻留在用户级服务器中。xv6 与大多数 Unix 操作系统一样，实现为宏内核。因此，xv6 的内核接口对应于操作系统接口，内核实现了完整的操作系统。由于 xv6 提供的服务不多，其内核比某些微内核更小，但从概念上讲，xv6 是单体的（monolithic）。

2.4 代码：xv6 组织结构

xv6 内核源码位于 `kernel/` 子目录中。源码按照粗略的模块化概念划分为多个文件；图 2.2 列出了这些文件。模块间的接口定义在 `defs.h` (`kernel/defs.h`) 中。

2.5 进程概述

xv6（与其他 Unix 操作系统一样）中的隔离单元是进程。进程抽象防止一个进程破坏或窥探另一个进程的内存、CPU、文件描述符等。它也防止进程破坏内核本身，从而使进程无法颠覆内核的隔离机制。内核必须谨慎地实现进程抽象，因为有缺陷或恶意的应用程序可能诱使内核或硬件执行有害操作（例如，绕过隔离）。内核用于实现

File	Description
bio.c	Disk block cache for the file system.
console.c	Connect to the user keyboard and screen.
entry.S	Very first boot instructions.
exec.c	exec() system call.
file.c	File descriptor support.
fs.c	File system.
kalloc.c	Physical page allocator.
kernelvec.S	Handle traps from kernel, and timer interrupts.
log.c	File system logging and crash recovery.
main.c	Control initialization of other modules during boot.
pipe.c	Pipes.
plic.c	RISC-V interrupt controller.
printf.c	Formatted output to the console.
proc.c	Processes and scheduling.
sleeplock.c	Locks that yield the CPU.
spinlock.c	Locks that don't yield the CPU.
start.c	Early machine-mode boot code.
string.c	C string and byte-array library.
swtch.S	Thread switching.
syscall.c	Dispatch system calls to handling function.
sysfile.c	File-related system calls.
sysproc.c	Process-related system calls.
trampoline.S	Assembly code to switch between user and kernel.
trap.c	C code to handle and return from traps and interrupts.
uart.c	Serial-port console device driver.
virtio_disk.c	Disk device driver.
vm.c	Manage page tables and address spaces.

图 2.2：Xv6 内核源文件。

进程的机制包括用户/超级用户模式标志、地址空间以及线程的时间片轮转。为了帮助强制执行隔离，进程抽象向程序提供了一种幻象，使其认为自己拥有独立的私有机器。进程为程序提供了看似私有的内存系统，即地址空间，其他进程无法读写该地址空间。进程还为程序提供了看似专属的 CPU 以执行程序指令。Xv6 使用页表（由硬件实现）为每个进程提供其自己的地址空间。RISC-V 页表将虚拟地址（RISC-V 指令所操作的地址）转换（或"映射"）为物理地址（CPU 芯片发送到主内存的地址）。Xv6 为每个进程维护一张独立的页表，该页表定义了该进程的地址空间。如图 2.3 所示，地址空间包含从虚拟

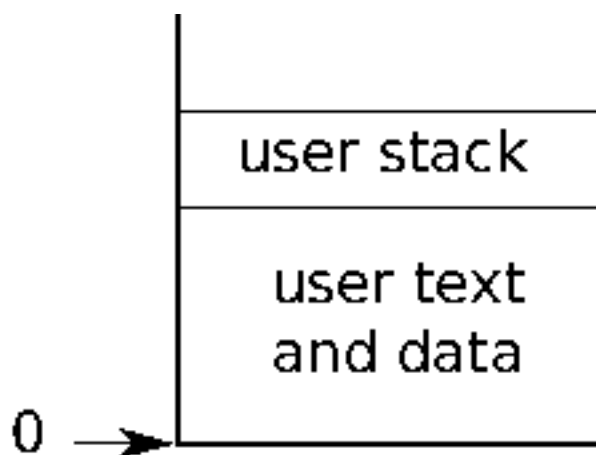


图 2.3：进程虚拟地址空间的布局

地址零开始的进程用户内存。指令排在最前面，其次是全局变量，然后是栈，最后是"堆"区域（用于 `malloc`），进程可以按需扩展该区域。有若干因素限制了进程地址空间的最大大小：RISC-V 上的指针宽度为 64 位；硬件在页表中查找虚拟地址时仅使用低 39 位；而 xv6 只使用了这 39 位中的 38 位。因此，最大地址为 $2^{38} - 1 = 0x3ffffffffff$ ，即

`MAXVA` (`kernel/riscv.h:348`)。在地址空间顶部，xv6 保留了一页用于蹦床（trampoline），以及一页用于映射进程的陷阱帧（trapframe）以切换到内核，我们将在第 4 章中加以说明。xv6 内核为每个进程维护许多状态信息，并将其汇集到一个 `struct proc` (`kernel/proc.h:86`) 中。进程最重要的内核状态包括其页表、内核

栈以及运行状态。我们将使用 `p->xxx` 的记法来引用 `proc` 结构体的成员；

例如，`p->pagetable` 是指向进程页表的指针。

每个进程都有一个执行线程（简称线程），用于执行进程的指令。线程可以被挂起，之后再恢复执行。为了在进程之间透明地切换，内核挂起当前正在运行的线程，并恢复另一个进程的线程。线程的大部分状态（局部变量、函数调用返回地址）存储在线程的栈上。

每个进程有两个栈：一个用户栈和一个内核栈（`p->kstack`）。当进程

执行用户指令时，只有用户栈在使用中，内核栈为空。当进程进入内核（用于系统调用或中断）时，内核代码在进程的内核栈上执行；当进程处于内核中时，其用户栈仍保存着数据，但并未被主动使用。进程的线程在主动使用用户栈和内核栈之间交替切换。内核栈是独立的（并受到保护，用户代码无法访问），因此即使进程破坏了其用户栈，内核仍能正常执行。进程可以通过执行 RISC-V `ecall` 指令来发起系统调用。该指令提升硬件特权级别，并将程序计数器更改为内核定义的入口点。入口点处的代码切换到内核栈，并执行实现系统调用的内核指令。系统调用完成后，内核切换回用户

栈，并通过调用 `sret` 指令返回用户空间，该指令降低硬件特权级别，并在系统调用指令之后继续执行用户指令。进程的线程可以在内核中"阻塞"以等待 I/O，并在 I/O 完成后从中断处恢复执行。

`p->state` 指示进程是否已分配、准备运行、正在运行、等待 I/O 或

正在退出。

`p->pagetable` 以 RISC-V 硬件所期望的格式保存进程的页表。`xv6` 使得分页硬件在用户空间中执行该

进程时使用进程的 `p->pagetable`。进程的页表也充当分配用于存储进程内存的物理页地址的记录。

2.6 代码：启动 xv6 与第一个进程

为了让 `xv6` 更加具体，我们将概述内核如何启动并运行第一个进程。后续章节将更详细地描述本概述中涉及的各种机制。当 RISC-V

计算机上电时，它会完成自身初始化并运行存储在只读内存中的引导加载程序（boot loader）。引导加载程序将 `xv6` 内核加载到内存中。随后，CPU 在机器模式（machine mode）下从 `_entry`（`kernel/entry.S:6`）处开始执行 `xv6`。RISC-V

启动时分页硬件处于禁用状态：虚拟地址直接映射到物理地址。加载程序将 `xv6` 内核加载到物理地址 `0x80000000` 处。之所以将内核放置在 `0x80000000` 而非 `0x0`，是因为地址范围 `0x0:0x80000000`

包含 I/O 设备。`_entry` 处的指令建立了一个栈，使得 `xv6` 能够运行 C 代码。`Xv6` 在文件

`start.c`（`kernel/start.c:11`）中为初始栈 `stack0` 声明了空间。`_entry` 处的代码将栈指针寄存器 `sp` 加载为 `stack0+4096`，即栈顶地址，这是因为 RISC-V 上的栈向下增长。内核建立栈之后，`_entry` 调用位于 `start`（`kernel/start.c:21`）处的 C 代码。函数 `start`

执行一些仅在机器模式下才被允许的配置，然后切换到监管者模式（supervisor mode）。为了进入监管者模式，RISC-V 提供了 `mret`

指令。该指令最常用于从监管者模式对机器模式的先前调用中返回。`start`

并非从此类调用中返回，而是将状态设置为仿佛曾经发生过这样一次调用：它在寄存器 `mstatus` 中将先前的特权模式设置为监管者模式，通过将 `main` 的地址写入寄存器 `mepc` 来将

```
return address to main by writing main's address into the register mepc, disables virtual address
```

通过向页表寄存器 `satp` 写入 0 来禁用监管者模式下的虚拟地址转换，并将所有中断和异常委托给监管者模式。在跳转到监管者模式之前，`start`

还执行了最后一项任务：对时钟芯片进行编程以产生定时器中断。完成上述准备工作后，`start`

通过调用 `mret` “返回”到监管者模式。这使得程序计数器跳转到

`main`（`kernel/main.c:11`）。`main`（`kernel/main.c:11`）初始化若干设备和子系统后，通过调用

`userinit`（`kernel/proc.c:212`）创建第一个进程。第一个进程执行一段用 RISC-V 汇编编写的小程序 `initcode.S`（`user/initcode.S:1`），该程序通过调用 `exec` 系统调用重新进入内核。如第 1 章所述，`exec`

用一个新程序（在本例中为 `/init`）替换当前进程的内存和寄存器。内核完成 `exec` 后，返回到 `/init` 进程的用户空间。`Init`（`user/init.c:15`）在需要时创建一个新的控制台设备文件，然后将其作为文件描述符 0、1 和 2 打开。随后，它在控制台上启动一个 shell。系统启动完成。

2.7 现实世界

在现实世界中，单体内核和微内核都有广泛应用。许多 Unix 内核是单体内核。例如，Linux 拥有单体内核，尽管某些操作系统功能以用户级服务器的形式运行（例如窗口系统）。L4、Minix 和 QNX 等内核被组织为带有服务器的微内核，并在嵌入式领域得到了广泛部署。大多数操作系统都采用了进程的概念，且大多数进程与 xv6 的进程十分相似。然而，现代操作系统支持在一个进程内运行多个线程，以允许单个进程利用多个 CPU。在进程中支持多线程涉及大量 xv6 所不具备的机制，包括潜在的接口变更（例如 Linux 的 `clone`，一种 `fork` 的变体），用于控制进程中各线程共享哪些资源。

2.8 练习

1. 你可以使用 `gdb` 来观察第一次内核到用户态的转换。运行 `make qemu-gdb`。在另一个窗口中，在同一目录下，运行 `gdb`。输入 `gdb` 命令 `break *0x3fffffff10e`，这会在内核中跳转到用户空间的 `sret` 指令处设置一个断点。输入 `gdb` 命令 `continue`。`gdb` 应该会在断点处停下来，即将执行 `sret`。输入 `stepi`。`gdb` 现在应该显示它正在地址 `0x0` 处执行，这是用户空间中 `initcode.S` 的起始位置。

第3章：页表

页表是操作系统为每个进程提供其独立私有地址空间和内存的机制。页表决定了内存地址的含义，以及哪些物理内存区域可以被访问。页表使 xv6 能够隔离不同进程的地址空间，并将它们多路复用到单一物理内存上。页表还提供了一层间接层，使 xv6 得以实现一些技巧：将同一段内存（蹦床页）映射到多个地址空间中，以及用未映射页来保护内核栈和用户栈。本章其余部分将介绍 RISC-V 硬件所提供的页表，以及 xv6 如何使用它们。

3.1 分页硬件

回顾一下，RISC-V 指令（包括用户态和内核态）操作的是虚拟地址。机器的 RAM，即物理内存，通过物理地址进行索引。RISC-V 分页硬件通过将每个虚拟地址映射到一个物理地址，将这两种地址连接起来。xv6 运行在 Sv39 RISC-V 上，这意味着 64 位虚拟地址中只有低 39 位被使用；高 25 位不被使用。在 Sv39 配置下，RISC-V 页表在逻辑上是一个包含 2^{27} (134,217,728) 个页表项 (PTE) 的数组。每个 PTE 包含一个 44 位的物理页号 (PPN) 和若干标志位。分页硬件使用 39 位中的高 27 位来索引页表以找到一个 PTE，从而将虚拟地址转换为 56 位物理地址，其中高 44 位来自 PTE 中的 PPN，低 12 位直接从原始虚拟地址复制而来。图 3.1 展示了这一过程，其中页表以 PTE 简单数组的逻辑视图呈现（更完整的说明见图 3.2）。页表使操作系统能够以 4096 (2^{12}) 字节对齐块的粒度控制虚拟地址到物理地址的转换。这样的块称为一个页。在 Sv39 RISC-V 中，虚拟地址的高 25 位不用于转换；未来 RISC-V 可能会使用这些位来定义更多层级的转换。物理地址也预留了增长空间：PTE 格式中为物理页号额外保留了 10 位的扩展空间。

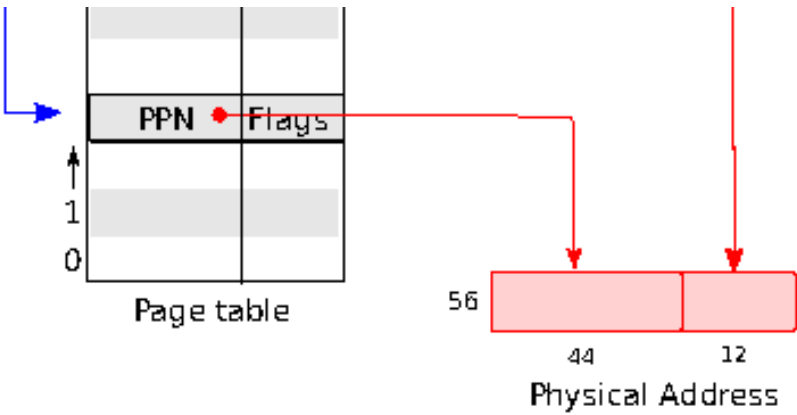


图 3.1：RISC-V 虚拟地址与物理地址，以及简化的逻辑页表。

如图 3.2 所示，实际的地址转换分三步完成。页表以三级树的形式存储在物理内存中。树的根节点是一个 4096 字节的页表页，包含 512 个 PTE，这些 PTE

存储着树的下一级页表页的物理地址。下一级的每个页表页同样包含 512 个 PTE，对应树的最后一级。分页硬件使用 27 位中的高 9 位在根页表页中选择一个 PTE，使用中间 9 位在下一级页表页中选择一个 PTE，使用低 9 位选择最终的 PTE。如果转换某个地址所需的三个 PTE 中有任何一个不存在，分页硬件就会触发一个缺页异常，由内核来处理该异常（见第 4 章）。这种三级结构使得页表在大范围虚拟地址没有映射的常见情况下，可以省略整个页表页。每个 PTE 包含标志位，用于告知分页硬件关联的虚拟地址允许如何被使用。PTE_V 表示该 PTE 是否存在：若未设置，对该页的引用将引发异常（即不被允许）。PTE_R 控制指令是否允许读取该页。PTE_W 控制指令是否允许写入该页。PTE_X 控制 CPU 是否可以将该页的内容解释为指令并执行。PTE_U 控制用户模式下的指令是否允许访问该页；若 PTE_U 未设置，该 PTE 只能在超级用户模式下使用。图 3.2 展示了整个工作机制。标志位及所有其他与分页硬件相关的结构定义在 (kernel/riscv.h) 中。为了让硬件使用某个页表，内核必须将根页表页的物理地址写入 satp 寄存器。每个 CPU 都有自己的 satp。CPU 将使用其 satp 所指向的页表来转换后续指令生成的所有地址。每个 CPU 拥有独立的 satp，使得不同的 CPU 可以运行不同的进程，每个进程都有由其自身页表描述的私有地址空间。以下对几个术语做简要说明。物理内存指 DRAM 中的存储单元。物理内存中的每个字节都有一个地址，称为物理地址。指令只使用虚拟地址，分页硬件将虚拟地址转换为物理地址，然后发送给 DRAM 硬件进行读

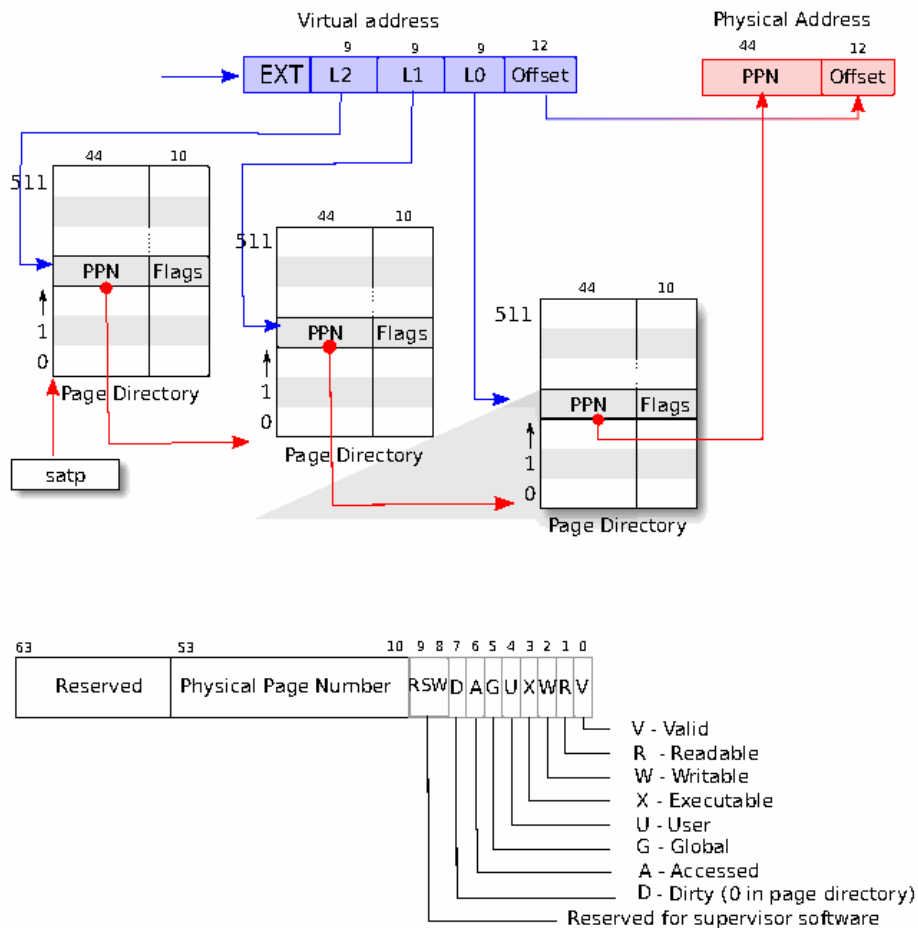


图 3.2：RISC-V 地址转换详情。

或写操作。与物理内存和虚拟地址不同，虚拟内存并不是一个物理实体，而是指内核为管理物理内存和虚拟地址所提供的一系列抽象和机制的总称。

3.2 内核地址空间

Xv6 为每个进程维护一张页表，描述该进程的用户地址空间，同时维护一张描述内核地址空间的单独页表。内核配置其地址空间的布局，以便通过可预测的虚拟地址访问物理内存和各种硬件资源。图 3.3 展示了该布局如何将内核虚拟地址映射到物理地址。文件 (`kernel/memlayout.h`) 声明了 xv6 内核内存布局所用的常量。QEMU 模拟的计算机包含从物理地址 0x80000000 开始、至少延伸到 0x86400000 的 RAM（物理内存），xv6 将该上限称为 PHYSTOP。QEMU 模拟环境还包含磁盘接口等 I/O 设备。QEMU 以内存映射控制寄存器的形式向软件暴露设备接口，这些寄存器位于物理地址空间中 0x80000000 以下的区域。内核可通过读写这些特殊物理地址与设备交互；此类读写操作与设备硬件通信，而非与 RAM 通信。第 4 章将介绍 xv6 如何与设备交互。内核通过"直接映射"访问 RAM 和内存映射设备寄存器，即将资源映射到与物理地址相等的虚拟地址。例如，

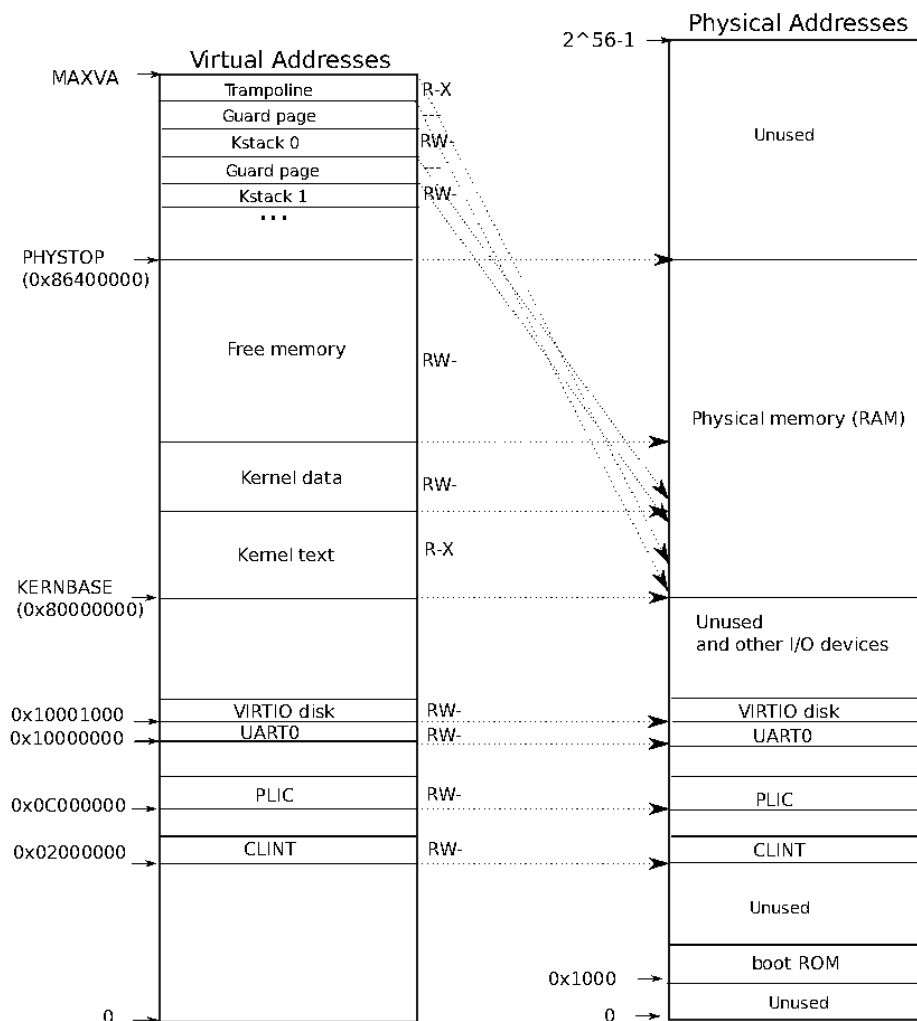


图 3.3：左侧为 xv6 的内核地址空间。RWX 分别表示 PTE 的读、写和执行权限。右侧为 xv6 预期所见的 RISC-V 物理地址空间。

内核本身在虚拟地址空间和物理内存中均位于 `KERNBASE=0x80000000`

处。直接映射简化了内核读写物理内存的代码。例如，当 `fork`

为子进程分配用户内存时，分配器返回该内存的物理地址；`fork` 在将父进程的用户内存复制到子进程时，直接将该地址作为虚拟地址使用。有少数几个内核虚拟地址并非直接映射：

- 蹦床页（trampoline page）。它被映射在虚拟地址空间的顶部；用户页表具有相同的映射。第 4 章将讨论蹦床页的作用，但在此我们可以看到页表的一个有趣用例：一个物理页（存放蹦床代码）在内核的虚拟地址空间中被映射了两次：一次位于虚拟地址空间的顶部，一次通过直接映射。
- 内核栈页（kernel stack pages）。每个进程都有自己的内核栈，它被映射在高地址处，以便 xv6 可以在其下方留出一个未映射的守护页（guard page）。守护页的 PTE 是无效的（即未设置 `PTE_V`），因此如果内核发生内核栈溢出，将很可能触发一个异常并导致内核

panic。若没有守护页，溢出的栈将覆盖其他内核内存，造成错误的运行结果。相比之下，panic 崩溃是更可取的结果。

虽然内核通过高地址映射使用其栈，但这些栈也可通过直接映射地址访问。另一种设计方案可以只使用直接映射，并通过直接映射地址使用栈。然而，在该方案中，提供守护页需要取消映射那些原本指向物理内存的虚拟地址，这将使该物理内存难以被利用。内核以 PTE_R 和 PTE_X 权限映射蹦床页和内核代码段（kernel text）所对应的页，内核从这些页中读取并执行指令。内核以 PTE_R 和 PTE_W 权限映射其他页，以便读写这些页中的内存。守护页的映射是无效的。

3.3 代码：创建地址空间

大多数 xv6 中用于操作地址空间和页表的代码位于 `vm.c`（`kernel/vm.c:1`）。核心数据结构是 `pagetable_t`，它实际上是一个指向 RISC-V 根页表页的指针；`pagetable_t` 可以是内核页表，也可以是某个进程的页表。核心函数是 `walk`（用于查找虚拟地址对应的 PTE）和 `mappages`（用于为新映射安装 PTE）。以 `kvm` 开头的函数操作内核页表；以 `uvm` 开头的函数操作用户页表；其他函数两者均可使用。`copyout` 和 `copyin` 用于向系统调用参数所提供的用户虚拟地址写入或从中读取数据；它们位于 `vm.c` 中，因为需要显式地转换这些地址以找到对应的物理内存。

在启动序列的早期，`main` 调用 `kvminit`（`kernel/vm.c:22`）来创建内核的页表。此调用发生在 xv6 启用 RISC-V 分页之前，因此地址直接指向物理内存。`kvminit` 首先分配一页物理内存来存放根页表页，然后调用 `kvmmap` 安装内核所需的映射，包括内核的指令和数据、直到 `PHYSTOP` 的物理内存，以及实际上是设备的内存范围。

`kvmmap`（`kernel/vm.c:118`）调用 `mappages`（`kernel/vm.c:149`），后者将一段虚拟地址范围到对应物理地址范围的映射安装到页表中。它对范围内的每个虚拟地址逐页分别处理。对于每个待映射的虚拟地址，`mappages` 调用 `walk` 来查找该地址对应 PTE 的地址，然后初始化该 PTE，使其保存相关的物理页号、所需权限（PTE_W、PTE_X 和/或 PTE_R），以及用于将 PTE 标记为有效的 PTE_V（`kernel/vm.c:161`）。

`walk`（`kernel/vm.c:72`）模拟 RISC-V 分页硬件查找虚拟地址对应 PTE 的过程（见图 3.2）。`walk` 每次取 9 位，逐级遍历三级页表。它使用每级虚拟地址中的 9 位来查找下一级页表或最终页面的 PTE（`kernel/vm.c:78`）。如果 PTE 无效，说明所需页面尚未分配；若设置了 `alloc` 参数，`walk` 会分配一个新的页表页并将其物理地址写入该 PTE。最终返回树中最底层 PTE 的地址（`kernel/vm.c:88`）。

上述代码依赖于物理内存被直接映射到内核虚拟地址空间这一前提。例如，当 `walk` 逐级向下遍历页表时，它从 PTE 中取出下一级页表的（物理）地址（`kernel/vm.c:80`），然后将该地址作为虚拟地址来获取下一级的 PTE（`kernel/vm.c:78`）。

`main` 调用 `kvminithart` (`kernel/vm.c:53`) 来安装内核页表。它将根页表页的物理地址写入寄存器 `satp`。此后 CPU 将使用内核页表进行地址转换。由于内核使用恒等映射，下一条指令的虚拟地址将映射到正确的物理内存地址。

从 `main` 调用的 `procinit` (`kernel/proc.c:26`) 为每个进程分配一个内核栈。它将每个栈映射到由 `KSTACK` 生成的虚拟地址处，从而为无效的栈保护页留出空间。`kvmmap` 将映射 PTE 添加到内核页表，对 `kvminithart` 的调用则将内核页表重新加载到 `satp`，使硬件感知新的 PTE。

每个 RISC-V CPU 在快表 (Translation Look-aside Buffer, TLB) 中缓存页表项，当 `xv6` 修改页表时，必须通知 CPU 使相应的已缓存 TLB 项失效。否则，TLB 可能在之后的某个时刻使用旧的缓存映射，指向一个已被分配给另一进程的物理页，从而导致某个进程可能篡改另一个进程的内存。RISC-V 提供了 `sfence.vma` 指令来刷新当前 CPU 的 TLB。`xv6` 在 `kvminithart` 重新加载 `satp` 寄存器之后执行 `sfence.vma`，并在切换到用户页表、返回用户空间之前的蹦床代码 (trampoline code) 中执行该指令 (`kernel/trampoline.S:79`)。

3.4 物理内存分配

内核必须在运行时为页表、用户内存、内核栈和管道缓冲区分配和释放物理内存。`xv6` 使用内核末尾到 `PHYSTOP` 之间的物理内存进行运行时分配。它每次以整页 (4096 字节) 为单位进行分配和释放。它通过在页面自身中穿插一个链表来跟踪哪些页面是空闲的。分配操作即从链表中移除一个页面；释放操作即将被释放的页面添加到链表中。

3.5 代码：物理内存分配器

分配器位于 `kalloc.c` (`kernel/kalloc.c:1`)。分配器的数据结构是一个可用于分配的物理内存页的空闲链表。每个空闲页的链表元素是一个

```
struct run (kernel/kalloc.c:17)。那么分配器从哪里获取内存来存储该数据结
```

构？它将每个空闲页的 `run` 结构存储在空闲页本身中，因为那里没有存储任何其他内容。空闲链表由一个自旋锁保护 (`kernel/kalloc.c:21-24`)。链表和锁被

封装在一个 `struct` 中，以明确表示该锁保护结构体中的字段。暂时忽略锁以及对 `acquire` 和 `release` 的调用；第 6 章将详细讨论锁机制。函数 `main` 调用 `kinit` 来初始化分配器 (`kernel/kalloc.c:27`)。`kinit` 将空闲链表初始化为包含内核末尾到 `PHYSTOP` 之间的每一页。`xv6`

本应通过解析硬件提供的配置信息来确定可用物理内存的大小，但 `xv6` 假设机器拥有 128 兆字节的 RAM。`kinit` 调用 `freerange`，通过逐页调用 `kfree` 将内存添加到空闲链表中。一个 PTE 只能引用在 4096 字节边界上对齐的物理地址 (即 4096 的倍数)，因此 `freerange` 使用 `PGROUNDUP` 来确保它只释放对齐的物理地址。分配器初始时没有任何内存，这些对 `kfree`

的调用为其提供了一些可管理的内存。分配器有时将地址视为整数以对其执行算术运算（例如，在 `freerange` 中遍历所有页），有时又将地址用作指针来读写内存（例如，操作存储在每页中的 `run` 结构）；这种对地址的双重使用是分配器代码中充满 C 类型转换的主要原因。另一个原因是，释放和分配内存本质上会改变内存的类型。函数 `kfree`（`kernel/kalloc.c:47`）首先将被释放内存的每个字节设置为值 1。这将导致在释放后仍使用该内存的代码（即使用“悬空引用”的代码）读取到垃圾数据，而非原有的有效内容；这样做有望使此类代码更快地暴露错误。然后 `kfree` 将该页前插到空闲链表：它将 `pa` 转换为指向 `struct run` 的指针，将空闲链表原来的起始位置记录在 `r->next` 中，并将空闲链表设置为 `r`。`kalloc` 则移除并返回

空闲链表中的第一个元素。

3.6 进程地址空间

每个进程都有独立的页表，当 `xv6` 在进程之间切换时，也会切换页表。如图 2.3 所示，进程的用户内存从虚拟地址零开始，最高可增长至 `MAXVA`（`kernel/riscv.h:348`），理论上允许进程寻址 256 吉字节的内存。当进程向 `xv6` 请求更多用户内存时，`xv6` 首先使用 `kalloc` 分配物理页，然后在进程的页表中添加指向新物理页的 PTE。`xv6` 会在这些 PTE 中设置 `PTE_W`、`PTE_X`、`PTE_R`、`PTE_U` 和 `PTE_V` 标志位。大多数进程并不会使用整个用户地址空间；`xv6` 对未使用的 PTE 保持 `PTE_V` 清零。

这里我们可以看到页表用法的几个典型示例。第一，不同进程的页表将用户地址映射到不同的物理内存页，因此每个进程都拥有私有的用户内存。第二，每个进程看到的内存是从零开始的连续虚拟地址，而进程的物理内存可以是非连续的。第三，内核在用户地址空间的顶部映射了一个包含 trampoline 代码的页，因此同一个物理内存页出现在所有地址空间中。

图 3.4 更详细地展示了 `xv6` 中一个正在执行的进程的用户内存布局。栈只有一个页，图中显示的是由 `exec` 创建时的初始内容。包含命令行参数的字符串以及指向它们的指针数组位于栈的最顶端。紧接其下是一些值，使得程序能够从 `main` 开始执行，就如同函数

```
MAXVA trampoline trapframe argument 0 ... heap argument N 0 nul-terminated string address of argument 0
argv[argc] ... address of argument N argv[0] PAGESIZE stack address of address of argv argument of main guard page
argument 0 argc argc argument of main data 0xFFFFFFFF return PC for main
```

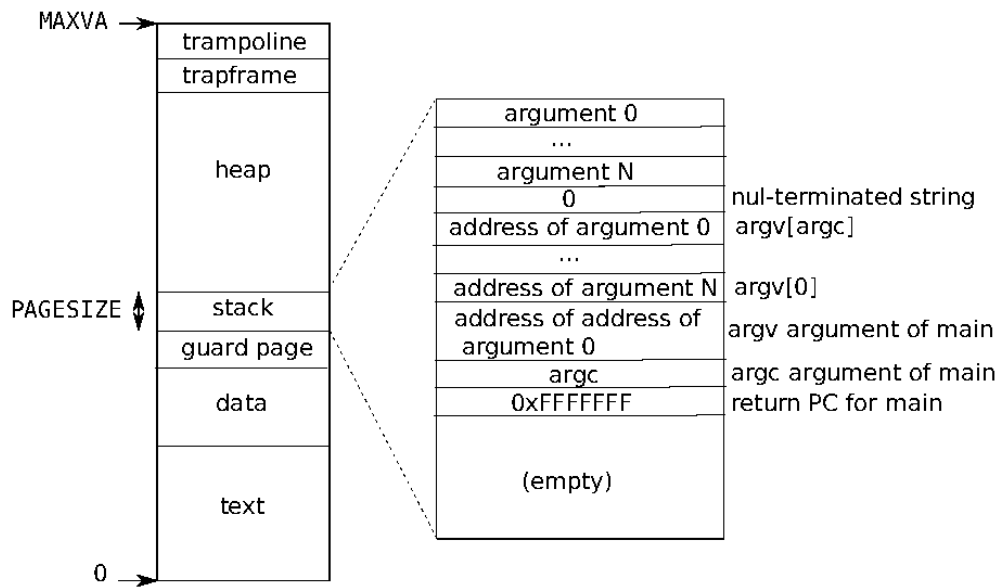


图 3.4：进程的用户地址空间及其初始栈。

`main(argc, argv)` 刚刚被调用一样。为了检测用户栈溢出已分配的栈内存，xv6 在栈的正下方放置了一个无效的保护页（guard page）。如果用户栈发生溢出，进程试图访问栈以下的地址，由于该映射无效，硬件将产生一个缺页异常（page-fault exception）。而现实世界中的操作系统在栈溢出时，可能会自动为用户栈分配更多内存。

3.7 代码：sbrk

`sbrk` 是进程用于缩减或扩展其内存的系统调用。该系统调用由函数 `growproc`（kernel/proc.c:239）实现。`growproc` 根据 `n` 是正数还是负数，分别调用 `uvmalloc` 或 `uvmdealloc`。`uvmalloc`（kernel/vm.c:229）通过 `kalloc` 分配物理内存，并通过 `mappages` 向用户页表添加页表项（PTE）。`uvmdealloc` 调用 `uvmunmap`（kernel/vm.c:174），后者使用 `walk` 查找页表项，并使用 `kfree` 释放它们所指向的物理内存。xv6 使用进程的页表不仅是为了告知硬件如何映射用户虚拟地址，也将其作为记录哪些物理内存页已分配给该进程的唯一依据。这正是释放用户内存（在 `uvmunmap` 中）需要检查用户页表的原因。

3.8 代码：exec

`exec` 是用于创建地址空间用户部分的系统调用。它从存储在文件系统文件中的文件初始化地址空间的用户部分。`Exec`（kernel/exec.c:13）使用 `namei`（kernel/exec.c:26）打开指定的二进制路径，`namei` 将在第 8 章中详细说明。随后，它读取 ELF 头。xv6 应用程序使用广泛采用的 ELF 格式描述，该格式定义于（kernel/elf.h）。一个 ELF 二进制文件由一个 ELF 头 `struct elfhdr`（kernel/elf.h:6），以及紧随其后的一系列程序节头 `struct`

`proghdr` (`kernel/elf.h:25`) 组成。每个 `proghdr` 描述了应用程序中必须加载到内存的一个节；`xv6` 程序只有一个程序节头，但其他系统可能为指令和数据设置独立的节。第一步是快速检查该文件是否可能包含一个 ELF 二进制文件。ELF 二进制文件以四字节“魔数”`0x7F`、`'E'`、`'L'`、`'F'`，即 `ELF_MAGIC` (`kernel/elf.h:3`) 开头。如果 ELF 头具有正确的魔数，`exec` 则假定该二进制文件格式合法。`Exec` 通过 `proc_pagetable` (`kernel/exec.c:38`) 分配一个不含用户映射的新页表，通过 `uvmalloc` (`kernel/exec.c:52`) 为每个 ELF 段分配内存，并通过 `loadseg` (`kernel/exec.c:10`) 将每个段加载到内存中。`loadseg` 使用 `walkaddr` 查找已分配内存的物理地址，以便将 ELF 段的每一页写入其中，并使用 `readi` 从文件中读取数据。由 `exec` 创建的第一个用户程序 `/init` 的程序节头如下所示：

```
# objdump -p _init user/_init:      file format elf64-littleriscv

Program Header:
  LOAD off  0x00000000000000b0 vaddr 0x0000000000000000 paddr 0x0000000000000000 align 2**3
           filesz 0x0000000000000840 memsz 0x0000000000000858 flags rwx
  STACK off 0x0000000000000000 vaddr 0x0000000000000000 paddr 0x0000000000000000 align 2**4
           filesz 0x0000000000000000 memsz 0x0000000000000000 flags rw-
```

程序节头中的 `filesz` 可能小于 `memsz`，这表明两者之间的差值部分应填充为零（用于 C 全局变量），而非从文件中读取。对于 `/init`，`filesz` 为 2112 字节，`memsz` 为 2136 字节，因此 `uvmalloc` 会分配足够容纳 2136 字节的物理内存，但仅从文件 `/init` 中读取 2112 字节。

接下来，`exec` 分配并初始化用户栈。它仅分配一个栈页。`exec` 将参数字符串逐一复制到栈顶，并在 `ustack` 中记录指向它们的指针。它在将传递给 `main` 的 `argv` 列表末尾放置一个空指针。`ustack` 中的前三个条目分别是伪造的返回程序计数器、`argc` 和 `argv` 指针。`exec` 在栈页的正下方放置一个不可访问的页，以便试图使用超过一页栈空间的程序触发错误。这个不可访问的页同样使 `exec` 能够处理过大的参数；在这种情况下，`exec` 用于将参数复制到栈上的 `copyout` (`kernel/vm.c:355`) 函数会发现目标页不可访问，并返回 `-1`。

在准备新内存映像的过程中，如果 `exec` 检测到错误（例如无效的程序段），它会跳转到标签 `bad`，释放新映像，并返回 `-1`。`exec` 必须等到确认系统调用将会成功后，才能释放旧映像：若旧映像已被销毁，系统调用将无法向其返回 `-1`。`exec` 中的错误情况仅发生在映像创建阶段。一旦映像创建完成，`exec` 便可提交新页表 (`kernel/exec.c:113`) 并释放旧页表 (`kernel/exec.c:117`)。

`exec` 按照 ELF 文件中指定的地址，将 ELF 文件的字节加载到内存中。用户或进程可以在 ELF 文件中填入任意地址。因此，`exec` 存在风险，因为 ELF 文件中的地址可能会无意或有意地指向内核。对于一个疏于防范的内核，后果轻则崩溃，重则内核隔离机制遭到恶意破坏（即安全漏洞利用）。`xv6` 执行了多项检查以规避这些风险。例如，`if(ph.vaddr + ph.memsz < ph.vaddr)` 用于检查求和是否发生 64 位整数溢出。其危险在于，用户可以构造一个 ELF 二进制文件，其中 `ph.vaddr` 指向用户自选的地址，而 `ph.memsz` 足够大，使得求和溢出为 `0x1000`，从而看起来像一个合法值。在旧版本的 `xv6` 中，用户地址空间也包含内核（但在用户模式下不

可读写），用户可以选择一个对应内核内存的地址，从而将 ELF 二进制文件中的数据复制到内核中。在 RISC-V 版本的 xv6 中，这种情况不会发生，因为内核拥有其独立的页表；loadseg 加载到的是进程的页表，而非内核的页表。

内核开发者很容易遗漏关键检查，现实世界中的内核长期以来存在因缺少检查而被用户程序利用以获取内核特权的历史。xv6 很可能没有对提交给内核的用户级数据进行完整的校验，恶意用户程序或许能够借此绕过 xv6 的隔离机制。

3.9 真实世界

与大多数操作系统一样，xv6 使用分页硬件来实现内存保护和映射。大多数操作系统通过将分页与页面错误异常相结合，对分页的使用远比 xv6 复杂，这将在第 4 章中讨论。xv6 的简化之处在于：内核在虚拟地址与物理地址之间使用了直接映射，并假设在地址 0x8000000 处存在物理 RAM，内核期望被加载到该地址。这在 QEMU 上可以正常工作，但在真实硬件上这是个糟糕的设计——真实硬件将 RAM 和设备放置在不可预测的物理地址上，因此（例如）在 0x8000000 处可能根本没有 RAM，而 xv6 却期望在那里存储内核。更严谨的内核设计会利用页表将任意的硬件物理内存布局转换为可预测的内核虚拟地址布局。RISC-V 支持在物理地址层面进行保护，但 xv6 没有使用该特性。在拥有大量内存的机器上，使用 RISC-V 对“超级页”（super pages）的支持可能是合理的。当物理内存较小时，小页面是合适的，以便以细粒度进行分配和换页到磁盘。例如，如果一个程序只使用 8 KB 的内存，却为其分配整个 4 MB 的超级页物理内存，则是一种浪费。在拥有大量 RAM 的机器上，较大的页面更为合适，并且可以减少页表操作的开销。xv6 内核缺少类似 malloc 的分配器来为小对象提供内存，这使得内核无法使用需要动态分配的复杂数据结构。

内存分配是一个长期热门的话题，其核心问题是有限内存的高效使用以及为未知的未来请求做好准备 [7]。如今人们更关注速度而非空间效率。此外，一个更复杂的内核可能会分配许多不同大小的小块，而不是像 xv6 那样只分配 4096 字节的块；真实的内核分配器需要同时处理小型和大型内存分配。

3.10 练习

1. 解析 RISC-V 的设备树，找出计算机拥有的物理内存大小。
2. 编写一个用户程序，通过调用 `sbrk(1)` 将其地址空间增长一个字节。运行该程序，并分别在调用 `sbrk` 之前和之后检查该程序的页表。内核分配了多少空间？新内存对应的 PTE 包含什么内容？
3. 修改 xv6，为内核使用超级页（super pages）。

4. 修改 xv6, 使得当用户程序解引用空指针时会收到一个异常。即修改 xv6, 使虚拟地址 0 不映射到用户程序的地址空间中。

5. Unix 的 `exec` 实现传统上包含对 shell 脚本的特殊处理。如果要执行的文件以文本 `#!` 开头, 则第一行被视为用于解释该文件的程序。例如, 如果调用 `exec` 来运行 `myprog arg1`, 且 `myprog` 的第一行是 `#!/interp`, 则 `exec` 会以命令行 `/interp myprog arg1` 来运行 `/interp`。在 xv6 中实现对该约定的支持。

6. 为内核实现地址空间随机化。

第 4 章：陷阱与系统调用

有三类事件会导致 CPU 暂停普通指令的执行，并强制将控制权转移到处理该事件的特殊代码。第一种情况是系统调用：当用户程序执行 `ecall` 指令，请求内核为其执行某些操作时。第二种情况是异常：某条指令（用户态或内核态）执行了非法操作，例如除以零或使用了无效的虚拟地址。第三种情况是设备中断：当设备发出需要处理的信号时，例如磁盘硬件完成了一次读写请求。

本书使用"陷阱" (trap) 作为这些情况的统称。通常，发生陷阱时正在执行的代码之后需要恢复执行，并且不应感知到发生了任何特殊情况。也就是说，我们通常希望陷阱对调用者是透明的；这对于中断尤为重要，因为被中断的代码通常并不预期中断的发生。标准流程如下：陷阱强制将控制权转入内核；内核保存寄存器和其他状态，以便后续恢复执行；内核执行相应的处理程序代码（例如系统调用实现或设备驱动程序）；内核恢复已保存的状态并从陷阱中返回；原始代码从中断处继续执行。

xv6 内核处理所有陷阱。这对系统调用而言是自然的。对于中断，这样做也合理，因为隔离性要求用户进程不能直接使用设备，而且只有内核才拥有设备处理所需的状态。对于异常同样如此，因为 xv6 对来自用户空间的所有异常的响应方式是终止引发异常的进程。

xv6 的陷阱处理分四个阶段进行：RISC-V CPU 执行的硬件动作、为内核 C 代码做好准备的汇编"向量" (vector)、决定如何处理陷阱的 C 陷阱处理程序，以及系统调用或设备驱动程序的服务例程。尽管三类陷阱的共性表明内核可以用单一代码路径处理所有陷阱，但实际上为三种不同情况分别设置独立的汇编向量和 C 陷阱处理程序更为方便：来自用户空间的陷阱、来自内核空间的陷阱，以及定时器中断。

4.1 RISC-V 陷阱机制

每个 RISC-V CPU 都有一组控制寄存器，内核通过写入这些寄存器来告知 CPU 如何处理陷阱，同时内核也可以读取这些寄存器来获取已发生陷阱的相关信息。RISC-V 文档包含了完整的说明 [1]。riscv.h (kernel/riscv.h:1) 包含了 xv6 所使用的相关定义。以下是最重要寄存器的概览：

- **stvec**：内核将其陷阱处理程序的地址写入此处；RISC-V 跳转到此地址来处理陷阱。
- **sepc**：当陷阱发生时，RISC-V 将程序计数器保存在此处（因为此时 `pc` 会被 `stvec` 的值覆盖）。`sret`（从陷阱返回）指令将 `sepc` 的值复制到 `pc`。内核可以写入 `sepc` 来控制 `sret` 的返回目标。
- **scause**：RISC-V 在此处存放一个数字，用于描述陷阱的原因。

- `sscratch`：内核在此处放置一个值，该值在陷阱处理程序最开始的阶段非常有用。
- `sstatus`：`sstatus` 中的 `SIE` 位控制设备中断是否启用。若内核清除 `SIE`，RISC-V 将推迟设备中断，直到内核重新置位 `SIE`。`SPP` 位指示陷阱来自用户模式还是监督者模式，并控制 `sret` 返回到哪个模式。

上述寄存器与在监督者模式下处理的陷阱相关，在用户模式下无法读写这些寄存器。对于在机器模式下处理的陷阱，存在一套等价的控制寄存器；xv6

仅在处理定时器中断这一特殊情况时才使用它们。多核芯片上的每个 CPU

都拥有各自独立的一套这类寄存器，且在任意给定时刻可能有多个 CPU

同时在处理陷阱。当需要强制触发陷阱时，RISC-V

硬件会对所有陷阱类型（定时器中断除外）执行以下操作：

1. 若陷阱为设备中断，且 `sstatus` 的 `SIE` 位已清零，则不执行以下任何操作。
2. 通过清除 `SIE` 来禁用中断。
3. 将 `pc` 复制到 `sepc`。
4. 将当前模式（用户模式或监督者模式）保存到 `sstatus` 的 `SPP` 位中。
5. 设置 `scause` 以反映陷阱的原因。
6. 将模式切换为监督者模式。
7. 将 `stvec` 复制到 `pc`。
8. 从新的 `pc` 处开始执行。

注意，CPU 不会切换到内核页表，不会切换到内核中的栈，也不会保存除 `pc`

之外的任何寄存器。这些任务必须由内核软件来完成。CPU 在陷阱期间只做最少工作的原因之一，是为了给软件提供灵活性；例如，某些操作系统在某些情况下不需要切换页表，这可以提升性能。你可能会想，CPU 硬件的陷阱处理序列是否可以进一步简化。例如，假设 CPU 不切换程序计数器，那么陷阱就可能在仍执行用户指令的情况下切换到监督者模式。这些用户指令可能会破坏用户/内核的隔离性，例如通过修改 `satp` 寄存器使其指向一个允许访问所有物理内存的页表。因此，CPU 切换到由内核指定的指令地址（即 `stvec`）是至关重要的。

4.2 来自用户空间的陷阱

如果用户程序发起系统调用（`ecall` 指令）、执行了非法操作，或设备产生中断，则在用户空间执行期间可能会发生陷阱。来自用户空间的陷阱的高层路径为：`uservec`（`kernel/trampoline.S:16`），然后是 `usertrap`（`kernel/trap.c:37`）；返回时依次经过 `usertrapret`（`kernel/trap.c:90`）和 `userret`（`kernel/trampoline.S:16`）。来自用户代码的陷阱比来自内核的陷阱更具挑战性，因为 `satp` 指向一个不映射内核的用户页表，而且栈指针可能包含无效甚至恶意的值。由于 RISC-V 硬件在陷阱期间不切换页表，用户页表必须包含对 `uservec` 的映射——即 `stvec` 所指向的陷阱向量指令。`uservec` 必须将 `satp` 切换为指向内核页表；为了在切换后能够继续执行指令，`uservec` 在内核页表中的映射地址必须与在用户页表中相同。Xv6 通过一个包含 `uservec` 的跳板页（trampoline page）来满足这些约束。Xv6 在内核页表和每个用户页表中将跳板页映射到相同的虚拟地址。该虚拟地址为 `TRAMPOLINE`（如图 2.3 和图 3.3 所示）。跳板页的内容定义在 `trampoline.S` 中，且（在执行用户代码时）`stvec` 被设置为 `uservec`（`kernel/trampoline.S:16`）。

当 `uservec` 开始执行时，所有 32 个寄存器都包含被中断代码所拥有的值。但 `uservec` 需要能够修改某些寄存器，以便设置 `satp` 并生成用于保存寄存器的地址。RISC-V 通过 `sscratch` 寄存器提供了帮助。`uservec` 开头的 `csrrw` 指令交换了 `a0` 和 `sscratch` 的内容。此时用户代码的 `a0` 已被保存；`uservec` 拥有一个可以操作的寄存器（`a0`）；而 `a0` 中包含内核此前存入 `sscratch` 的值。

`uservec` 的下一项任务是保存用户寄存器。在进入用户空间之前，内核已将 `sscratch` 设置为指向每个进程的陷阱帧（trapframe），该陷阱帧（除其他内容外）具有保存所有用户寄存器的空间（`kernel/proc.h:44`）。由于 `satp` 仍然指向用户页表，`uservec` 要求陷阱帧必须映射在用户地址空间中。在创建每个进程时，xv6 为该进程的陷阱帧分配一个页面，并安排它始终映射在用户虚拟地址 `TRAPFRAME` 处，该地址位于 `TRAMPOLINE` 的正下方。进程的

```
p->trapframe also points to the trapframe, though at its physical address so the kernel can use it through the kernel page table. Thus after swapping a0 and sscratch, a0 holds a pointer to the current process's trapframe. uservec now saves all user registers there, including the user's a0, read from sscratch.
```

陷阱帧包含指向当前进程内核栈的指针、当前 CPU 的 `hartid`、`usertrap` 的地址以及内核页表的地址。`uservec` 获取这些值，将 `satp` 切换到内核页表，并调用 `usertrap`。

`usertrap` 的任务是确定陷阱的原因、处理陷阱并返回（`kernel/trap.c:37`）。如上所述，它首先修改 `stvec`，使得在内核中发生的陷阱由 `kernelvec` 处理。它保存 `sepc`（已保存的用户程序计数器），原因同样是 `usertrap` 中可能发生进程切换，导致 `sepc` 被覆盖。如果陷阱是系统调用，则由 `syscall` 处理；如果是设备中断，则由 `devintr` 处理；否则为异常，内核将终止发生故障的进程。系统调用路径会将已保存的用户 `pc` 加四，因为在系统调用情况下，RISC-V 将程序指针保留在 `ecall` 指令处。在返回途中，`usertrap` 检查进程是否已被终止或是否应该让出 CPU（如果此次陷阱是定时器中断）。

返回用户空间的第一步是调用 `usertrapret` (`kernel/trap.c:90`)。该函数设置 RISC-V 控制寄存器，为将来来自用户空间的陷阱做准备。这包括将 `stvec` 改为指向 `uservec`、准备 `uservec` 所依赖的陷阱帧字段，以及将 `sepc` 设置为之前保存的用户程序计数器。最后，`usertrapret` 在同时映射于用户页表和内核页表的跳板页上调用 `userret`；这样做的原因是 `userret` 中的汇编代码将切换页表。`usertrapret` 对 `userret` 的调用在 `a0` 中传入指向进程用户页表的指针，在 `a1` 中传入 `TRAPFRAME` (`kernel/trampoline.S:88`)。`userret` 将 `satp` 切换到进程的用户页表。回想一下，用户页表同时映射跳板页和 `TRAPFRAME`，但不映射内核的其他任何内容。同样，跳板页在用户页表和内核页表中映射于相同虚拟地址，这正是允许 `uservec` 在修改 `satp` 后继续执行的原因。`userret` 将陷阱帧中保存的用户 `a0` 复制到 `sscratch`，为稍后与 `TRAPFRAME` 的交换做准备。从此时起，`userret` 能够使用的数据仅限于寄存器内容和陷阱帧的内容。接下来，`userret` 从陷阱帧中恢复已保存的用户寄存器，最后交换 `a0` 和 `sscratch` 以恢复用户的 `a0` 并为下次陷阱保存 `TRAPFRAME`，然后使用 `sret` 返回用户空间。

4.3 代码：调用系统调用

第 2 章以 `initcode.S` 调用 `exec`

系统调用结束 (`user/initcode.S:11`)。让我们来看看用户态的调用是如何一步步到达内核中 `exec` 系统调用实现的。用户代码将 `exec` 的参数放入寄存器 `a0` 和 `a1`，并将系统调用号放入 `a7`。系统调用号与 `syscalls` 数组中的条目相对应，该数组是一个函数指针表 (`kernel/syscall.c:108`)。`ecall` 指令陷入内核，并依次执行 `uservec`、`usertrap`，然后是 `syscall`，正如我们在前文所见。`syscall` (`kernel/syscall.c:133`) 从陷阱帧中保存的 `a7` 中取出系统调用号，并用其作为索引在 `syscalls` 中查找。对于第一个系统调用，`a7` 包含 `SYS_exec` (`kernel/syscall.h:8`)，从而调用系统调用实现函数 `sys_exec`。当系统调用实现函数返回时，`syscall` 将其返回值记录在

```
p->trapframe->a0
```

中。这将使原本在用户空间对 `exec()` 的调用返回该值，因为 RISC-V 的 C 调用约定将返回值放在 `a0` 中。系统调用通常以返回负数表示错误，返回零或正数表示成功。如果系统调用号无效，`syscall` 将打印一条错误信息并返回 `-1`。

4.4 代码：系统调用参数

内核中的系统调用实现需要找到用户代码传递的参数。由于用户代码通过调用系统调用包装函数来发起系统调用，这些参数最初按照 RISC-V C 调用约定存放在寄存器中。内核陷阱代码将用户寄存器保存到当前进程的陷阱帧 (`trap frame`) 中，内核代码可以从中读取这些参数。函数 `argint`、`argaddr` 和 `argfd`

分别以整数、指针或文件描述符的形式从陷阱帧中获取第 n 个系统调用参数。它们都通过调用 `argraw` 来读取相应的已保存用户寄存器 (`kernel/syscall.c:35`)。

某些系统调用以指针作为参数传递，内核必须使用这些指针来读写用户内存。例如，`exec` 系统调用向内核传递一个指针数组，这些指针指向用户空间中的字符串参数。这些指针带来了两个挑战。第一，用户程序可能存在缺陷或恶意行为，可能向内核传递无效指针，或者传递一个意图欺骗内核、使其访问内核内存而非用户内存的指针。第二，`xv6` 内核页表的映射与用户页表的映射并不相同，因此内核不能使用普通指令从用户提供的地址进行加载或存储。

内核实现了一组函数，用于安全地在用户提供的地址与内核之间传输数据。`fetchstr` 就是一个例子 (`kernel/syscall.c:25`)。`exec` 等文件系统调用使用 `fetchstr` 从用户空间获取字符串文件名参数。`fetchstr` 调用 `copyinstr` 来完成实际的工作。`copyinstr` (`kernel/vm.c:406`) 从用户页表 `pagetable` 中的虚拟地址 `srcva` 处，向 `dst` 最多复制 `max` 个字节。它使用 `walkaddr` (内部调用 `walk`) 在软件中遍历页表，以确定 `srcva` 对应的物理地址 `pa0`。由于内核将所有物理 RAM 地址映射到相同的内核虚拟地址，`copyinstr` 可以直接将字符串字节从 `pa0` 复制到 `dst`。`walkaddr` (`kernel/vm.c:95`) 会检查用户提供的虚拟地址是否属于该进程的用户地址空间，从而防止程序欺骗内核读取其他内存。与之类似的函数 `copyout` 则负责将数据从内核复制到用户提供的地址。

4.5 来自内核空间的陷阱

`Xv6` 根据当前执行的是用户代码还是内核代码，以不同的方式配置 CPU 的陷阱寄存器。当内核在 CPU 上执行时，内核将 `stvec` 指向 `kernelvec` 处的汇编代码 (`kernel/kernelvec.S:10`)。由于 `xv6` 此时已处于内核中，`kernelvec` 可以依赖 `satp` 已被设置为内核页表，以及栈指针指向一个有效的内核栈。`kernelvec` 保存所有寄存器，以便被中断的代码最终能够不受干扰地恢复执行。`kernelvec` 将寄存器保存在被中断的内核线程的栈上，这是合理的，因为这些寄存器的值归属于该线程。如果陷阱导致切换到另一个线程，这一点尤为重要——在这种情况下，陷阱实际上会在新线程的栈上返回，而被中断线程的已保存寄存器则安全地留在其自己的栈上。`kernelvec` 在保存寄存器后跳转至 `kerneltrap` (`kernel/trap.c:134`)。

`kerneltrap` 为两类陷阱做好了准备：设备中断和异常。它调用 `devintr` (`kernel/trap.c:177`) 来检测并处理前者。如果陷阱不是设备中断，则必定是异常，而异常一旦发生在 `xv6` 内核中，始终是致命错误；内核会调用 `panic` 并停止执行。如果 `kerneltrap` 是因时钟中断而被调用，并且某个进程的内核线程正在运行（而非调度器线程），`kerneltrap` 会调用 `yield`，以便给其他线程运行的机会。在某个时刻，其中某个线程会让出 CPU，使我们的线程及其 `kerneltrap` 得以再次恢复。第 7 章解释了 `yield` 中发生的事情。

当 `kerneltrap` 的工作完成后，它需要返回到被陷阱打断的代码处。由于 `yield` 可能已经破坏了已保存的 `sepc` 和 `sstatus` 中已保存的先前模式，`kerneltrap`

在启动时会将它们保存下来。此时它恢复这些控制寄存器，并返回到 `kernelvec` (`kernel/kernelvec.S:48`)。`kernelvec` 从栈上弹出已保存的寄存器，并执行 `sret`，该指令将 `sepc` 复制到 `pc`，从而恢复被中断的内核代码。

值得仔细思考的是，如果 `kerneltrap` 因时钟中断而调用了 `yield`，陷阱返回是如何发生的。当某个 CPU 从用户空间进入内核时，`xv6` 会将该 CPU 的 `stvec` 设置为 `kernelvec`；可以在 `usertrap` (`kernel/trap.c:29`) 中看到这一点。存在一段时间窗口，此时内核正在执行，但 `stvec` 仍被设置为 `uservec`，在这段窗口期间，设备中断必须被禁用，这一点至关重要。幸运的是，RISC-V 在开始处理陷阱时总是会禁用中断，而 `xv6` 在设置好 `stvec` 之前不会重新启用中断。

4.6 页错误异常

`Xv6` 对异常的处理相当简单粗暴：如果异常发生在用户空间，内核会杀死出错的进程；如果异常发生在内核中，内核则会 `panic`。真实的操作系统通常会以更复杂有趣的方式来响应异常。举一个例子，许多内核利用页错误来实现写时复制 (COW) `fork`。为了说明写时复制 `fork`，我们先回顾第 3 章中描述的 `xv6` 的 `fork`。`fork` 通过调用 `uvmcopy` (`kernel/vm.c:309`) 为子进程分配物理内存，并将父进程的内存内容复制进去，从而使子进程拥有与父进程相同的内存内容。如果子进程和父进程能够共享父进程的物理内存，效率会更高。然而，这种方式的直接实现并不可行，因为父子进程对共享栈和堆的写操作会相互干扰彼此的执行。借助页错误驱动的写时复制 `fork`，父子进程可以安全地共享物理内存。

当 CPU 无法将虚拟地址转换为物理地址时，CPU 会产生一个页错误异常。RISC-V 定义了三种不同类型的页错误：加载页错误（当 `load` 指令无法转换其虚拟地址时）、存储页错误（当 `store` 指令无法转换其虚拟地址时）以及指令页错误（当某条指令的地址无法转换时）。`scause` 寄存器中的值指示页错误的类型，`stval` 寄存器则包含无法被转换的地址。

COW `fork`

的基本思路是：父子进程初始时共享所有物理页，但将它们映射为只读。这样，当子进程或父进程执行 `store` 指令时，RISC-V CPU 会触发一个页错误异常。内核响应该异常，对包含出错地址的页面进行复制：将一份副本以读/写权限映射到子进程的地址空间，另一份副本以读/写权限映射到父进程的地址空间。更新页表之后，内核在引发错误的指令处恢复出错进程的执行。由于内核已将相关 PTE 更新为允许写入，出错的指令现在将顺利执行而不再触发错误。

这套 COW 方案非常适合 `fork`，因为子进程通常在 `fork` 之后会立即调用 `exec`，用一个新的地址空间替换自己的地址空间。在这种常见情况下，子进程只会经历少量页错误，内核便可避免进行完整的内存复制。此外，COW `fork` 对应用程序是透明的：应用程序无需做任何修改即可从中受益。

页表与页错误的结合，除了 COW `fork`

之外，还开辟了许多其他有趣的应用可能。另一种被广泛使用的特性称为惰性分配 (lazy allocation)，它由两部分组成。第一，当应用程序调用 `sbrk` 时，内核扩大地址空间，但在页表中将新地址标记为无效。第二，当对这些新地址之一发生页错误时，内核分配物理内存并将其映射到页表中。由于

应用程序通常会申请比实际需要更多的内存，惰性分配带来了实质性的性能收益：内核只在应用程序真正使用内存时才进行分配。与 COW fork 类似，内核可以对应用程序完全透明地实现这一特性。

另一种广泛利用页错误的特性是磁盘换页（paging from disk）。如果应用程序需要的内存超过了可用的物理 RAM，内核可以换出（evict）一些页面：将它们写入磁盘等存储设备，并将其 PTE 标记为无效。若应用程序读取或写入一个已被换出的页面，CPU 将触发页错误。内核随后检查出错地址：如果该地址对应的页面在磁盘上，内核会分配一个物理内存页面，将该页面从磁盘读入内存，将 PTE 更新为有效并指向该内存，然后恢复应用程序的执行。为了给新页面腾出空间，内核可能需要换出另一个页面。这一特性无需对应用程序做任何修改，并且在应用程序具有访问局部性（即在任意时刻只使用其内存的一个子集）的情况下表现良好。

其他将分页与页错误异常相结合的特性还包括：自动扩展栈（automatically extending stacks）和内存映射文件（memory-mapped files）。

4.7 真实世界

如果将内核内存映射到每个进程的用户页表中（并设置适当的 PTE 权限标志），则可以消除对特殊 trampoline 页的需求。这样也可以省去从用户空间陷入内核时切换页表的开销。进而，内核中系统调用的实现可以利用当前进程的用户内存已被映射这一特性，使内核代码能够直接解引用用户指针。许多操作系统采用了这些思路来提升效率。Xv6 之所以刻意回避这些做法，是为了降低内核中因无意间使用用户指针而引入安全漏洞的风险，同时也避免了为确保用户虚拟地址与内核虚拟地址互不重叠所需的额外复杂性。

4.8 练习

1. 函数 `copyin` 和 `copyinstr` 在软件中遍历用户页表。设置内核页表，使内核中映射了用户程序，从而 `copyin` 和 `copyinstr` 可以使用 `memcpy` 将系统调用参数复制到内核空间，并依赖硬件完成页表遍历。

2. 实现懒惰内存分配

3. 实现 COW fork

第 5 章：中断与设备驱动程序

驱动程序是操作系统中管理特定设备的代码：它负责配置设备硬件、指示设备执行操作、处理由此产生的中断，以及与可能正在等待设备 I/O 的进程进行交互。驱动程序代码可能较为复杂，因为驱动程序与其所管理的设备是并发执行的。此外，驱动程序必须理解设备的硬件接口，而这些接口往往复杂且文档匮乏。需要操作系统关注的设备通常可以配置为产生中断，中断是陷阱的一种类型。内核的陷阱处理代码能够识别设备何时触发了中断，并调用驱动程序的中断处理程序；在 xv6 中，这一分发过程发生在 `devintr` (`kernel/trap.c:177`) 中。许多设备驱动程序在两个上下文中执行代码：一个是运行在进程内核线程中的上半部，另一个是在中断时执行的下半部。上半部通过系统调用（如 `read` 和 `write`）被调用，这些系统调用希望设备执行 I/O 操作。该代码可能会请求硬件启动某个操作（例如，请求磁盘读取一个块）；然后等待操作完成。最终，设备完成操作并触发中断。驱动程序的中断处理程序作为下半部，负责判断哪个操作已完成，在适当时唤醒等待中的进程，并通知硬件开始处理下一个待处理的操作。

5.1 代码：控制台输入

控制台驱动程序 (`console.c`) 是驱动程序结构的一个简单示例。控制台驱动程序通过连接到 RISC-V 的 UART 串行端口硬件，接受用户输入的字符。控制台驱动程序每次累积一行输入，处理特殊输入字符，例如退格和 `control-u`。用户进程（例如 `shell`）使用 `read` 系统调用从控制台获取输入行。当你在 QEMU 中向 xv6 输入内容时，你的按键通过 QEMU 模拟的 UART 硬件传递给 xv6。驱动程序所使用的 UART 硬件是 QEMU 模拟的 16550 芯片 [11]。在真实计算机上，16550 用于管理连接到终端或其他计算机的 RS232 串行链路。在运行 QEMU 时，它连接到你的键盘和显示器。

UART 硬件以一组内存映射控制寄存器的形式呈现给软件。也就是说，RISC-V 硬件将某些物理地址连接到 UART 设备，因此对这些地址的加载和存储操作会与设备硬件交互，而不是与 RAM 交互。UART 的内存映射地址从 `0x10000000` 开始，即

`UART0` (`kernel/memlayout.h:21`)。UART

有若干控制寄存器，每个寄存器宽度为一个字节。它们相对于 `UART0`

的偏移量定义在 (`kernel/uart.c:22`) 中。例如，`LSR`

寄存器包含若干位，用于指示是否有输入字符等待软件读取。这些字符（如果有）可从 `RHR`

寄存器读取。每次读取一个字符后，UART 硬件会将其从内部等待字符的 FIFO 中删除，并在 FIFO

为空时清除 `LSR` 中的“就绪”位。UART 的发送硬件与接收硬件基本相互独立；如果软件向 `THR`

写入一个字节，UART 就会发送该字节。

Xv6 的 `main` 函数调用 `consoleinit` (`kernel/console.c:184`) 来初始化 UART 硬件。该代码将 UART 配置为：每次接收到一个输入字节时产生接收中断，每次完成发送一个输出字节时产生发送完成中断 (`kernel/uart.c:53`)。

xv6 的 shell 通过 `init.c` 打开的文件描述符 (`user/init.c:19`) 从控制台读取数据。对 `read` 系统调用的调用会经过内核传递到 `consoleread` (`kernel/console.c:82`)。`consoleread` 等待输入到达 (通过中断) 并被缓冲到 `cons.buf` 中, 将输入复制到用户空间, 并在整行输入到达后返回给用户进程。如果用户尚未输入完整的一行, 任何正在读取的进程都将在 `sleep` 调用中等待 (`kernel/console.c:98`) (第 7 章详细解释了 `sleep` 的细节)。

当用户输入一个字符时, UART 硬件请求 RISC-V 触发一个中断, 从而激活 xv6 的陷阱处理程序。陷阱处理程序调用 `devintr` (`kernel/trap.c:177`), 它查看 RISC-V 的 `scause` 寄存器, 以确认该中断来自外部设备。然后它向一个名为 PLIC [1] 的硬件单元查询是哪个设备触发了中断 (`kernel/trap.c:186`)。如果是 UART, `devintr` 就会调用 `uartintr`。`uartintr` (`kernel/uart.c:180`) 从 UART 硬件读取所有等待中的输入字符, 并将它们传递给 `consoleintr` (`kernel/console.c:138`); 它不会等待字符, 因为后续输入会触发新的中断。`consoleintr` 的工作是将输入字符累积到 `cons.buf` 中, 直到整行到达。`consoleintr` 对退格和其他几个字符进行特殊处理。当换行符到达时, `consoleintr` 唤醒正在等待的 `consoleread` (如果有的话)。一旦被唤醒, `consoleread` 将读取 `cons.buf` 中的完整一行, 将其复制到用户空间, 并

```
return (via the system call machinery) to user space.
```

5.2 代码：控制台输出

对连接到控制台的文件描述符执行 `write` 系统调用, 最终会到达 `uartputc` (`kernel/uart.c:87`)。设备驱动程序维护一个输出缓冲区 (`uart_tx_buf`), 使得写入进程无需等待 UART 完成发送; `uartputc` 将每个字符追加到缓冲区, 调用 `uartstart` 启动设备传输 (如果尚未启动), 然后返回。`uartputc` 等待的唯一情况是缓冲区已满。每当 UART 完成一个字节的发送, 就会产生一个中断。`uartintr` 调用 `uartstart`, 后者检查设备是否真正完成了发送, 并将下一个已缓冲的输出字符交给设备。因此, 如果一个进程向控制台写入多个字节, 通常第一个字节由 `uartputc` 调用 `uartstart` 发送, 其余已缓冲的字节则在传输完成中断到来时, 由 `uartintr` 调用 `uartstart` 逐一发送。

值得注意的一个通用模式是：通过缓冲和中断, 将设备活动与进程活动解耦。控制台驱动程序即使在没有进程等待读取时也能处理输入; 后续的读取操作将看到这些输入。类似地, 进程可以发送输出而无需等待设备。这种解耦可以让进程与设备 I/O 并发执行, 从而提升性能, 在设备较慢 (如 UART) 或需要即时响应 (如回显输入字符) 时尤为重要。这一思想有时被称为 I/O 并发 (I/O concurrency)。

5.3 驱动程序中的并发

你可能已经注意到在 `consoleread` 和 `consoleintr` 中有对 `acquire` 的调用。这些调用会获取一个锁, 以保护控制台驱动程序的数据结构免受并发访问的影响。这里存在三种并发危险：两个进程可能在不同

的 CPU 上同时调用 `consoleread`；硬件可能在某个 CPU 正在 `consoleread` 内部执行时，要求该 CPU 处理一个控制台（实际上是 UART）中断；以及硬件可能在 `consoleread` 执行期间，在另一个 CPU 上触发控制台中断。第 6 章将探讨锁在这些场景中的作用。驱动程序中另一个需要谨慎处理并发的方面是：一个进程可能正在等待来自某个设备的输入，但表示输入到达的中断可能在另一个进程（甚至没有任何进程）运行时触发。因此，中断处理程序不允许考虑它所中断的进程或代码。例如，中断处理程序不能安全地使用当前进程的页表调用 `copyout`。中断处理程序通常只做相对较少的工作（例如，仅将输入数据复制到缓冲区），然后唤醒上半部分代码来完成其余工作。

5.4 定时器中断

Xv6 使用定时器中断来维护其时钟，并使其能够在计算密集型进程之间进行切换；`usertrap` 和 `kerneltrap` 中的 `yield` 调用会触发这种切换。定时器中断来自连接到每个 RISC-V CPU 的时钟硬件。Xv6 对该时钟硬件进行编程，使其周期性地中断每个 CPU。RISC-V 要求定时器中断在机器模式下处理，而非监督模式。RISC-V 机器模式在没有分页的情况下执行，并使用一组独立的控制寄存器，因此在机器模式下运行普通的 xv6 内核代码并不可行。因此，xv6 完全独立于上述陷阱机制来处理定时器中断。在 `start.c` 中以机器模式执行的代码，在 `main` 之前，设置了接收定时器中断的准备工作（`kernel/start.c:57`）。其中一部分工作是对 CLINT 硬件（核心本地中断器）进行编程，使其在一定延迟后产生中断。另一部分是设置一个暂存区，类似于 `trapframe`，以帮助定时器中断处理程序保存寄存器和 CLINT 寄存器的地址。最后，`start` 将 `mtvec` 设置为 `timerverec` 并启用定时器中断。定时器中断可以在用户代码或内核代码执行的任意时刻发生；内核无法在关键操作期间禁用定时器中断。因此，定时器中断处理程序必须以保证不干扰被中断的内核代码的方式完成其工作。基本策略是让处理程序请求 RISC-V 触发一个“软件中断”并立即返回。RISC-V 通过普通的陷阱机制将软件中断传递给内核，并允许内核禁用它们。处理由定时器中断生成的软件中断的代码可以在 `devintr` 中看到（`kernel/trap.c:204`）。机器模式定时器中断向量是 `timerverec`（`kernel/kernelvec.S:93`）。它在 `start` 准备好的暂存区中保存若干寄存器，告知 CLINT 何时生成下一次定时器中断，请求 RISC-V 触发软件中断，恢复寄存器，然后返回。定时器中断处理程序中没有任何 C 代码。

5.5 现实世界

Xv6 在内核中执行时允许设备中断和定时器中断，在执行用户程序时同样如此。定时器中断会强制从定时器中断处理程序发起线程切换（调用 `yield`），即使在内核中执行时也不例外。在内核线程有时会花费大量时间进行计算而不返回用户空间的情况下，能够在内核线程之间公平地对 CPU 进行时间片划分是很有用的。然而，内核代码需要时刻注意自身可能被挂起（由于定时器中断）并在之后于不同的 CPU 上恢复执行，这正是 xv6 中某些复杂性的来源。如果设备中断和定时器中断仅在执行用户代码时发生，内核可以做得更简单一些。

在一台典型计算机上完整地支持所有设备是一项繁重的工作，因为设备种类繁多，每种设备都有许多功能，且设备与驱动程序之间的协议可能既复杂又缺乏完善的文档。在许多操作系统中，驱动程序的代码量比内核核心代码还要多。

UART 驱动程序通过读取 UART 控制寄存器每次获取一个字节的数据；这种模式称为编程式 I/O (programmed I/O)，因为数据的移动由软件驱动。编程式 I/O 实现简单，但速度太慢，无法用于高数据速率的场景。需要高速传输大量数据的设备通常使用直接内存访问 (DMA)。DMA 设备硬件直接将接收到的数据写入 RAM，并从 RAM 读取待发送的数据。现代磁盘和网络设备均使用 DMA。DMA 设备的驱动程序会在 RAM 中准备好数据，然后向控制寄存器写入一次，通知设备处理已准备好的数据。

当设备需要在不可预测的时间获得响应，且响应频率不太高时，使用中断是合理的。但中断会带来较高的 CPU 开销。因此，网络和磁盘控制器等高速设备会使用一些技巧来减少对中断的需求。一种技巧是对一整批传入或传出请求只触发一次中断。另一种技巧是驱动程序完全禁用中断，并定期轮询设备以检查其是否需要处理。这种技术称为轮询 (polling)。如果设备操作完成得非常快，轮询是合理的；但如果设备大多数时候处于空闲状态，则会浪费 CPU 时间。有些驱动程序会根据当前设备负载在轮询和中断之间动态切换。

UART 驱动程序首先将接收到的数据复制到内核中的缓冲区，然后再复制到用户空间。在低数据速率下这是合理的，但对于高速产生或消费数据的设备来说，这种双重复制会显著降低性能。某些操作系统能够直接在用户空间缓冲区与设备硬件之间传输数据，通常借助 DMA 实现。

5.6 练习

1. 修改 `uart.c`，使其完全不使用中断。你可能还需要修改 `console.c`。
2. 为以太网卡添加一个驱动程序。

第6章：锁

大多数内核，包括

xv6，会将多个活动的执行交错进行。交错的来源之一是多处理器硬件：具有多个独立运行 CPU 的计算机，例如 xv6 的 RISC-V。这些 CPU 共享物理内存，xv6 利用这种共享来维护所有 CPU 都会读写的数据结构。这种共享带来了一种可能性：一个 CPU 在读取数据结构的同时，另一个 CPU 正处于更新该数据结构的中途，甚至多个 CPU 同时更新同一数据；如果没有精心的设计，这种并行访问很可能产生错误的结果或损坏数据结构。即使在单处理器上，内核也可能在多个线程之间切换 CPU，导致它们的执行被交错进行。最后，如果中断恰好在错误的时刻发生，修改与某些可中断代码相同数据的设备中断处理程序可能会损坏数据。并发一词指的是由于多处理器并行、线程切换或中断，导致多个指令流交错执行的情形。内核中充满了被并发访问的数据。例如，两个 CPU 可能同时调用 `kalloc`，从而并发地从空闲链表的头部弹出元素。内核设计者倾向于允许大量并发，因为它可以通过并行性提升性能，并提高响应能力。然而，正因如此，内核设计者需要花费大量精力来说服自己，尽管存在这种并发，代码仍然是正确的。实现正确代码的方法有很多，有些比其他方法更容易推理。以并发正确性为目标的策略及其支持的抽象，被称为并发控制技术。xv6 根据不同情况使用多种并发控制技术，还有更多技术可供选择。本章重点介绍一种广泛使用的技术：锁。锁提供互斥，确保每次只有一个 CPU 能持有该锁。如果程序员为每个共享数据项关联一把锁，并且代码在使用某个数据项时始终持有相应的锁，那么该数据项每次就只会被一个 CPU 使用。在这种情况下，我们说锁保护该数据项。尽管锁是一种易于理解的并发控制机制，但其缺点在于可能降低性能，因为它会将并发操作串行化。本章其余部分将解释为何 xv6 需要锁、xv6 如何实现锁，以及如何使用锁。

Memory

list BUS

```
l->next = list
```

```
l->next = list
```

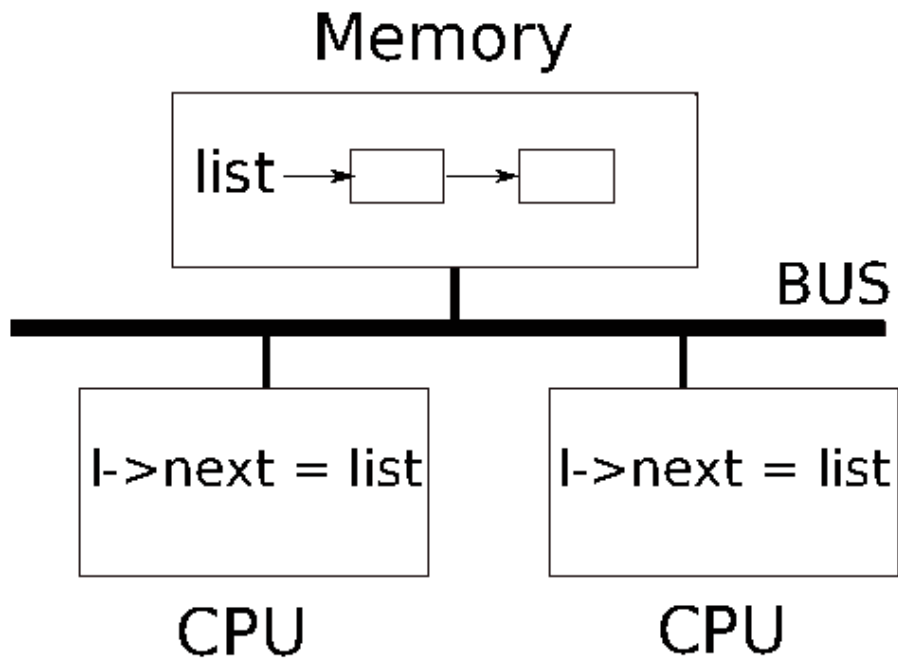


图 6.1：简化的 SMP 架构

6.1 竞争条件

以锁为例，说明我们为何需要锁：考虑两个进程在两个不同的 CPU 上调用 `wait`。`wait` 会释放子进程的内存。因此，在每个 CPU 上，内核会调用 `kfree` 来释放子进程的页面。内核分配器维护一个链表：`kalloc()`（`kernel/kalloc.c:69`）从空闲页列表中弹出一个内存页，`kfree()`（`kernel/kalloc.c:47`）则将一个页面推入空闲列表。为了获得最佳性能，我们可能希望两个父进程的 `kfree` 操作能并行执行，而无需任何一方等待另一方，但这在 `xv6` 的 `kfree` 实现下是不正确的。图 6.1 更详细地说明了这一情形：链表位于两个 CPU 共享的内存中，两个 CPU 使用加载和存储指令对链表进行操作。（实际上，处理器有缓存，但从概念上讲，多处理器系统的行为就如同存在一块单一的共享内存。）如果没有并发请求，你可以按如下方式实现链表的 `push` 操作：

```

1      struct element {
2          int data;
3          struct element *next;
4      };

6      struct element *list = 0;

8 void 9 push(int data) 10 {
11         struct element *l;

```

```

13      l = malloc(sizeof *l);
14      l->data = data;
15      l->next = list;
16      list = l;

```

15 16 CPU 1

Memory

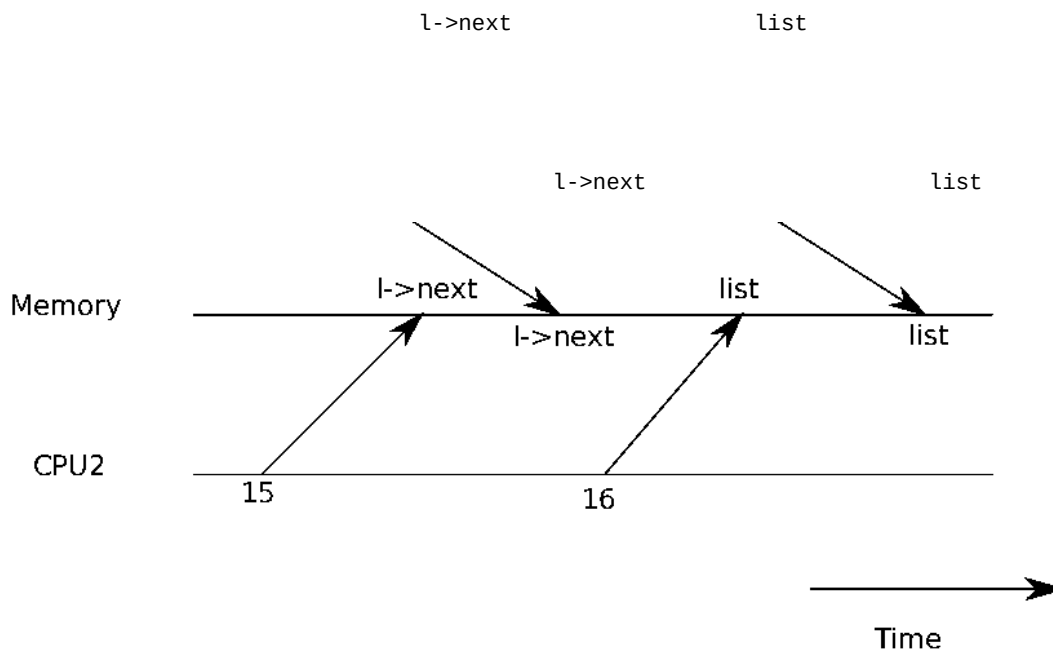


Figure 6.2: Example race

17 }

如果单独执行，该实现是正确的。然而，如果有多个副本并发执行，则该代码是不正确的。如果两个 CPU 同时执行 `push`，两者可能都在执行第 16 行之前先执行了第 15 行（如图 6.1 所示），这会导致如图 6.2 所示的错误结果。此时会存在两个链表元素，其 `next` 都指向 `list` 的原来值。当第 16 行的两次对 `list` 的赋值发生时，第二次会覆盖第一次；第一次赋值所涉及的元素将会丢失。第 16 行的更新丢失是竞争条件的一个例子。竞争条件是指某个内存位置被并发访问，且至少有一次访问是写操作的情形。竞争通常是 bug 的信号，可能是更新丢失（如果访问都是写操作），也可能是读取到了未完全更新的数据结构。竞争的结果取决于两个相关 CPU 的精确时序，以及内存系统对其内存操作的排序方式，这使得由竞争引发的错误难以复现和调试。例如，在调试 `push` 时添加 `print` 语句可能会使执行时序发生足够的变化，从而使竞争消失。

避免竞争的常用方法是使用锁。锁确保互斥，使得在同一时刻只有一个 CPU 能够执行 `push` 的敏感代码行；这使得上述场景不可能发生。上述代码加锁后的正确版本只需添加几行（以黄色高亮显示）：

```

6      struct element *list = 0;
7      struct lock listlock;

```

```

9 void 10 push(int data) 11 {

    12         struct element *l;
    13         l = malloc(sizeof *l);
    14         l->data = data;

    16         acquire(&listlock);
    17         l->next = list;
    18         list = l;
    19         release(&listlock);

20 }

```

`acquire` 和 `release` 之间的指令序列通常称为临界区。通常称该锁“保护”`list`。当我们说锁保护数据时，实际上是指该锁保护适用于该数据的一组不变量。不变量是数据结构在操作过程中始终保持的属性。通常，一个操作的正确行为依赖于操作开始时不变量成立。操作可能会暂时违反不变量，但必须在结束前将其重新建立。例如，在链表的例子中，不变量是 `list` 指向链表的第一个元素，且每个元素的 `next` 字段指向下一个元素。`push` 的实现暂时违反了这一不变量：在第 17 行，`l` 的 `next` 指向了链表的下一个元素，但 `list` 尚未指向 `l`（在第 18 行才重新建立）。我们上面考察的竞争条件之所以发生，是因为第二个 CPU 在不变量（暂时）被违反时执行了依赖链表不变量的代码。正确使用锁可以确保在同一时刻只有一个 CPU 能够在临界区内操作数据结构，从而使得任何 CPU 都不会在数据结构的不变量不成立时执行数据结构操作。

你可以将锁理解为将并发的临界区串行化，使其一次只运行一个，从而保持不变量（假设各临界区在单独执行时是正确的）。你也可以将由同一把锁保护的临界区理解为彼此原子的，即每个临界区只能看到之前临界区完整的修改结果，而永远不会看到部分完成的更新。

尽管正确使用锁能使不正确的代码变得正确，但锁会限制性能。例如，如果两个进程并发调用 `kfree`，锁会将这两次调用串行化，我们就无法从在不同 CPU 上运行中获益。如果多个进程同时争用同一把锁，我们称它们发生了冲突，或称该锁存在争用。内核设计中的一个主要挑战是避免锁争用。xv6 在这方面做得很少，但复杂的内核会专门针对避免锁争用来组织数据结构和算法。在链表示例中，内核可以为每个 CPU 维护一个独立的空闲列表，只有当该 CPU 的列表为空且需要从其他 CPU 窃取内存时，才会访问其他 CPU 的空闲列表。其他使用场景可能需要更复杂的设计。

锁的放置位置对性能也很重要。例如，将 `acquire` 提前到 `push` 中更靠前的位置是正确的：将 `acquire` 调用移到第 13 行之前是可以的。但这可能会降低性能，因为这样 `malloc` 的调用也会被串行化。下面的“使用锁”一节提供了一些关于在何处插入 `acquire` 和 `release` 调用的指导方针。

6.2 代码：锁

Xv6 有两种类型的锁：自旋锁和睡眠锁。我们先从自旋锁开始介绍。Xv6 用 `struct spinlock` (`kernel/spinlock.h:2`) 来表示一个自旋锁。该结构体中的重要字段是 `locked`，当锁可用时其值为零，当锁被持有时其值为非零。从逻辑上讲，xv6 应该通过执行如下代码来获取锁：

```
21     void
22     acquire(struct spinlock *lk) // does not work!
23     {
24         for(;;) {
25             if(lk->locked == 0) {
26                 lk->locked = 1;
27                 break;
28             }
29         }
30     }
```

不幸的是，这个实现在多处理器上无法保证互斥性。可能出现这样的情况：两个 CPU 同时执行到第 25 行，都发现 `lk->locked` 为零，然后都通过执行第 26 行来抢占该锁。此时，两个不同的 CPU 同时持有该锁，这违反了互斥性。我们需要的是一种能让第 25 行和第 26 行作为一个原子（即不可分割的）步骤执行的方法。由于锁被广泛使用，多核处理器通常提供实现第 25 行和第 26 行原子版本的指令。在 RISC-V 上，这条指令是 `amoswap r, a`。`amoswap` 读取内存地址 `a` 处的值，将寄存器 `r` 的内容写入该地址，并将读取到的值存入 `r`。也就是说，它交换寄存器与内存地址中的内容。它以原子方式执行这一操作序列，使用专用硬件防止任何其他 CPU 在读操作和写操作之间访问该内存地址。Xv6 的 `acquire` (`kernel/spinlock.c:22`) 使用了可移植的 C 库调用 `__sync_lock_test_and_set`，该调用最终归结为 `amoswap` 指令；其返回值是 `lk->locked` 的旧（被交换出的）内容。`acquire` 函数将这个交换操作包裹在一个循环中，不断重试（自旋），直到成功获取锁为止。每次迭代都会将 1 交换入 `lk->locked` 并检查之前的值；如果之前的值为零，则表示我们已获取到锁，且此次交换已将 `lk->locked` 设置为 1。如果之前的值为 1，则说明其他某个 CPU 持有该锁，而我们以原子方式将 1 交换入 `lk->locked` 并不改变其值。

一旦获取到锁，`acquire` 会出于调试目的记录获取该锁的 CPU。`lk->cpu` 字段受该锁保护，只能在持有锁的情况下修改。

函数 `release` (`kernel/spinlock.c:47`) 与 `acquire` 相反：它清除 `lk->cpu` 字段，然后释放锁。从概念上讲，释放锁只需将 `lk->locked` 赋值为零即可。C 标准允许编译器用多条存储指令来实现一次赋值，因此一个 C 赋值操作相对于并发代码可能是非原子的。为此，`release` 使用 C 库函数 `__sync_lock_release` 来执行原子赋值。这个函数同样最终归结为一条 RISC-V `amoswap` 指令。

6.3 代码：使用锁

Xv6 在许多地方使用锁来避免竞争条件。如上所述, `kalloc` (`kernel/kalloc.c:69`) 和 `kfree` (`kernel/kalloc.c:47`) 是一个很好的例子。可以尝试练习 1 和练习 2, 看看如果这些函数省略锁会发生什么。你可能会发现很难触发错误行为, 这说明要可靠地测试代码是否存在加锁错误和竞争条件是很困难的。Xv6 存在一些竞争条件也并非不可能。使用锁的一个难点在于决定使用多少个锁, 以及每个锁应该保护哪些数据和不变量。有几条基本原则。第一, 任何时候当一个变量可能被某个 CPU 写入, 同时另一个 CPU 也可能读取或写入它, 就应该使用锁来防止这两个操作重叠。第二, 请记住锁是用来保护不变量的: 如果一个不变量涉及多个内存位置, 通常需要用同一把锁来保护所有这些位置, 以确保不变量得以维持。

上述规则说明了何时需要加锁, 但没有说明何时不需要加锁。出于效率考虑, 不过度加锁非常重要, 因为锁会降低并行性。如果并行性不重要, 则可以安排只有单个线程运行, 无需考虑锁。一个简单的内核可以在多处理器上通过只使用一把锁来实现这一点——在进入内核时获取该锁, 在退出内核时释放它(尽管像管道读取或 `wait` 这样的系统调用会带来问题)。许多单处理器操作系统已经使用这种方法被移植到多处理器上运行, 这种方法有时被称为“大内核锁”, 但该方法牺牲了并行性: 每次只有一个 CPU 能够在内核中执行。如果内核需要执行大量计算, 使用更多粒度更细的锁将更为高效, 从而使内核能够在多个 CPU 上同时执行。

以粗粒度加锁为例, xv6 的 `kalloc.c` 分配器有一个由单一锁保护的空闲链表。如果不同 CPU 上的多个进程同时尝试分配页面, 每个进程都必须通过在 `acquire` 中自旋来等待轮到自己。自旋会降低性能, 因为它不产生有用的工作。如果锁竞争浪费了大量 CPU 时间, 或许可以通过修改分配器设计, 使其拥有多个空闲链表, 每个链表有自己的锁, 从而实现真正的并行分配, 以提升性能。

以细粒度加锁为例, xv6 为每个文件单独设置一把锁, 这样操作不同文件的进程通常无需等待彼此的锁。如果希望允许进程同时写入同一文件的不同区域, 文件加锁方案可以做得粒度更细。最终, 锁粒度的决策需要由性能测量和复杂性权衡共同驱动。

后续章节在介绍 xv6 的各个部分时, 将会提及 xv6 使用锁处理并发的相关示例。作为预览, Figure 6.3 列出了 xv6 中所有的锁。

6.4 死锁与锁排序

如果内核中的某条代码路径需要同时持有多个锁, 那么所有代码路径必须以相同的顺序获取这些锁, 这一点非常重要。如果顺序不一致, 就存在死锁的风险。假设 xv6 中有两条代码路径都需要锁 A 和锁 B, 但代码路径 1 按照先 A 后

锁	描述
<code>bcache.lock</code>	保护块缓冲区缓存条目的分配
<code>cons.lock</code>	串行化对控制台硬件的访问, 避免输出混乱
<code>ftable.lock</code>	串行化文件表中 <code>struct file</code> 的分配
<code>icache.lock</code>	保护 inode 缓存条目的分配
<code>vdisk_lock</code>	串行化对磁盘硬件及 DMA 描述符队列的访问
<code>kmem.lock</code>	串行化内存的分配
<code>log.lock</code>	串行化事务日志上的操作
<code>pipe</code> 的 <code>pi->lock</code>	

串行化每个管道上的操作 || pid_lock | 串行化 next_pid 的递增操作 || proc 的 p->lock |
串行化对进程状态的修改 |

Lock	Description
bcache.lock	Protects allocation of block buffer cache entries
cons.lock	Serializes access to console hardware, avoids intermixed output
ftable.lock	Serializes allocation of a struct file in file table
icache.lock	Protects allocation of inode cache entries
vdisk_lock	Serializes access to disk hardware and queue of DMA descriptors
kmem.lock	Serializes allocation of memory
log.lock	Serializes operations on the transaction log
pipe's pi->lock	Serializes operations on each pipe
pid_lock	Serializes increments of next_pid
proc's p->lock	Serializes changes to process's state
tickslock	Serializes operations on the ticks counter
inode's ip->lock	Serializes operations on each inode and its content
buf's b->lock	Serializes operations on each block buffer

图 6.3：xv6 中的锁

B 的顺序获取锁，而另一条路径按照先 B 后 A 的顺序获取锁。假设线程 T1 执行代码路径 1 并获取了锁 A，线程 T2 执行代码路径 2 并获取了锁 B。接下来 T1 将尝试获取锁 B，T2 将尝试获取锁 A。两次获取都将无限期阻塞，因为在两种情况下，所需的锁都被另一个线程持有，而该线程在其 acquire 返回之前不会释放锁。为了避免此类死锁，所有代码路径必须以相同的顺序获取锁。全局锁获取顺序的要求意味着锁实际上是每个函数规范的一部分：调用者必须以能够按照约定顺序获取锁的方式调用函数。由于 sleep 的工作方式（参见第 7 章），xv6 中存在大量涉及每个进程锁（即每个 struct proc 中的锁）的长度为二的锁排序链。例如，consoleintr（kernel/console.c:138）是处理键入字符的中断例程。当换行符到达时，任何正在等待控制台输入的进程都应被唤醒。为此，consoleintr 在调用 wakeup 时持有 cons.lock，而 wakeup 会获取等待进程的锁以将其唤醒。因此，全局避免死锁的锁顺序中包含这样一条规则：cons.lock 必须在任何进程锁之前获取。文件系统代码包含 xv6 中最长的锁链。例如，创建一个文件需要同时持有目录上的锁、新文件 inode 上的锁、磁盘块缓冲区上的锁、磁盘驱动程序的 vdisk_lock，以及调用进程的 p->lock。为了避免死锁，文件系统代码始终按照上一句中提到的顺序获取锁。遵守全局避免死锁的顺序有时出人意料地困难。有时锁的顺序与逻辑程序结构相冲突，例如，代码模块 M1 调用模块 M2，但锁的顺序要求在 M1 中的锁之前先获取 M2 中的锁。有时锁的标识无法提前得知，可能是因为必须持有某个锁才能确定下一个要获取的锁的标识。这种情况在文件系统沿路径名逐级查找各组成部分时，以及在 wait 和 exit 的代码中搜索进程表以查找子进程时都会出现。最后，死锁的风险往往是限制锁方案粒度的一个约束因素，因为锁越多，通常意味着死锁的机会越多。避免死锁的需求往往是内核实现中的一个主要制约因素。

6.5 锁与中断处理程序

某些 xv6 自旋锁保护的数据会被线程和中断处理程序同时使用。例如，`clockintr` 定时器中断处理程序可能在大约相同的时间递增 `ticks` (`kernel/trap.c:163`)，而内核线程也在 `sys_sleep` (`kernel/sysproc.c:64`) 中读取 `ticks`。锁 `tickslock` 将这两次访问串行化。自旋锁与中断的交互引发了一个潜在的危险。假设 `sys_sleep` 持有 `tickslock`，而其所在的 CPU 被定时器中断打断。`clockintr` 将尝试获取 `tickslock`，发现它已被持有，并等待其被释放。在这种情况下，`tickslock` 永远不会被释放：只有 `sys_sleep` 才能释放它，但 `sys_sleep` 在 `clockintr` 返回之前不会继续运行。因此 CPU 将发生死锁，任何需要该锁的代码也将冻结。为了避免这种情况，如果某个自旋锁被中断处理程序使用，CPU 在持有该锁时必须不能开启中断。xv6 采取了更为保守的策略：当 CPU 获取任意锁时，xv6 总是禁用该 CPU 上的中断。中断仍然可以在其他 CPU 上发生，因此中断中的 `acquire` 可以等待某个线程在另一个 CPU 上释放自旋锁；只是不能在同一个 CPU 上等待。当 CPU 不持有任何自旋锁时，xv6 重新启用中断；为了处理嵌套临界区，需要进行一些计数管理。`acquire` 调用 `push_off` (`kernel/spinlock.c:89`)，`release` 调用 `pop_off` (`kernel/spinlock.c:100`)，以追踪当前 CPU 上锁的嵌套层级。当计数归零时，`pop_off` 恢复最外层临界区开始时的中断使能状态。`intr_off` 和 `intr_on` 函数分别执行 RISC-V 指令来禁用和启用中断。

It is important that `acquire` call `push_off` strictly before setting `lk->locked` (`kernel/spin-`

`lock.c:28`)。如果二者顺序颠倒，将存在一个短暂的窗口期，在该窗口期内锁已被持有但中断仍处于启用状态，一次时机不当的中断将导致系统死锁。同样，`release` 必须在释放锁之后才调用 `pop_off` (`kernel/spinlock.c:66`)，这一点同样重要。

6.6 指令与内存排序

自然地，我们会认为程序按照源代码语句出现的顺序依次执行。然而，许多编译器和 CPU 为了获得更高的性能，会乱序执行代码。如果一条指令需要许多个时钟周期才能完成，CPU 可能会提前发出该指令，以便与其他指令重叠执行，从而避免 CPU 停顿。例如，CPU 可能会注意到在一个串行指令序列中，指令 A 和 B 彼此不相互依赖。CPU 可能会先执行指令 B，原因可能是 B 的输入比 A 的输入更早就绪，或者是为了让 A 和 B 的执行过程相互重叠。编译器也可能执行类似的重排序，在源代码中先出现的语句对应的指令之前，先输出后出现语句的指令。编译器和 CPU 在重排序时会遵循相应规则，以确保不改变正确编写的串行代码的执行结果。然而，这些规则确实允许会改变并发代码执行结果的重排序，并且很容易在多处理器上导致错误行为 [2, 3]。CPU 的排序规则被称为内存模型。例如，在下面这段 `push` 的代码中，如果编译器或 CPU 将第 4 行对应的存储操作移到第 6 行的 `release` 之后，将会造成灾难性后果：

```
1      l = malloc(sizeof *l);
2      l->data = data;
3      acquire(&listlock);
4      l->next = list;
5      list = l;
6      release(&listlock);
```

如果发生这样的重排序，将会出现一个时间窗口，在此期间另一个 CPU 可能会获取

the lock and observe the updated list, but see an uninitialized list->next.

为了告知硬件和编译器不要执行此类重排序，xv6 在 `acquire` (kernel/spinlock.c:22) 和 `release` (kernel/spinlock.c:47) 中均使用了 `__sync_synchronize()`。`__sync_synchronize()` 是一个内存屏障：它告知编译器和 CPU 不得将加载或存储操作跨越该屏障重排序。xv6 的 `acquire` 和 `release` 中的屏障在几乎所有关键情况下都强制执行了顺序，因为 xv6 在访问共享数据时使用了锁。第 9 章将讨论少数例外情况。

6.7 睡眠锁

有时 xv6 需要长时间持有一把锁。例如，文件系统（第 8 章）在读写磁盘上的文件内容时会保持文件锁定，而这些磁盘操作可能耗费数十毫秒。如果另一个进程想要获取该锁，长时间持有自旋锁将造成浪费，因为获取锁的进程会在自旋过程中长时间空耗

CPU。自旋锁的另一个缺点是，进程在持有自旋锁期间无法让出

CPU；而我们希望能够这样做，以便持有锁的进程在等待磁盘时，其他进程可以使用

CPU。在持有自旋锁期间让出 CPU

是非法的，因为如果第二个线程随后尝试获取该自旋锁，可能会导致死锁——由于 `acquire` 不会让出

CPU，第二个线程的自旋可能会阻止第一个线程运行并释放锁。在持有锁期间让出 CPU

还会违反“持有自旋锁期间必须关闭中断”这一要求。因此，我们希望有一种锁，能够在等待获取时让出

CPU，并且在持有锁期间允许让出（及开启中断）。Xv6

以睡眠锁的形式提供了此类锁。`acquiresleep` (kernel/sleeplock.c:22) 在等待时会让出

CPU，具体技术将在第 7 章中介绍。从高层次来看，睡眠锁有一个 `locked`

字段，该字段由一把自旋锁保护，`acquiresleep` 调用 `sleep` 时会原子性地让出 CPU

并释放自旋锁。这样一来，其他线程可以在 `acquiresleep`

等待期间执行。由于睡眠锁保持中断启用，它们不能在中断处理程序中使用。由于 `acquiresleep`

可能让出 CPU，睡眠锁也不能在自旋锁临界区内使用（尽管自旋锁可以在睡眠锁临界区内使用）。自旋锁最适合短临界区，因为等待自旋锁会空耗 CPU 时间；而睡眠锁则适合耗时较长的操作。

6.8 现实世界

尽管对并发原语和并行性的研究已进行多年，使用锁进行编程依然充满挑战。通常最好将锁封装在更高层次的构造中，例如同步队列，尽管 xv6 并没有这样做。如果你使用锁编程，明智的做法是使用能够识别竞争条件的工具，因为很容易遗漏某个需要加锁的不变量。大多数操作系统支持 POSIX

线程 (Pthreads)，它允许一个用户进程在不同 CPU 上并发运行多个线程。Pthreads

支持用户级锁、屏障等机制。支持 Pthreads 需要操作系统的配合。例如，如果某个 pthread

在系统调用中阻塞，同一进程的另一个 pthread 应当能够在该 CPU 上继续运行。再例如，如果某个 pthread

修改了其进程的地址空间（例如映射或取消映射内存），内核必须安排运行同一进程线程的其他 CPU

更新其硬件页表，以反映地址空间的变化。不使用原子指令实现锁是可能的

[8]，但代价高昂，大多数操作系统还是使用原子指令。如果多个 CPU

同时尝试获取同一把锁，锁的开销可能会很大。如果某个 CPU 在其本地缓存中缓存了一把锁，而另一个 CPU 需要获取该锁，则更新持有该锁的缓存行的原子指令必须将该缓存行从一个 CPU 的缓存移动到另一个 CPU 的缓存，并且可能需要使该缓存行的其他所有副本失效。从另一个 CPU 的缓存中获取缓存行，其代价可能比从本地缓存中获取高出数个数量级。为了避免与锁相关的开销，许多操作系统使用无锁数据结构和算法 [5, 10]。例如，可以实现本章开头那样的链表，在链表搜索时无需加锁，而插入元素时只需一条原子指令。然而，无锁编程比使用锁编程更为复杂；例如，必须考虑指令和内存重排序的问题。使用锁编程本已不易，因此 xv6 避免了无锁编程带来的额外复杂性。

6.9 练习

1. 注释掉 `kalloc` 中对 `acquire` 和 `release` 的调用 (`kernel/kalloc.c:69`)。这看起来应该会给调用 `kalloc` 的内核代码带来问题；你预期会看到哪些症状？当你运行 xv6 时，是否看到了这些症状？运行 `usertests` 时呢？如果你没有发现问题，原因是什么？试试在 `kalloc` 的临界区中插入虚循环，看是否能重现问题。
2. 假设你改为注释掉 `kfree` 中的锁（在恢复 `kalloc` 中的锁之后）。现在可能会出什么问题？`kfree` 中缺少锁是否比 `kalloc` 中缺少锁的危害更小？
3. 如果两个 CPU 同时调用 `kalloc`，其中一个必须等待另一个，这对性能不利。修改 `kalloc.c` 以提高并行度，使来自不同 CPU 的对 `kalloc` 的同时调用无需互相等待即可继续执行。
4. 使用 POSIX 线程编写一个并行程序，大多数操作系统都支持 POSIX 线程。例如，实现一个并行哈希表，并测量 `put/get` 的数量是否随核心数的增加而线性扩展。
5. 在 xv6 中实现 Pthreads 的一个子集。即，实现一个用户级线程库，使用户进程可以拥有多于 1 个线程，并安排这些线程能够在不同 CPU 上并行运行。设计一个能够正确处理线程发起阻塞系统调用以及修改共享地址空间的方案。

第七章：调度

任何操作系统运行的进程数量都可能多于计算机所拥有的 CPU

数量，因此需要一套方案来在各进程之间分时共享

CPU。理想情况下，这种共享对用户进程来说是透明的。一种常见的做法是通过将进程多路复用到硬件 CPU 上，使每个进程产生拥有独立虚拟 CPU 的错觉。本章将介绍 xv6 如何实现这种多路复用。

7.1 多路复用

Xv6 通过在两种情况下将每个 CPU

从一个进程切换到另一个进程来实现多路复用。第一，当进程等待设备或管道 I/O

完成、等待子进程退出，或在 `sleep` 系统调用中等待时，xv6 的睡眠与唤醒机制会触发切换。第二，xv6

周期性地强制切换，以应对长时间计算而不休眠的进程。这种多路复用造成了每个进程都拥有自己的 CPU 的假象，就像 xv6 使用内存分配器和硬件页表来造成每个进程都拥有自己的内存的假象一样。实现多路复用面临几个挑战。第一，如何从一个进程切换到另一个进程？尽管上下文切换的概念很简单，但其实实现却是 xv6 中最晦涩难懂的代码之一。第二，如何以对用户进程透明的方式强制切换？Xv6

使用计时器中断驱动上下文切换的标准技术。第三，多个 CPU 可能同时在进程间切换，因此需要一套加锁方案来避免竞争。第四，当进程退出时，其内存和其他资源必须被释放，但进程无法自行完成所有这些工作，例如，它不能在仍使用自己的内核栈的同时将其释放。第五，多核机器的每个核心必须记住它正在执行哪个进程，以便系统调用能够影响正确进程的内核状态。最后，睡眠与唤醒机制允许进程放弃 CPU 并进入睡眠状态等待某个事件，同时允许另一个进程将第一个进程唤醒。需要小心避免导致唤醒通知丢失的竞争。Xv6 尽量以最简单的方式解决这些问题，但尽管如此，最终的代码仍然颇为棘手。

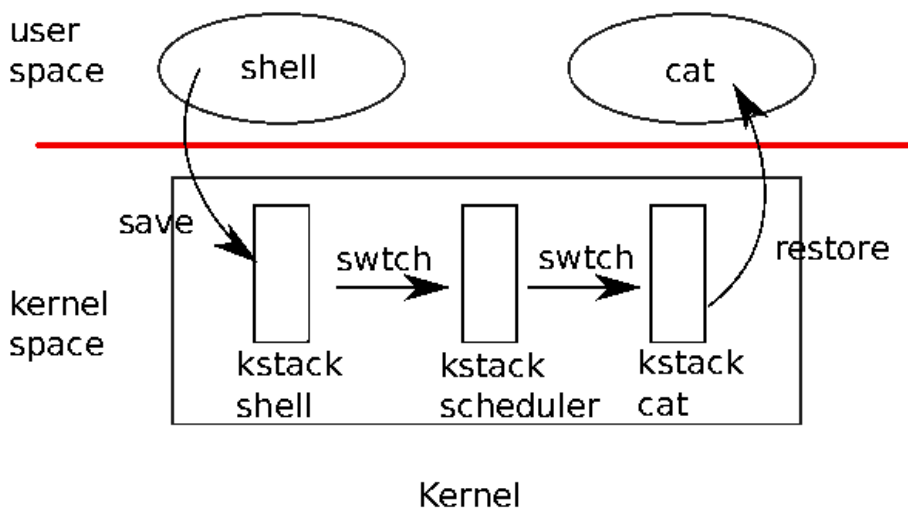


图 7.1：从一个用户进程切换到另一个用户进程。在此示例中，xv6 运行在单个 CPU 上（因此只有一个调度器线程）。

7.2 代码：上下文切换

图 7.1 概述了从一个用户进程切换到另一个用户进程所涉及的步骤：从旧进程的内核线程进行用户态到内核态的转换（系统调用或中断），切换到当前 CPU

的调度器线程的上下文，再切换到新进程内核线程的上下文，最后陷阱返回到用户态进程。xv6

调度器为每个 CPU 配备了一个专用线程（保存寄存器和栈），这是因为在旧进程的内核栈上执行调度器是不安全的：其他某个核心可能会唤醒该进程并运行它，在两个不同的核心上使用同一个栈将会造成灾难性后果。在本节中，我们将深入研究内核线程与调度器线程之间切换的机制。从一个线程切换到另一个线程，需要保存旧线程的 CPU

寄存器，并恢复新线程之前保存的寄存器；由于栈指针和程序计数器也被保存和恢复，这意味着 CPU 将切换栈并切换所执行的代码。函数 `swtch` 负责执行内核线程切换时的保存与恢复操作。`swtch` 并不直接了解线程的概念；它只是保存和恢复寄存器集合，这些集合称为上下文。当一个进程需要放弃 CPU 时，该进程的内核线程调用 `swtch`

保存自身的上下文，并返回到调度器上下文。每个上下文存储在一个 `struct`

`context` (`kernel/proc.h:2`) 中，该结构体本身包含在进程的 `struct proc` 或 CPU 的 `struct cpu` 中。`swtch` 接受两个参数：`struct context *old` 和 `struct context`

`*new`。它将当前寄存器保存到 `old` 中，从 `new` 中加载寄存器，然后返回。让我们跟踪一个进程通过 `swtch` 进入调度器的过程。我们在第 4 章中看到，中断结束时的一种可能是 `usertrap` 调用 `yield`。`yield` 继而调用 `sched`，`sched` 调用

`swtch` 将当前上下文保存到 `p->context` 中，并切换到之前保存在 `cpu->scheduler` 中的调度器上下文 (`kernel/proc.c:509`)。

`swtch` (`kernel/swtch.S:3`) 只保存被调用者保存的寄存器；调用者保存的寄存器由调用方的 C 代码保存在栈上（如有需要）。`swtch` 知道每个寄存器字段在

`struct context` 中的偏移量。它不保存程序计数器。取而代之，`swtch` 保存 `ra` 寄存器，

该寄存器保存着调用 `swtch` 时的返回地址。随后 `swtch`

从新上下文中恢复寄存器，新上下文中保存着由前一次 `swtch` 保存的寄存器值。当 `swtch` 返回时，它返回到恢复后的 `ra` 寄存器所指向的指令处，即新线程之前调用 `swtch` 时所在的那条指令。此外，它在新线程的栈上返回。

在我们的示例中，`sched` 调用 `swtch` 切换到 `cpu->scheduler`，即每个 CPU 的调度器上下文。

该上下文是由调度器调用 `swtch` (`kernel/proc.c:475`) 时保存的。当我们一直追踪的这次 `swtch` 返回时，它返回的不是 `sched`，而是 `scheduler`，且其栈指针指向当前 CPU 的调度器栈。

7.3 代码：调度

上一节研究了 `swtch` 的底层细节；现在让我们把 `swtch`

视为已知，来考察从一个进程的内核线程经由调度器切换到另一个进程的过程。调度器以每个 CPU 一个特殊线程的形式存在，每个线程运行 `scheduler` 函数。该函数负责选择下一个要运行的进程。想要让出

CPU 的进程必须获取自身的进程锁 `p->lock`，释放其持有的所有其他锁，更新自身状态 (`p->state`)，然后调用 `sched.Yield` (`kernel/proc.c:515`) 遵循这一约定，

`sleep` 和 `exit` 也同样如此，我们将在后面进行介绍。`sched`

会再次检查这些条件 (`kernel/proc.c:499-504`)，以及由这些条件推导出的一个结论：由于持有了锁，中断

应当被禁用。最后，`sched` 调用 `swtch` 将当前上下文保存到 `p->context`，并

切换到 `cpu->scheduler` 中的调度器上下文。`Swtch` 在调度器的栈上返回，就如同

调度器的 `swtch` 已经返回一样。调度器继续执行 `for`

循环，找到一个可运行的进程，切换到该进程，如此循环往复。

我们刚才看到，`xv6` 在调用 `swtch` 的过程中持有 `p->lock`：`swtch` 的调用者必须已经

持有该锁，并且锁的控制权会传递给被切换到的代码。这与锁的通常使用惯例不同；通常情况下，获取锁的线程也负责释放该锁，这样更便于推断正确性。对于上下文切换而言，有必要打破这一

约定，因为 `p->lock` 保护的是进程状态和上下文字段的不变量，而这些不变量在执行

`swtch` 期间并不成立。如果在 `swtch` 期间不持有 `p->lock` 可能引发的一个问题示例：

在 `yield` 将进程状态设置为 `RUNNABLE` 之后、`swtch` 使其停止使用自身内核栈之前，另一个 CPU 可能决定运行该进程。结果将是两个 CPU 在同一个栈上运行，这显然是不正确的。内核线程总是在 `sched` 中让出

CPU，并且总是切换到调度器中的同一位置，而调度器（几乎）总是切换到某个之前调用过 `sched` 的内核线程。因此，如果打印出 `xv6` 进行线程切换的行号，会观察到如下简单的模式：(`kernel/proc.c:475`)、(`kernel/proc.c:509`)、(`kernel/proc.c:475`)、(`kernel/proc.c:509`)，如此循环。这种在两个线程之间进行程式化切换的函数有时被称为协程；在本例中，`sched` 和 `scheduler`

互为对方的协程。有一种情况下，调度器对 `swtch` 的调用不会最终进入

`sched`。当一个新进程首次被调度时，它从 `forkret` (`kernel/proc.c:527`) 开始执行。`forkret` 的存在是为了释放

`p->lock`；否则，新进程可以从 `usertrapret` 开始执行。

`scheduler` (`kernel/proc.c:457`) 运行一个简单的循环：找到一个可运行的进程，运行它直到它让出 CPU，

然后重复。调度器遍历进程表寻找可运行的进程，即满足

`p->state == RUNNABLE` 的进程。一旦找到这样的进程，调度器便设置每个 CPU 的当前进程变量 `c->proc`，将进程标记为 `RUNNING`，然后调用 `swtch` 开始运行它（`kernel/proc.c:470-`

475）。理解调度代码结构的一种方式，它为每个进程强制维护一组不变量，

并在这些不变量不成立时持有 `p->lock`。其中一个不变量是：

如果一个进程处于 `RUNNING` 状态，计时器中断的 `yield` 必须能够安全地从该进程切换走；这意味着 CPU 寄存器必须保存该进程的寄存器值（即 `swtch`

尚未将它们移入上下文），且 `c->proc` 必须指向该进程。另一个不变量是：

如果一个进程处于 `RUNNABLE` 状态，空闲 CPU 的调度器必须能够安全地运行它；这意味着

`p->context` 必须保存该进程的寄存器（即它们实际上并不在真实寄存器中），没有任何 CPU 正在该进程的内核栈上执行，且没有任何 CPU 的 `c->proc` 指向该进程。注意，这些属性在持有 `p->lock` 期间往往并不成立。

维护上述不变量正是 `xv6` 经常在一个线程中获取 `p->lock` 而在另一个线程中

释放它的原因，例如在 `yield` 中获取、在 `scheduler` 中释放。一旦 `yield` 开始将运行中进程的状态修改为

`RUNNABLE`，锁必须保持持有状态，直到不变量得到恢复：最早的正确释放点是在调度器（运行于

自己的栈上）清除 `c->proc` 之后。类似地，一旦调度器开始将 `RUNNABLE` 进程

转换为 `RUNNING`，则在内核线程完全运行起来之前（例如在 `swtch` 之后、在 `yield` 中），不能释放该锁。

`p->lock` 还保护其他内容：`exit` 与 `wait` 之间的交互、避免丢失唤醒的机制

（见第 7.5 节），以及避免进程退出与其他进程读写其状态之间的竞争（例如，`exit` 系统调用

查看 `p->pid` 并设置 `p->killed`（`kernel/proc.c:611`））。或许值得考虑的是，

`p->lock` 的不同功能是否可以拆分，以提升清晰度，乃至改善性能。

7.4 代码：mycpu 与 myproc

Xv6 经常需要一个指向当前进程 `proc` 结构体的指针。在单处理器上，可以用一个全局变量指向当前进程。但在多核机器上这行不通，因为每个核心执行的是不同的进程。解决这一问题的方法是利用每个核心都拥有自己独立寄存器组这一特性；我们可以使用其中某个寄存器来帮助查找每个核心的相关信息。Xv6 为每个 CPU 维护一个 `struct cpu` (`kernel/proc.h:22`)，其中记录了该 CPU 当前正在运行的进程（如果有的话）、CPU 调度器线程的已保存寄存器，以及用于管理中断禁用所需的嵌套自旋锁计数。函数 `mycpu` (`kernel/proc.c:60`) 返回一个指向当前 CPU 的 `struct cpu` 的指针。RISC-V 对其 CPU 进行编号，为每个 CPU 分配一个 `hartid`。Xv6 确保在内核中运行时，每个 CPU 的 `hartid` 存储在该 CPU 的 `tp` 寄存器中。这样 `mycpu` 就可以使用 `tp` 作为索引，在 `cpu` 结构体数组中找到对应的那一项。确保 CPU 的 `tp` 始终保存该 CPU 的 `hartid` 需要一些处理。`mstart` 在 CPU 启动序列的早期、仍处于机器模式时设置 `tp` 寄存器 (`kernel/start.c:46`)。`usertrapret` 将 `tp` 保存在蹦床页 (`trampoline page`) 中，因为用户进程可能会修改 `tp`。最后，

`uservec` 在从用户空间进入内核时恢复已保存的 `tp` (`kernel/trampoline.S:70`)。编译器保证不会使用 `tp` 寄存器。如果 RISC-V 允许 `xv6` 直接读取当前 `hartid` 会更方便，但这只在机器模式下允许，在监督模式 (`supervisor mode`) 下不允许。`cpuuid` 和 `mycpu` 的返回值是脆弱的：如果计时器中断导致线程让出 CPU 并移动到另一个 CPU 上，之前返回的值就不再正确了。为了避免这一问题，`xv6` 要求调用者在使用完返回的 `struct cpu` 之前禁用中断，使用完毕后才重新启用中断。函数 `myproc` (`kernel/proc.c:68`) 返回当前 CPU 上正在运行的进程的 `struct proc` 指针。`myproc` 禁用中断，调用 `mycpu`，从 `struct cpu` 中取出当前

进程指针 (`c->proc`)，然后重新启用中断。即使在中断启用的情况下，`myproc`

的返回值也是安全的：如果计时器中断将调用进程移动到另一个 CPU，其 `struct proc` 指针仍然保持不变。

7.5 Sleep 与 Wakeup

调度器和锁有助于对一个进程隐藏另一个进程的存在，但到目前为止，我们还没有任何抽象来帮助进程有意地相互交互。为解决这一问题，人们发明了许多机制。Xv6 使用了一种称为 `sleep` 和 `wakeup` 的机制，它允许一个进程在等待某个事件时进入睡眠，而另一个进程在事件发生后将其唤醒。`Sleep` 和 `wakeup` 通常被称为序列协调 (`sequence coordination`) 或条件同步 (`conditional synchronization`) 机制。为了说明这一点，我们来考虑一种称为信号量 (`semaphore`) [4] 的同步机制，它用于协调生产者和消费者。信号量维护一个计数值并提供两种操作。"V" 操作（用于生产者）将计数值加一。"P" 操作（用于消费者）等待直到计数值非零，然后将其减一并返回。如果只有一个生产者线程和一个消费者线程，它们在不同的 CPU 上执行，且编译器没有过于激进地优化，那么下面这个实现是正确的：

```
100 struct semaphore {
101     struct spinlock lock;
102     int count;
```

```
103 }; 105 void 106 V(struct semaphore *s) 107 {
```

```
    108         acquire(&s->lock);
    109         s->count += 1;
    110         release(&s->lock);
```

```
111 } 113 void 114 P(struct semaphore *s) 115 {
```

```
    116         while(s->count == 0)
```

```
117 ;
```

```
    118         acquire(&s->lock);
    119         s->count -= 1;
    120         release(&s->lock);
```

121 } 上面这个实现的代价很高。如果生产者很少执行，消费者会将大部分时间花在 while 循环中自旋，期待计数值变为非零。消费者的 CPU

与其通过反复轮询 `s->count` 进行忙等待，不如去做更有意义的工作。避免

忙等待需要一种方式，让消费者在 V 将计数值加一之前让出 CPU，并在此之后恢复执行。下面是朝这个方向迈出的一步，尽管我们将会看到这还不够。让我们设想一对调用——`sleep` 和 `wakeup`，其工作方式如下：**`Sleep(chan)`** 在任意值 `chan` 上睡眠，`chan` 称为等待通道（wait channel）。**`Sleep`** 使调用进程进入睡眠，将 CPU 让给其他工作。**`wakeup(chan)`** 唤醒所有在 `chan` 上睡眠的进程（如果有的话），使它们的 `sleep` 调用返回。如果没有进程在 `chan` 上等待，`wakeup` 什么也不做。我们可以修改信号量的实现以使用 `sleep` 和 `wakeup`（更改部分以黄色高亮显示）：200 void

```
201 V(struct semaphore *s) 202 {
```

```
    203         acquire(&s->lock);
    204         s->count += 1;
    205         wakeup(s);
    206         release(&s->lock);
```

```
207 } 209 void 210 P(struct semaphore *s) 211 {
```

```
    212         while(s->count == 0)
    213             sleep(s);
    214         acquire(&s->lock);
    215         s->count -= 1;
    216         release(&s->lock);
```

217 } P 现在放弃 CPU 而不是自旋，这很好。然而，事实证明，要在这个接口下设计出不受

所谓“丢失唤醒（lost wake-up）”问题影响的 `sleep` 和 `wakeup` 并不简单。假设 P 在第 212 行发现 `s->count == 0`。当

P 处于第 212 行和第 213 行之间时，V 在另一个 CPU 上运行：它将 s->count 改为非零并

调用 wakeup，但此时没有进程在睡眠，因此什么也不做。现在 P 继续执行到第 213 行：它调用 sleep 并进入睡眠。这就产生了一个问题：P 正在睡眠等待一个已经发生过的 V 调用。除非我们足够幸运，生产者再次调用 V，否则消费者将永远等待，即使计数值已经非零。

这个问题的根源在于：P 只有在 s->count == 0 时才睡眠这一不变式，被恰好在错误时刻

运行的 V 所违反。一种错误的保护该不变式的方法是，在 P 中提前获取锁（下方以黄色高亮显示），使其对计数值的检查与调用 sleep 成为原子操作：300 void 301 V(struct semaphore *s) 302 {

```
303     acquire(&s->lock);
304     s->count += 1;
305     wakeup(s);
306     release(&s->lock);
```

307 } 309 void 310 P(struct semaphore *s) 311 {

```
312     acquire(&s->lock);
313     while(s->count == 0)
314         sleep(s);
315     s->count -= 1;
316     release(&s->lock);
```

317 } 人们可能希望这个版本的 P 能够避免丢失唤醒，因为锁阻止了 V 在第 313 行和第 314 行之间执行。确实如此，但这也会造成死锁：P 在睡眠时持有锁，因此 V 将永远阻塞在等待锁上。我们将通过修改 sleep 的接口来修复前面的方案：调用者必须将条件锁（condition lock）传递给 sleep，以便在调用进程被标记为睡眠并在等待通道上等待之后，sleep 可以释放该锁。该锁将强制并发的 V 等待，直到 P 完成让自己进入睡眠，这样 wakeup 就能找到正在睡眠的消费者并将其唤醒。一旦消费者再次被唤醒，sleep 在返回之前会重新获取锁。我们新的正确的 sleep/wakeup 方案可按如下方式使用（更改部分以黄色高亮显示）：400 void 401 V(struct semaphore *s) 402 {

```
403     acquire(&s->lock);
404     s->count += 1;
405     wakeup(s);
406     release(&s->lock);
```

407 } 409 void 410 P(struct semaphore *s) 411 {

```
412     acquire(&s->lock);

413     while(s->count == 0)
414         sleep(s, &s->lock);
415     s->count -= 1;
416     release(&s->lock);
```

P 持有 `s->lock` 这一事实阻止了 V 在 P 检查 `c->count` 与调用 `sleep` 之间尝试唤醒它。然而请注意，我们需要 `sleep` 原子地释放 `s->lock`

并使消费进程进入睡眠。

7.6 代码：Sleep 与 Wakeup

让我们来看看 `sleep` (`kernel/proc.c:548`) 和 `wakeup` (`kernel/proc.c:582`) 的实现。基本思路是：让 `sleep` 将当前进程标记为 `SLEEPING`，然后调用 `sched` 释放 CPU；`wakeup` 则查找在给定的等待通道上睡眠的进程，并将其标记为 `RUNNABLE`。`sleep` 和 `wakeup` 的调用者可以使用任何双方约定的数字作为通道。`xv6` 通常使用与等待相关的内核数据结构的地址。

`Sleep acquires p->lock (kernel/proc.c:559). Now the process going to sleep holds both p->lock`

和 `lk`。在调用者处持有 `lk` 是必要的（在示例中为 P）：它确保没有其他进程（在示例中为正在运行 V 的进程）能够开始调用 `wakeup(chan)`。由于 `sleep` 此时已持有

`p->lock`，释放 `lk` 是安全的：其他进程可能会开始调用 `wakeup(chan)`，但 `wakeup` 会等待获取 `p->lock`，因此会等到 `sleep` 完成将进程置入睡眠状态后再执行，

从而避免 `wakeup` 错过 `sleep`。

有一个小的复杂情况：如果 `lk` 与 `p->lock` 是同一把锁，那么 `sleep` 在尝试获取 `p->lock` 时会与自身发生死锁。但如果调用 `sleep` 的进程已经持有 `p->lock`，

它就无需采取任何额外操作来避免错过并发的 `wakeup`。这种情况

出现在 `wait (kernel/proc.c:582)` 以 `p->lock` 为参数调用 `sleep` 时。

此时 `sleep` 持有 `p->lock` 且不持有其他锁，它可以通过记录睡眠通道、

将进程状态改为 `SLEEPING`，并调用 `sched (kernel/proc.c:564-567)` 来使进程进入睡眠。

稍后将会清楚地看到，为何 `p->lock` 必须在进程被标记为 `SLEEPING` 之后才能被（调度器）释放，这一点至关重要。

在某个时刻，某个进程将获取条件锁，设置睡眠者正在等待的条件，并调用 `wakeup(chan)`。重要的是，`wakeup` 必须在持有

条件锁的情况下调用¹。`Wakeup` 遍历进程表 (`kernel/proc.c:582`)。它获取所检查的每个进程的 `p->lock`，

既因为它可能需要修改该进程的状态，也因为 `p->lock`

确保 `sleep` 和 `wakeup` 不会互相错过。当 `wakeup` 发现某个处于 `SLEEPING` 状态且通道匹配的进程时，它将该进程的状态改为 `RUNNABLE`。调度器下次运行时，就会看到该进程已准备好运行。

为什么 `sleep` 和 `wakeup` 的加锁规则能确保睡眠进程不会错过唤醒？从进程检查条件之前到被标记为 `SLEEPING` 之后，睡眠进程在这整个期间持有条件锁或其自身的 `p->lock`，或同时持有两者。调用 `wakeup` 的进程在 `wakeup` 的循环中同时持有这两把锁。因此，唤醒方要么在消费线程检查条件之前就使条件为真；要么唤醒方的 `wakeup` 严格在睡眠线程被标记为 `SLEEPING` 之后才检查它。这样 `wakeup` 就能看到睡眠中的进程并将其唤醒（除非有其他操作先将其唤醒）。

¹ 严格来说，只要 `wakeup` 在 `acquire` 之后执行即可（即可以在释放锁之后再调用 `wakeup`）。

有时多个进程会在同一通道上睡眠；例如，多个进程从同一个管道读取数据。对 `wakeup` 的一次调用会将它们全部唤醒。其中一个会首先运行并获取 `sleep` 调用时所使用的锁，并（在管道的情况下）读取管道中等待的数据。其他进程会发现，尽管被唤醒了，却没有数据可读。从它们的角度来看，这次唤醒是“虚假的”，它们必须再次进入睡眠。因此，`sleep` 总是在一个检查条件的循环内被调用。即使两处 `sleep/wakeup` 的使用意外选择了相同的通道，也不会造成危害：它们会看到虚假的唤醒，但如上所述的循环可以容忍这一问题。

`sleep/wakeup` 的魅力在于它既轻量（无需创建特殊的数据结构来充当睡眠通道），又提供了一层间接性（调用者无需知道自己具体在与哪个进程交互）。

7.7 代码：管道

一个使用 `sleep` 和 `wakeup` 来同步生产者与消费者的更复杂示例，是 xv6 的管道实现。我们在第 1 章中已经了解了管道的接口：写入管道一端的字节被复制到内核内的缓冲区，然后可以从管道的另一端读取。后续章节将研究围绕管道的文件描述符支持，但现在我们来看看 `pipewrite` 和 `piperead` 的实现。每个管道由一个 `struct pipe` 表示，其中包含一个锁和一个数据缓冲区。字段 `nread` 和 `nwrite` 分别记录从缓冲区读取和写入的字节总数。缓冲区是循环使用的：在 `buf[PIPE_SIZE-1]` 之后写入的下一个字节是 `buf[0]`。

计数不会循环。这种约定使得实现可以区分满缓冲区（`nwrite ==`

`nread+PIPE_SIZE`）和空缓冲区（`nwrite == nread`），但这意味着对

缓冲区的索引必须使用 `buf[nread % PIPESIZE]` 而不是直接使用 `buf[nread]` (`nwrite` 同理)。假设对 `piperead` 和 `pipewrite` 的调用同时发生在两个不同的 CPU 上。`Pipewrite` (`kernel/pipe.c:77`) 首先获取管道的锁, 该锁保护计数、数据及其相关的不变量。`Piperead` (`kernel/pipe.c:103`) 随后也尝试获取该锁, 但无法成功。它在 `acquire` (`kernel/spinlock.c:22`) 中自旋等待锁。在 `piperead` 等待期间, `pipewrite` 循环遍历待写入的字节 (`addr[0..n-1]`), 依次将每个字节添加到管道中 (`kernel/pipe.c:95`)。在此循环过程中, 可能会发生缓冲区填满的情况 (`kernel/pipe.c:85`)。在这种情况下, `pipewrite` 调用 `wakeup` 来通知任何正在睡眠的读取者缓冲区中有数据等待,

然后在 `&pi->nwrite` 上睡眠, 等待读取者从缓冲区取出一些字节。

`Sleep` 在将 `pipewrite` 的进程置于睡眠状态的过程中释放了 `pi->lock`。

现在 `pi->lock` 已可用, `piperead` 成功获取该锁并进入其临界区:

它发现 `pi->nread != pi->nwrite` (`kernel/pipe.c:110`) (`pipewrite` 进入睡眠是因为

`pi->nwrite == pi->nread+PIPESIZE` (`kernel/pipe.c:85`)), 因此它进入 `for`

循环, 将数据从管道中复制出来 (`kernel/pipe.c:117`), 并将 `nread` 增加所复制的字节数。现在有同样数量的字节可供写入, 因此 `piperead` 在返回之前调用 `wakeup` (`kernel/pipe.c:124`) 来唤醒任何正在睡眠的写入者。`Wakeup` 找到一个在

`&pi->nwrite` 上睡眠的进程, 即正在运行 `pipewrite` 但因缓冲区填满而停止的进程。它

将该进程标记为 `RUNNABLE`。

管道代码为读取者和写入者使用了独立的睡眠通道 (`pi->nread` 和 `pi->nwrite`);

在多个读取者和写入者等待同一管道这种不太可能发生的情况下, 这可能会使系统更加高效。管道代码在检查睡眠条件的循环内调用 `sleep`; 如果存在多个读取者或写入者, 除第一个被唤醒的进程外, 其余进程都将发现条件仍为假, 并再次进入睡眠。

7.8 代码: wait、exit 与 kill

`sleep` 和 `wakeup` 可以用于多种类型的等待。第 1 章介绍过一个有趣的例子, 即子进程退出与父进程 `wait` 之间的交互。子进程终止时, 父进程可能已经在 `wait` 中睡眠, 也可能正在做其他事情; 在后一种情况下, 后续对 `wait` 的调用必须能观察到子进程的终止, 即便那是在子进程调用 `exit` 很久之后的事。xv6 记录子进程终止状态直到 `wait` 观察到它的方式是: `exit` 将调用者置于 `ZOMBIE` 状态, 子进程在该状态下一直等待, 直到父进程的 `wait` 发现它, 将子进程的状态改为 `UNUSED`, 复制子进程的退出状态, 并向父进程返回子进程的进程 ID。如果父进程在子进程之前退出, 父进程会将子进程托付给 `init` 进程, `init` 进程会持续调用 `wait`; 因此每个子进程都有一个父进程来负责清理。主要的实现挑战在于父子进程的 `wait` 与 `exit`

之间，以及 `exit` 与 `wait` 之间可能出现的竞争和死锁。

`wait` 使用调用进程的 `p->lock` 作为条件锁以避免丢失唤醒，并在开始时获取该锁（`kernel/proc.c:398`）。然后它扫描进程表。如果发现某个子进程处于 `ZOMBIE` 状态，则释放该子进程的资源及其 `proc` 结构，将子进程的退出状态复制到传给 `wait` 的地址（如果该地址不为 0），并返回子进程的进程 ID。如果 `wait` 找到了子进程但没有子进程已退出，则调用 `sleep` 等待其中某个子进程退出（`kernel/proc.c:445`），然后

再次扫描。这里，在 `sleep` 中被释放的条件锁是等待进程的

`p->lock`，即上面提到的特殊情况。注意，`wait`

通常同时持有两把锁；它在尝试获取任何子进程的锁之前会先获取自身的锁；因此，`xv6`

中所有代码必须遵循相同的加锁顺序（先父进程，再子进程），以避免死锁。

`wait` 通过查看每个进程的 `np->parent` 来找到其子进程。它在不持有 `np->lock` 的情况下访问 `np->parent`，这违反了共享变量必须由锁保护的一般规则。

`np` 有可能是当前进程的祖先进程，在这种情况下获取 `np->lock`

可能导致死锁，因为那会违反上述加锁顺序。在此情况下，不加锁地检查 `np->parent`

看起来是安全的；进程的 `parent` 字段只会被其父进程修改，因此如果 `np->parent==p`

为真，该值就不可能改变，除非当前进程自己去修改它。

`exit`（`kernel/proc.c:333`）记录退出状态，释放部分资源，将所有子进程托付给 `init` 进程，在父进程处于 `wait` 时将其唤醒，将调用者标记为僵尸进程，并永久让出 CPU。最后这一系列操作有些微妙。退出进程在将自身状态设置为 `ZOMBIE` 并唤醒父进程时，必须持有父进程的锁，因为父进程的锁是防止 `wait` 中丢失唤醒的条件锁。子进程还必须持有自身的

`p->lock`，否则父进程可能看到子进程处于 `ZOMBIE`

状态并在其仍在运行时将其释放。加锁顺序对于避免死锁至关重要：由于 `wait`

在获取子进程的锁之前先获取父进程的锁，`exit` 必须使用相同的顺序。`exit` 调用一个特殊的唤醒函数

`wakeup1`，该函数仅唤醒父进程，且仅在父进程正在 `wait`

中睡眠时才唤醒它（`kernel/proc.c:598`）。子进程在将自身状态设置为 `ZOMBIE`

之前就唤醒父进程，看起来可能有误，但这是安全的：尽管 `wakeup1` 可能使父进程开始运行，

但 `wait` 中的循环无法检查子进程，直到调度器释放子进程的 `p->lock`，

因此 `wait` 要等到 `exit` 将子进程状态设置为 `ZOMBIE` 之后很久才能查看该退出进程（`kernel/proc.c:386`）。

`exit` 允许进程终止自身，而 `kill`（`kernel/proc.c:611`）则允许一个进程请求终止另一个进程。让 `kill`

直接销毁受害进程过于复杂，因为受害进程可能正在另一个 CPU

上执行，也许正处于对内核数据结构进行敏感的一系列更新的中途。因此 `kill`

做的事情很少：它只是设置受害进程的

`p->killed`，如果受害进程正在睡眠则将其唤醒。最终受害进程会进入或离开内核，

此时 `usertrap` 中的代码会在 `p->killed` 被设置时调用 `exit`。如果受害进程正运行在

用户空间，它很快就会通过发起系统调用或因定时器（或其他设备）中断而进入内核。如果受害进程正在 `sleep` 中，`kill` 对 `wakeup` 的调用会使受害进程从 `sleep` 中返回。这存在潜在风险，因为所等待的条件可能尚未为真。然而，`xv6` 中对 `sleep` 的调用总是被包裹在一个 `while` 循环中，该循环在

`sleep` 返回后重新检测条件。部分对 `sleep` 的调用还会在循环中检测 `p->killed`，并在其被设置时放弃当前

活动。这种做法仅在放弃当前活动是正确的时候才会这样做。例如，管道的读写代码在 `killed` 标志被设置时会返回；最终代码会返回到陷阱处理程序，陷阱处理程序会再次检查该标志并调用 `exit`。

有些 `xv6` 的 `sleep` 循环不检查 `p->killed`，因为代码正处于一个应当原子执行的多

步系统调用的中途。`virtio` 驱动（`kernel/virtio_disk.c:242`）就是一个例子：它

不检查 `p->killed`，因为一次磁盘操作可能是一组写操作中的一个，这些写操作全部

完成才能使文件系统保持在正确的状态。一个在等待磁盘 I/O 时被 `kill` 的进程，要等到完成当前系统调用且 `usertrap` 看到 `killed` 标志后才会退出。

7.9 真实世界

`xv6` 调度器实现了一种简单的调度策略，依次运行每个进程。这种策略称为轮转调度（round robin）。真实的操作系统实现了更复杂的策略，例如允许进程拥有优先级。其核心思想是，调度器会优先选择可运行的高优先级进程，而非可运行的低优先级进程。这些策略很快就会变得复杂，因为往往存在相互竞争的目标：例如，操作系统可能还需要保证公平性和高吞吐量。此外，复杂的策略可能导致意想不到的交互问题，例如优先级反转（priority inversion）和护航现象（convoys）。当低优先级进程和高优先级进程共享一把锁时，就可能发生优先级反转——低优先级进程持有锁时，高优先级进程将无法继续执行。当许多高优先级进程等待某个持有共享锁的低优先级进程时，就会形成一条长长的等待进程护航队列；护航队列一旦形成，可能会持续很长时间。为了避免此类问题，复杂的调度器中需要引入额外的机制。

睡眠与唤醒是一种简单而有效的同步方法，但类似的方法还有很多。所有这些方法面临的首要挑战，都是如何避免本章开头所提到的“丢失唤醒”问题。原始 Unix 内核的 `sleep` 仅通过禁用中断来解决这一问题，这在 Unix 运行于单 CPU 系统时已经足够。由于 `xv6` 运行在多处理器上，它为 `sleep` 增加了一个显式锁。FreeBSD 的 `msleep` 采用了相同的方法。Plan 9 的 `sleep` 使用一个回调函数，该函数在进入睡眠前持有调度锁时运行，用于在最后时刻检查睡眠条件，以避免丢失唤醒。Linux 内核的 `sleep` 使用一个称为等待队列（wait queue）的显式进程队列，而非等待通道（wait channel）；该队列拥有自己的内部锁。

在 `wakeup` 中扫描整个进程列表以寻找匹配 `chan` 的进程效率低下。更好的解决方案是将 `sleep` 和 `wakeup` 中的 `chan` 替换为一种数据结构，该结构持有在其上睡眠的进程列表，类似于 Linux 的等待队列。Plan 9 的 `sleep` 和 `wakeup` 将该结构称为汇合点（rendezvous point）或

Rendez。许多线程库将相同的结构称为条件变量（condition variable）；在该上下文中，`sleep` 和 `wakeup` 操作分别称为 `wait` 和 `signal`。所有这些机制都具有相同的特点：睡眠条件由某种锁保护，并在睡眠期间以原子方式释放。

`wakeup` 的实现会唤醒所有在特定通道上等待的进程，而等待该通道的进程可能有很多。操作系统将调度所有这些进程，它们将竞相检查睡眠条件。以这种方式运行的进程有时被称为惊群（thundering herd），应当尽量避免。大多数条件变量提供两种唤醒原语：`signal`（唤醒一个进程）和 `broadcast`（唤醒所有等待进程）。

信号量（semaphore）常用于同步。其计数值通常对应于某种具体含义，例如管道缓冲区中可用的字节数，或某进程拥有的僵尸子进程数量。将显式计数作为抽象的一部分，可以避免“丢失唤醒”问题：唤醒发生的次数有明确的计数记录。计数机制还能避免虚假唤醒（spurious wakeup）和惊群问题。

终止进程并对其进行清理在 xv6 中引入了相当多的复杂性。在大多数操作系统中，这一过程更为复杂，例如，目标进程可能深陷内核睡眠之中，展开其调用栈需要大量谨慎的编程工作。许多操作系统使用异常处理的显式机制（例如 `longjmp`）来展开调用栈。此外，还存在其他可能导致睡眠进程被唤醒的事件，即便其等待的事件尚未发生。例如，当一个 Unix 进程正在睡眠时，另一个进程可能向其发送信号。在这种情况下，该进程将从被中断的系统调用中返回，返回值为 `-1`，并将错误码设置为 `EINTR`。应用程序可以检查这些值并决定如何处理。xv6 不支持信号，因此不存在这一复杂性。

xv6 对 `kill` 的支持并不完全令人满意：有些睡眠循环可能应当检查

`p->killed`。一个相关的问题是，即使对于会检查 `p->killed` 的睡眠循环，`sleep` 与 `kill` 之间仍存在竞争条件；`kill` 可能在目标进程的循环检查完 `p->killed` 之后、调用 `sleep` 之前，就已设置 `p->killed`

并尝试唤醒目标进程。若出现这种情况，目标进程直到其等待的条件满足时才会注意到 `p->killed`。这可能要等很长时间（例如，等到 `virtio` 驱动返回目标进程正在等待的磁盘块时），甚至永远不会发生（例如，若目标进程正在等待来自控制台的输入，但用户始终未输入任何内容）。

真实的操作系统会使用显式的空闲列表，以常数时间找到空闲的 `proc` 结构体，而非像 `allocproc` 中那样进行线性时间搜索；xv6 为了简洁起见采用了线性扫描。

7.10 练习

1. `sleep` 必须检查 `lk != &p->lock` 以避免死锁（kernel/proc.c:558-561）。假设

通过将以下代码

```
if(lk != &p->lock){
    acquire(&p->lock);
    release(lk);
```

```
}
```

替换为

```
release(lk);  
acquire(&p->lock);
```

来消除这个特殊情况。这样做会破坏 sleep。为什么？

2. 大多数进程清理工作既可以由 exit 完成，也可以由 wait

完成。但事实证明，关闭已打开文件的工作必须由 exit 来完成。为什么？答案与管道有关。

3. 在 xv6 中不使用 sleep 和 wakeup 来实现信号量（但可以使用自旋锁）。用信号量替换 xv6 中 sleep 和 wakeup 的所有使用。评估结果。

4. 修复上述 kill 与 sleep 之间的竞态条件，使得在受害进程的 sleep 循环检查 p->killed 之后、调用 sleep 之前发生的 kill，能够导致受害进程放弃当前系统调用。

5. 设计一个方案，使每个 sleep 循环都检查 p->killed，从而使例如处于 virtio

驱动中的进程在被另一个进程杀死时能够快速从 while 循环中返回。

6. 修改 xv6，使得从一个进程的内核线程切换到另一个进程的内核线程时只进行一次上下文切换，而不是通过调度器线程中转切换。让出 CPU 的线程需要自行选择下一个线程并调用 swtch。挑战在于：防止多个核心意外执行同一线程；正确处理锁；以及避免死锁。

7. 修改 xv6 的调度器，在没有可运行进程时使用 RISC-V 的 WFI (wait for interrupt) 指令。尽量确保任何时候只要有可运行的进程等待运行，就不会有核心停留在 WFI 中。

8. 锁 p->lock 保护着许多不变量，当查看 xv6 中受 p->lock 保护的某段特定代码时，

很难弄清楚正在强制执行的是哪个不变量。设计一个更清晰的方案，将 p->lock

拆分为多个锁。

第 8 章：文件系统

文件系统的目的是组织和存储数据。文件系统通常支持用户和应用程序之间的数据共享，以及持久性，使得数据在重启后仍然可用。xv6 文件系统提供了类 Unix 的文件、目录和路径名（见第 1 章），并将数据存储于 virtio 磁盘上以实现持久性（见第 4 章）。文件系统需要应对若干挑战：

- 文件系统需要磁盘上的数据结构来表示由命名目录和文件组成的树形结构，记录保存每个文件内容的块的标识，以及记录磁盘中哪些区域是空闲的。
- 文件系统必须支持崩溃恢复。也就是说，如果发生崩溃（例如断电），文件系统在重启后必须仍能正常工作。风险在于，崩溃可能会中断一系列更新操作，留下不一致的磁盘数据结构（例如，某个块既被文件使用，又被标记为空闲）。
- 不同的进程可能同时操作文件系统，因此文件系统代码必须进行协调以维护不变量。
- 访问磁盘比访问内存慢几个数量级，因此文件系统必须在内存中维护常用块的缓存。

本章其余部分将解释 xv6 如何应对这些挑战。

8.1 概述

xv6 文件系统的实现分为七层，如图 8.1 所示。磁盘层负责在 virtio 硬盘上读写数据块。缓冲区缓存层对磁盘块进行缓存并同步对其的访问，确保任意时刻只有一个内核进程能够修改存储在某个特定块中的数据。日志层允许上层将对若干块的更新封装在一个事务中，并确保在发生崩溃时这些块能够原子性地完成更新。

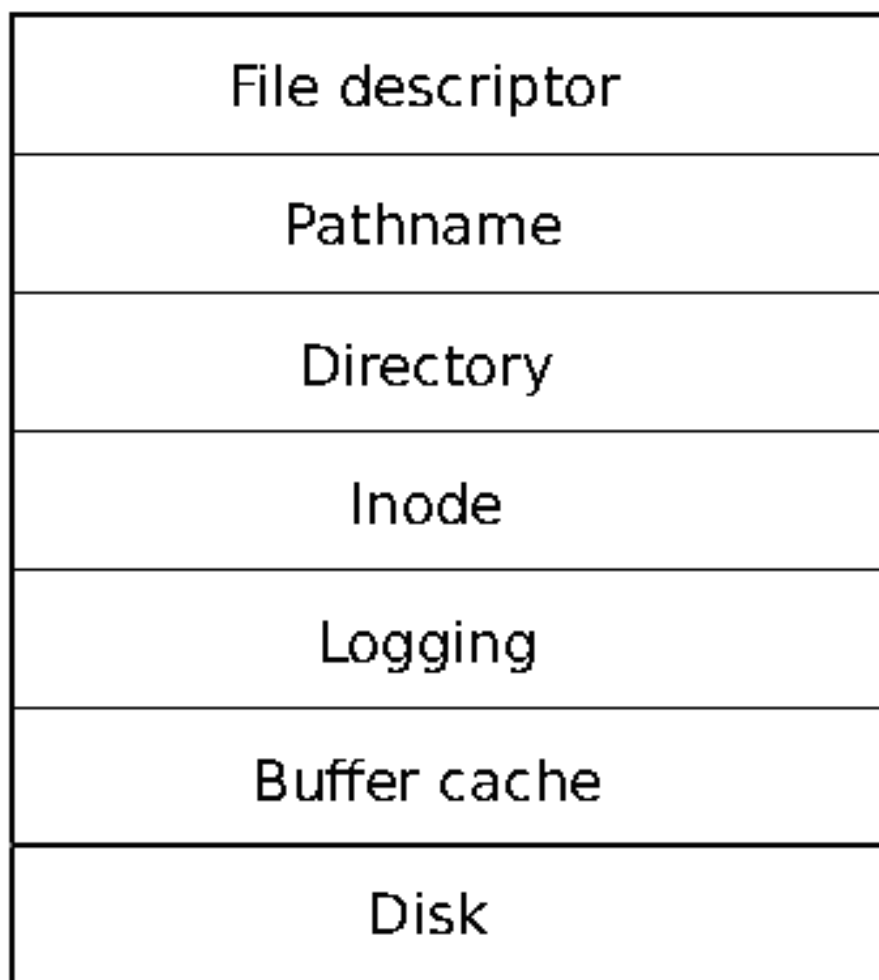


图 8.1：xv6 文件系统的各层结构。

（即要么全部更新，要么全部不更新）。inode 层提供单个文件的抽象，每个文件以一个 inode 表示，具有唯一的 i-number 以及若干存储文件数据的数据块。目录层将每个目录实现为一种特殊的 inode，其内容是一系列目录项，每个目录项包含一个文件的名称和 i-number。路径名层提供类似 `/usr/rtn/xv6/fs.c` 这样的层级路径名，并通过递归查找来解析它们。文件描述符层使用文件系统接口对众多 Unix 资源（例如管道、设备、文件等）进行抽象，简化了应用程序开发者的工作。

文件系统必须规划 inode 和内容块在磁盘上的存储位置。为此，xv6 将磁盘划分为若干区域，如图 8.2 所示。文件系统不使用块 0（该块保存引导扇区）。块 1 称为超级块（superblock），其中包含文件系统的元数据（文件系统的块大小、数据块数量、inode 数量以及日志中的块数量）。从块 2 开始存放日志。日志之后是 inode，每个块中存放多个 inode。inode 块之后是位图块，用于追踪哪些数据块正在使用中。其余的块为数据块，每个数据块要么在位图块中被标记为空闲，要么存储某个文件或目录的内容。

超级块由一个名为 `mkfs` 的独立程序负责填写，该程序用于构建初始文件系统。本章其余部分将依次讨论每一层，从缓冲区缓存开始。请留意下层精心选择的抽象如何简化上层的设计。

8.2 缓冲区缓存层

缓冲区缓存有两项职责：(1) 同步对磁盘块的访问，以确保内存中每个块只有一份副本，且每次只有一个内核线程使用该副本；(2) 缓存常用块，使其无需从速度较慢的磁盘重新读取。相关代码位于 `bio.c` 中。缓冲区缓存对外导出的主要接口包括 `bread` 和 `bwrite`；前者获取一个包含某个块副本的 `buf`，该副本可在内存中被读取或修改，后者则将已修改的缓冲区写入磁盘上对应的块。内核线程在使用完缓冲区后，必须调用 `brelse` 将其释放。缓冲区缓存使用每个缓冲区独立的睡眠锁来确保

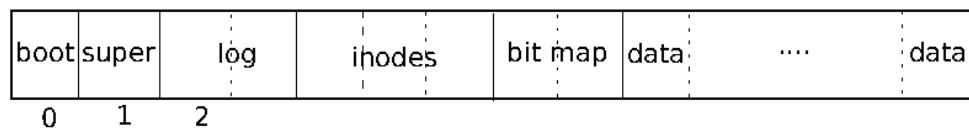


图 8.2：xv6 文件系统的结构。

每次只有一个线程使用每个缓冲区（从而每个磁盘块）；`bread` 返回一个已加锁的缓冲区，`brelse` 则释放该锁。让我们回到缓冲区缓存。缓冲区缓存中用于存放磁盘块的缓冲区数量是固定的，这意味着如果文件系统请求的块尚未存在于缓存中，缓冲区缓存必须回收一个当前持有其他块的缓冲区。缓冲区缓存会将最近最少使用的缓冲区回收给新块使用。其假设前提是：最近最少使用的缓冲区，是最不可能在近期再次被使用的缓冲区。

8.3 代码：缓冲区缓存

缓冲区缓存是一个由缓冲区组成的双向链表。函数 `binit` 由 `main` (`kernel/main.c:27`) 调用，使用静态数组 `buf` 中的 `NBUF` 个缓冲区初始化该链表 (`kernel/bio.c:43-52`)。所有其他对缓冲区缓存的访问均通过 `bcache.head` 引用链表，而非直接访问 `buf` 数组。每个缓冲区有两个与之关联的状态字段。字段 `valid` 表示该缓冲区包含块的一份副本。字段 `disk` 表示缓冲区内容已移交给磁盘，磁盘可能会修改该缓冲区（例如，将磁盘数据写入 `data`）。`bread` (`kernel/bio.c:93`) 调用 `bget` 为给定扇区获取一个缓冲区 (`kernel/bio.c:97`)。如果该缓冲区需要从磁盘读取，`bread` 会在返回缓冲区之前调用 `virtio_disk_rw` 来完成读取。`bget` (`kernel/bio.c:59`) 扫描缓冲区链表，查找具有给定设备号和扇区号的缓冲区 (`kernel/bio.c:65-73`)。如果存在这样的缓冲区，`bget` 会获取该缓冲区的睡眠锁，然后返回已加锁的缓冲区。如果给定扇区没有对应的缓存缓冲区，`bget` 必须创建一个，这可能需要复用了一个保存了不同扇区数据的缓冲区。它会再次扫描缓冲区链表，寻找一个当前未被使用的缓冲区


```
(b->refcnt = 0); any such buffer can be used. Bget edits the buffer metadata to record the new device and sector number and acquires its sleep-lock. Note that the assignment b->valid = 0
```

确保 **bread** 会从磁盘读取块数据，而不是错误地使用缓冲区中原有的内容。每个磁盘扇区最多只能有一个对应的缓存缓冲区，这一点非常重要，既是为了确保读者能看到写入的内容，也是因为文件系统使用缓冲区上的锁来进行同步。**bget**

通过在第一个循环检查块是否已被缓存、到第二个循环声明块已被缓存（通过设置 **dev**、**blockno** 和 **refcnt**）的整个过程中持续持有 **bcache.lock**，来维护这一不变性。这使得检查块是否存在，以及（若不存在时）指定一个缓冲区来保存该块这两个操作成为原子操作。**bget** 在 **bcache.lock** 临界区之外获取缓冲区的睡眠锁是安全的，

```
section, since the non-zero b->refcnt prevents the buffer from being re-used for a different
```

磁盘块。睡眠锁保护块的缓冲内容的读写操作，而 **bcache.lock** 则保护关于哪些块被缓存的元信息。如果所有缓冲区都处于忙碌状态，则说明有太多进程在同时执行文件系统调用；此时 **bget** 会触发 **panic**。一种更优雅的处理方式是睡眠等待直到某个缓冲区空闲，但这样可能会引发死锁。一旦 **bread** 完成了磁盘读取（如有必要）并将缓冲区返回给调用者，调用者便独占该缓冲区，可以读取或写入其中的数据字节。如果调用者修改了缓冲区，则必须在释放缓冲区之前调用

bwrite，将修改后的数据写入磁盘。**bwrite** (kernel/bio.c:107) 调用 **virtio_disk_rw**

与磁盘硬件进行通信。当调用者使用完缓冲区后，必须调用 **brelease** 来释放它。（**brelease** 这个名称是 **b-release** 的缩写，虽然晦涩，但值得记忆：它起源于 Unix，并在 BSD、Linux 和 Solaris 中沿用至今。）**brelease** (kernel/bio.c:117) 释放睡眠锁，并将该缓冲区移动到链表的头部 (kernel/bio.c:128-133)。移动缓冲区使链表按缓冲区最近被使用（即被释放）的时间顺序排列：链表中第一个缓冲区是最近使用的，最后一个是最久未使用的。**bget** 中的两个循环充分利用了这一特性：在最坏情况下，查找已有缓冲区的扫描需要遍历整个链表，但优先检查最近使用的缓冲区（从 **bcache.head** 出发，沿 **next** 指针前进）可以在引用局部性良好的情况下缩短扫描时间。选取待复用缓冲区的扫描则通过反向遍历（沿 **prev** 指针前进）来选出最久未使用的缓冲区。

8.4 日志层

文件系统设计中最有趣的问题之一是崩溃恢复。该问题的根源在于，许多文件系统操作涉及对磁盘的多次写入，而在部分写入完成后发生崩溃，可能会使磁盘上的文件系统处于不一致的状态。例如，假设在文件截断（将文件长度设为零并释放其内容块）期间发生崩溃。根据磁盘写入的顺序，崩溃可能导致一个 **inode** 引用了一个被标记为空闲的内容块，也可能留下一个已分配但未被引用的内容块。后者相对无害，但一个引用了已释放块的 **inode** 在重启后很可能引发严重问题。重启后，内核可能将该块分配给另一个文件，这样就出现了两个不同的文件无意间指向同一个块的情况。如果 **xv6** 支持多用户，这种情况还可能构成安全问题，因为旧文件的所有者将能够读写属于不同用户的新文件中的块。

Xv6 通过一种简单的日志形式来解决文件系统操作期间的崩溃问题。**xv6** 的系统调用不直接写入磁盘上的文件系统数据结构，而是将其希望执行的所有磁盘写入的描述记录在磁盘上的一个日志中。一旦系统调用将所有写入都记录到日志中，它就会向磁盘写入一条特殊的提交记录，表明该日志包含一个完整的操作。

此时，系统调用再将这些写入复制到磁盘上的文件系统数据结构中。在这些写入完成后，系统调用会擦除磁盘上的日志。

如果系统崩溃并重启，文件系统代码会在运行任何进程之前按如下方式从崩溃中恢复：如果日志被标记为包含一个完整的操作，则恢复代码将这些写入复制到磁盘文件系统中它们应在的位置；如果日志未被标记为包含完整操作，则恢复代码忽略该日志。恢复代码最终通过擦除日志来完成恢复。

为什么 xv6 的日志能解决文件系统操作期间的崩溃问题？如果崩溃发生在操作提交之前，磁盘上的日志将不会被标记为完整，恢复代码会忽略它，磁盘的状态将与该操作从未开始时相同。如果崩溃发生在操作提交之后，恢复过程将重放该操作的所有写入，如果该操作已经开始将它们写入磁盘数据结构，则可能会重复执行这些写入。无论哪种情况，日志都使操作在崩溃方面具有原子性：恢复完成后，该操作的所有写入要么全部出现在磁盘上，要么一个都不出现。

8.5 日志设计

日志位于超级块中指定的一个已知固定位置。它由一个头块和一系列已更新块的副本（“日志块”）组成。头块包含一个扇区号数组，每个日志块对应一个扇区号，以及日志块的计数。磁盘上头块中的计数要么为零，表示日志中没有事务；要么不为零，表示日志中包含一个已完整提交的事务，并记录了对应数量的日志块。Xv6 在事务提交时写入头块，但在此之前不会写入，并在将日志块复制到文件系统之后将计数清零。因此，在事务中途发生的崩溃将导致日志头块中的计数为零；而在提交之后发生的崩溃将导致计数不为零。每个系统调用的代码会标明必须相对于崩溃保持原子性的写操作序列的开始和结束。为了允许不同进程并发执行文件系统操作，日志系统可以将多个系统调用的写操作累积到一个事务中。因此，一次单独的提交可能涉及多个完整系统调用的写操作。为了避免将一个系统调用拆分到多个事务中，日志系统仅在没有文件系统系统调用正在进行时才执行提交。将多个事务合并提交的思想被称为组提交（group commit）。组提交通过将提交的固定开销分摊到多个操作上，从而减少了磁盘操作次数。组提交还能同时向磁盘系统提交更多并发写请求，或许可以让磁盘在一次磁盘旋转期间完成所有写入。Xv6 的 virtio 驱动不支持这种批处理方式，但 xv6 的文件系统设计允许这样做。Xv6 在磁盘上分配了固定大小的空间来存储日志。一个事务中所有系统调用写入的块总数必须能够容纳在该空间内。这带来了两个后果。不允许任何单个系统调用写入超过日志空间大小的不同块数。对于大多数系统调用来说这不是问题，但其中两个可能会写入大量块：`write` 和 `unlink`。一次大文件写操作可能会写入许多数据块、许多位图块以及一个 inode 块；而删除一个大文件可能会写入许多位图块和一个 inode。Xv6 的 `write` 系统调用会将大写操作拆分为多个能够放入日志的较小写操作，而 `unlink` 不会造成问题，因为在实践中 xv6 文件系统只使用一个位图块。日志空间有限的另一个后果是，日志系统不允许一个系统调用启动，除非它确定该系统调用的写操作能够放入日志中剩余的空间。

8.6 代码：日志

在系统调用中使用日志的典型方式如下：

```
begin_op();
```

...

```
bp = bread(...);
bp->data[...] = ...;
log_write(bp);
```

...

```
end_op();
```

begin_op (kernel/log.c:126) 会等待，直到日志系统当前没有在提交，并且有足够的未预留日志空间来容纳本次调用的写入。**log.outstanding** 记录了已预留日志空间的系统调用数量；总预留空间为 **log.outstanding** 乘以 **MAXOPBLOCKS**。递增 **log.outstanding** 既预留了空间，又防止了在本次系统调用期间发生提交。代码保守地假设每个系统调用最多可能写入 **MAXOPBLOCKS** 个不同的块。**log_write** (kernel/log.c:214) 充当 **bwrite** 的代理。它将块的扇区号记录在内存中，在磁盘上的日志中为其预留一个槽位，并将缓冲区固定在块缓存中，以防止块缓存将其驱逐。该块必须保留在缓存中直到提交：在此之前，缓存中的副本是该修改的唯一记录；它在提交之前不能被写入磁盘上的对应位置；同一事务中的其他读取操作必须能看到这些修改。**log_write** 会注意到某个块在单次事务中被多次写入，并为该块分配日志中的同一个槽位。这种优化通常称为吸收 (absorption)。一个常见的情况是，包含多个文件索引节点的磁盘块在一次事务中被多次写入。通过将多次磁盘写入合并为一次，文件系统可以节省日志空间，并获得更好的性能，因为只需将该磁盘块的一个副本写入磁盘。**end_op** (kernel/log.c:146) 首先递减未完成系统调用的计数。如果计数此时变为零，则通过调用 **commit()** 提交当前事务。该过程分为四个阶段。**write_log()** (kernel/log.c:178) 将事务中修改的每个块从缓冲区缓存复制到其在磁盘日志中的槽位。**write_head()** (kernel/log.c:102) 将头块写入磁盘：这是提交点，写入之后发生崩溃将导致恢复时从日志中重放该事务的写入。**install_trans** (kernel/log.c:69) 从日志中读取每个块，并将其写入文件系统中的正确位置。最后，**end_op** 将日志头中的计数写为零；这必须在下一个事务开始写入已记录的块之前完成，以防止崩溃后恢复时将某个事务的头与后续事务已记录的块混用。**recover_from_log** (kernel/log.c:116) 由 **initlog** (kernel/log.c:55) 调用，而 **initlog** 在启动期间由 **fsinit** (kernel/fs.c:42) 调用，时间在第一个用户进程运行之前 (kernel/proc.c:539)。它读取日志头，如果头部表明日志中包含一个已提交的事务，则模拟 **end_op** 的操作。

日志使用的一个示例出现在 **filewrite** (kernel/file.c:135) 中。该事务如下所示：

```
begin_op();
ilock(f->ip);
r = writei(f->ip, ...);
iunlock(f->ip);
end_op();
```

这段代码被包裹在一个循环中，该循环将大型写入拆分为每次仅涉及少数几个扇区的独立事务，以避免日志溢出。对 **writei** 的调用在该事务中写入了许多块：文件的索引节点、一个或多个位图块，以及若干数据块。

8.7 代码：块分配器

文件和目录的内容存储在磁盘块中，这些磁盘块必须从空闲池中分配。xv6 的块分配器在磁盘上维护一个空闲位图，每个块对应一个位。零位表示对应的块是空闲的；一位表示该块正在使用中。程序 `mkfs` 设置与引导扇区、超级块、日志块、inode 块和位图块对应的位。块分配器提供两个函数：`balloc` 分配一个新的磁盘块，`bfree` 释放一个块。

`balloc` 在 (`kernel/fs.c:71`) 中的循环会考虑每一个块，从块 0 开始，直到 `sb.size`（即文件系统中块的总数）。它寻找位图位为零的块，表示该块是空闲的。如果 `balloc` 找到这样的块，它会更新位图并返回该块。为了提高效率，该循环被分成两部分。外层循环读取位图位的每个块。内层循环检查单个位图块中所有 BPB 个位。如果两个进程同时尝试分配一个块，可能发生的竞争条件被缓冲区缓存所防止——缓冲区缓存在同一时刻只允许一个进程使用任意一个位图块。

`bfree` (`kernel/fs.c:90`) 找到正确的位图块并清除相应的位。同样，`bread` 和 `brelease` 所隐含的独占使用避免了显式加锁的需要。与本章其余部分描述的大多数代码一样，`balloc` 和 `bfree` 必须在一个事务内被调用。

8.8 Inode 层

术语 inode

有两种相关含义。它可以指磁盘上的数据结构，其中包含文件的大小和数据块编号列表。或者，“inode”也可以指内存中的 inode，它包含磁盘上 inode 的副本以及内核所需的额外信息。磁盘上的 inode 被紧密排列在磁盘的一段连续区域中，称为 inode 块。每个 inode 的大小相同，因此给定编号 `n`，很容易在磁盘上找到第 `n` 个 inode。事实上，这个编号 `n` 被称为 inode 编号或 `i-number`，是实现中标识 inode 的方式。磁盘上的 inode 由 `struct dinode` (`kernel/fs.h:32`) 定义。`type` 字段用于区分文件、目录和特殊文件（设备）。`type` 为零表示该磁盘上的 inode 是空闲的。`nlink` 字段统计引用该 inode 的目录项数量，以便识别何时应释放磁盘上的 inode 及其数据块。`size` 字段记录文件内容的字节数。`addrs` 数组记录保存文件内容的磁盘块的块编号。

内核将活跃 inode 的集合保存在内存中；`struct inode` (`kernel/file.h:17`) 是磁盘上 `struct dinode` 的内存副本。内核仅在存在指向某个 inode 的 C 指针时才将其存储在内存中。`ref` 字段统计指向内存中 inode 的 C 指针数量，当引用计数降为零时，内核将该 inode 从内存中丢弃。`iget` 和 `iput` 函数用于获取和释放指向 inode 的指针，并相应修改引用计数。指向 inode 的指针可以来自文件描述符、当前工作目录以及诸如 `exec` 之类的瞬态内核代码。

xv6 的 inode 代码中存在四种锁或类锁机制。`icache.lock` 保护以下不变式：一个 inode 在缓存中最多出现一次，以及缓存中 inode 的 `ref` 字段统计指向该缓存 inode 的内存指针数量。每个内存中的 inode 都有一个包含睡眠锁的 `lock` 字段，用于确保对 inode 字段（如文件长度）以及 inode 的文件或目录内容块的独占访问。如果 inode 的 `ref` 大于零，系统将把该

inode 保留在缓存中，不会将缓存项复用给其他 inode。最后，每个 inode 包含一个 `nlink` 字段（存在于磁盘上，若已缓存则复制到内存中），用于统计引用该文件的目录项数量；若 inode 的链接计数大于零，xv6 不会释放该 inode。

由 `iget()` 返回的 `struct inode` 指针保证在对应的 `iput()` 调用之前始终有效；该 inode 不会被删除，指针所引用的内存也不会被复用给其他 inode。`iget()` 提供对 inode 的非独占访问，因此可以有多个指针指向同一个 inode。文件系统代码的许多部分都依赖于 `iget()` 的这一行为，既用于长期持有对 inode 的引用（如打开的文件和当前目录），也用于在操作多个 inode 时（如路径名查找）避免死锁的情况下防止竞争。

`iget` 返回的 `struct inode` 可能不包含任何有效内容。为了确保它持有磁盘上 inode 的副本，代码必须调用 `ilock`。这将锁定该 inode（使其他进程无法对其调用 `ilock`），并在尚未读取的情况下从磁盘读取该 inode。`iunlock` 释放对 inode 的锁定。将获取 inode 指针与锁定分离，有助于在某些情况下避免死锁，例如在目录查找期间。多个进程可以持有指向由 `iget` 返回的同一 inode 的 C 指针，但同一时刻只有一个进程可以锁定该 inode。

inode 缓存仅缓存内核代码或数据结构持有 C 指针的 inode。其主要职责实际上是同步多个进程的访问；缓存是次要功能。如果某个 inode 被频繁使用，即使 inode 缓存不再持有它，缓冲区缓存也可能将其保留在内存中。inode 缓存是写透式（write-through）的，这意味着修改缓存中 inode 的代码必须立即通过 `iupdate` 将其写入磁盘。

8.9 代码：Inode

要分配一个新的 inode（例如在创建文件时），xv6 会调用 `ialloc`（kernel/fs.c:196）。`Ialloc` 与 `balloc` 类似：它逐块遍历磁盘上的 inode 结构，寻找标记为空闲的那一个。当找到后，它通过向磁盘写入新的类型来声明该 inode，然后通过尾调用 `iget`（kernel/fs.c:210）从 inode 缓存中返回一个条目。`ialloc` 能够正确运行，依赖于以下事实：同一时刻只有一个进程能持有对 `bp` 的引用，因此 `ialloc` 可以确保不会有其他进程同时发现该 inode 可用并尝试声明它。

```
Iiget (kernel/fs.c:243) looks through the inode cache for an active entry (ip->ref > 0) with
```

所需设备和 inode 编号。如果找到，则返回对该 inode 的一个新引用（kernel/fs.c:252-256）。在扫描过程中，`iget` 会记录第一个空槽的位置（kernel/fs.c:257-258），以便在需要分配缓存条目时使用。在读写 inode 的元数据或内容之前，代码必须使用 `ilock` 对 inode 加锁。`Ilock`（kernel/fs.c:289）为此目的使用了睡眠锁。一旦 `ilock` 获得对 inode 的独占访问权，就会在必要时从磁盘（更可能是从缓冲区缓存）读取 inode。函数 `iunlock`（kernel/fs.c:317）释放睡眠锁，这可能会唤醒正在睡眠的进程。`Iput`（kernel/fs.c:333）通过递减引用计数（kernel/fs.c:356

）来释放对 inode 的 C 指针。如果这是最后一个引用，则该 inode 在 inode 缓存中的槽位现在变为空闲，可以被其他 inode 重用。如果 `iput` 发现没有任何 C 指针引用该 inode，且该 inode 没有任何指向它的链接（即不存在于任何目录中），则必须释放该 inode 及其数据块。`iput` 调用 `itrunc` 将文件截断为零字节，释放数据块；将 inode 类型设置为 0（未分配）；并将 inode 写入磁盘（`kernel/fs.c:338`）。

`iput` 在释放 inode 时所采用的加锁协议值得仔细研究。一个潜在的危险是，某个并发线程可能正在 `ilock` 中等待使用该 inode（例如，读取文件或列出目录内容），并且不会预料到该 inode 已不再被分配。但这种情况不会发生，因为如果一个 inode 没有链接且 `ip->ref` 为 1，系统调用就无法获得指向该已缓存 inode 的指针。

```
to it and ip->ref is one. That one reference is the reference owned by the thread calling iput. It's
```

确实，`iput` 在 `icache.lock` 临界区之外检查引用计数是否为 1，但在那时已知链接计数为零，因此不会有线程试图获取新引用。另一个主要危险是，并发的 `ialloc` 调用可能选中 `iput` 正在释放的同一个 inode。这只可能在 `iupdate` 将 inode 类型写为零后才能发生。这种竞争是无害的；分配线程在读写 inode 之前会礼貌地等待获取该 inode 的睡眠锁，而此时 `iput` 已经完成对它的操作。

`iput()` 可能会写磁盘。这意味着任何使用文件系统的系统调用都可能写磁盘，因为该系统调用可能是最后一个持有文件引用的调用。即使是看似只读的调用（如 `read()`），也可能最终调用 `iput()`。这反过来意味着，即使是只读的系统调用，如果使用了文件系统，也必须包裹在事务中。

`iput()` 与崩溃之间存在一个棘手的交互问题。当文件的链接计数降至零时，`iput()` 不会立即截断文件，因为某些进程可能仍在内存中持有对该 inode 的引用：某个进程可能仍在读写该文件，因为它已成功打开了它。但是，如果在最后一个进程关闭该文件的文件描述符之前发生崩溃，

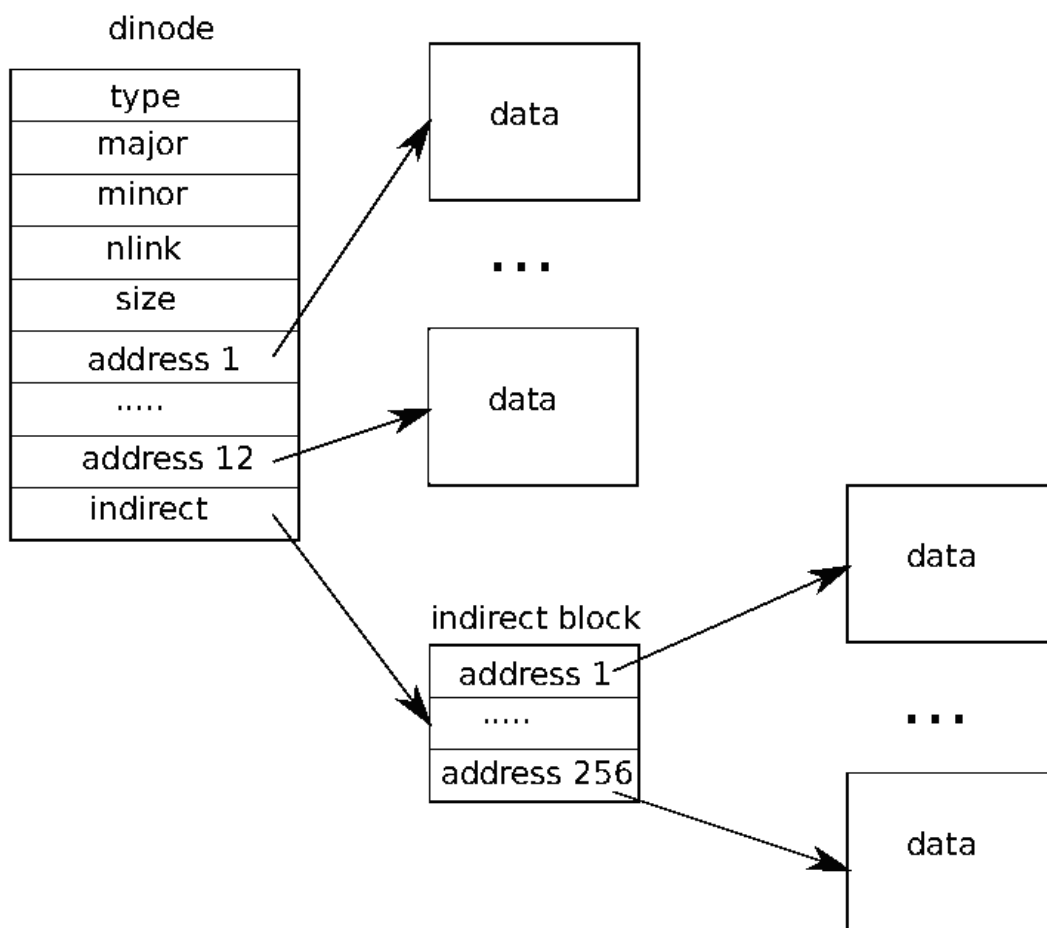


图 8.3：文件在磁盘上的表示形式。

则该文件在磁盘上会被标记为已分配，但没有任何目录条目指向它。文件系统通常以两种方式处理这种情况。简单的解决方案是：在重启恢复后，文件系统扫描整个文件系统，查找那些被标记为已分配但没有目录条目指向的文件，如果存在则释放它们。第二种方案不需要扫描文件系统：文件系统将链接计数降至零但引用计数不为零的文件的 inode 编号记录在磁盘上（例如在超级块中）。当文件的引用计数降为 0 时，文件系统删除该文件，并通过从列表中移除该 inode 来更新磁盘上的列表。在恢复时，文件系统释放列表中的所有文件。

Xv6 两种方案都未实现，这意味着 inode 可能在磁盘上被标记为已分配，即便它们实际上已不再使用。这意味着随着时间推移，xv6 面临耗尽磁盘空间的风险。

8.10 代码：inode 内容

磁盘上的 inode 结构体 `struct dinode` 包含一个大小字段和一个块号数组（见图 8.3）。inode 的数据存储在 `dinode` 的 `addrs` 数组所列出的块中。前 `NDIRECT` 个数据块列在数组的前 `NDIRECT` 个条目中；这些块称为

直接块。接下来的 `NINDIRECT` 个数据块不是列在 inode 中，而是列在一个称为间接块的数据块中。`addrs` 数组的最后一个条目给出了间接块的地址。因此，文件的前 12 kB (`NDIRECT x BSIZE`) 字节可以从 inode 中列出的块加载，而接下来的 256 kB (`NINDIRECT x BSIZE`) 字节只能在查阅间接块之后才能加载。这是一种良好的磁盘表示方式，但对客户端来说较为复杂。函数 `bmap` 负责管理这种表示，以便高层例程（如我们稍后将看到的 `readi` 和 `writei`）使用。`Bmap` 返回 inode `ip` 的第 `bn` 个数据块的磁盘块号。如果 `ip` 还没有这样的块，`bmap` 会分配一个。函数 `bmap` (`kernel/fs.c:378`) 首先处理简单情况：前 `NDIRECT` 个块直接列在 inode 自身中 (`kernel/fs.c:383-387`)。接下来的 `NINDIRECT` 个块列在

位于 `ip->addrs[NDIRECT]` 的间接块中。`Bmap` 读取间接块 (`kernel/fs.c:394`)，

然后从块内的正确位置读取块号 (`kernel/fs.c:395`)。如果块号超过 `NDIRECT+NINDIRECT`，`bmap` 会触发 `panic`；`writei` 中包含防止这种情况发生的检查 (`kernel/fs.c:490`)。

`Bmap` 按需分配块。`ip->addrs[]` 或间接块条目为零表示尚未分配块。

当 `bmap` 遇到零时，会将其替换为按需分配的新块的块号 (`kernel/fs.c:384-385`) (`kernel/fs.c:392-393`)。`itrunc` 释放文件的所有块，并将 inode 的大小重置为零。`itrunc` (`kernel/fs.c:410`) 首先释放直接块 (`kernel/fs.c:416-421`)，然后释放间接块中列出的块 (`kernel/fs.c:426-429`)，最后释放间接块本身 (`kernel/fs.c:431-432`)。`Bmap` 使 `readi` 和 `writei` 能够方便地访问 inode 的数据。`Readi` (`kernel/fs.c:456`) 首先确保偏移量和字节数不超过文件末尾。从文件末尾之后开始的读操作返回错误 (`kernel/fs.c:461-462`)，而从文件末尾处开始或跨越文件末尾的读操作返回的字节数少于请求的字节数 (`kernel/fs.c:463-464`)。主循环处理文件的每个块，将数据从缓冲区复制到 `dst` (`kernel/fs.c:466-474`)。`writei` (`kernel/fs.c:483`) 与 `readi` 基本相同，但有三点不同：从文件末尾处开始或跨越文件末尾的写操作会增大文件，直至达到最大文件大小 (`kernel/fs.c:490-491`)；循环将数据复制进缓冲区而非从缓冲区复制出 (`kernel/fs.c:36`)；如果写操作扩展了文件，`writei` 必须更新文件大小 (`kernel/fs.c:504-511`)。

`readi` 和 `writei` 都首先检查 `ip->type == T_DEV`。该情况处理数据不存储在

文件系统上的特殊设备；我们将在文件描述符层再回到这一情况。函数 `stat` (`kernel/fs.c:442`) 将 inode 元数据复制到 `stat` 结构体中，该结构体通过 `stat` 系统调用暴露给用户程序。

8.11 代码：目录层

目录在内部的实现方式与文件非常相似。其 inode 的类型为 `T_DIR`，数据是一系列目录项。每个目录项是一个 `struct dirent`（kernel/fs.h:56），包含一个名称和一个 inode 编号。名称最多为 `DIRSIZ`（14）个字符；如果更短，则以 `NUL`（0）字节结尾。inode 编号为零的目录项表示空闲。函数 `dirlookup`（kernel/fs.c:527）在目录中搜索具有给定名称的目录项。

如果找到，它返回一个指向对应 inode 的指针（未加锁状态），并将 `*poff` 设置为该目录项在目录中的字节偏移量，以便调用者在需要时对其进行编辑。如果 `dirlookup` 找到了名称匹配的目录项，它会更新 `*poff`，并返回通过 `iget` 获取的未加锁 inode。`dirlookup` 正是 `iget` 返回未加锁 inode 的原因。调用者已经对 `dp` 加了锁，因此如果查找的是 `.`（当前目录的别名），在返回前尝试对该 inode 加锁会导致对 `dp` 的重复加锁，从而产生死锁。（还存在涉及多个进程以及 `..`（父目录的别名）的更复杂的死锁场景；`.` 并非唯一的问题所在。）调用者可以先解锁 `dp`，再对 `ip` 加锁，从而确保同一时刻只持有一把锁。函数 `dirlink`（kernel/fs.c:554）将具有给定名称和 inode 编号的新目录项写入目录 `dp`。如果该名称已存在，`dirlink` 返回一个错误（kernel/fs.c:560-564）。主循环读取目录项，寻找未分配的目录项。当找到时，循环提前终止（kernel/fs.c:538-539），此时 `off` 被设置为该可用目录项的偏移量。否

则，循环结束时 `off` 被设置为 `dp->size`。无论哪种情况，`dirlink` 随后通过在偏移量 `off`

处写入，将新目录项添加到目录中（kernel/fs.c:574-577）。

8.12 代码：路径名

路径名查找涉及对 `dirlookup`

的一系列调用，每个路径组件调用一次。`Namei`（kernel/fs.c:661）对路径进行求值并返回对应的 inode。函数 `nameiparent` 是其变体：它在最后一个元素之前停止，返回父目录的 inode 并将最终元素复制到 `name` 中。两者都调用通用函数 `namex` 来完成实际工作。`Namex`（kernel/fs.c:626）首先确定路径求值从哪里开始。如果路径以斜杠开头，则从根目录开始求值；否则从当前目录开始（kernel/fs.c:630-633）。然后它使用 `skipelem` 依次处理路径的每个组件（kernel/fs.c:635）。循环的每次迭代都必须在当前 inode `ip` 中查找 `name`。迭代开始时锁定 `ip` 并检查它是否为目录。如果不是，查找失败（kernel/fs.c:636-640）。（锁定 `ip` 是必要的，

not because `ip->type` can change underfoot—it can't—but because until `ilock` runs, `ip->type`

不能保证已从磁盘加载。）如果调用的是 `nameiparent`

且当前为最后一个路径元素，则循环提前结束，符合 `nameiparent` 的定义；最终路径元素已被复制到 `name` 中，因此 `namex` 只需返回未锁定的 `ip`（kernel/fs.c:641-645）。最后，循环使用 `dirlookup` 查找路径元素，并通过设置 `ip = next` 为下一次迭代做准备（kernel/fs.c:646-651）。当循环遍历完所有路径元素后，返回 `ip`。

过程 `namex` 可能需要较长时间才能完成：如果路径中遍历的目录的 `inode` 和目录块不在缓冲区缓存中，则可能需要多次磁盘操作来读取它们。Xv6 经过精心设计，使得当一个内核线程对 `namex` 的调用因磁盘 I/O 而阻塞时，另一个正在查找不同路径名的内核线程可以并发继续执行。`namex` 分别锁定路径中的每个目录，从而使不同目录中的查找可以并行进行。

这种并发性带来了一些挑战。例如，当一个内核线程正在查找某个路径名时，另一个内核线程可能正在通过取消链接某个目录来修改目录树。潜在的风险是，某次查找可能正在搜索一个已被另一个内核线程删除的目录，且其块已被重新用于另一个目录或文件。

Xv6 避免了此类竞争条件。例如，当在 `namex` 中执行 `dirlookup` 时，查找线程持有该目录的锁，而 `dirlookup` 返回一个通过 `iget` 获取的 `inode`。`iget` 会增加 `inode` 的引用计数。只有在从 `dirlookup` 接收到 `inode` 之后，`namex` 才会释放目录上的锁。此时另一个线程可能会将该 `inode` 从目录中取消链接，但 xv6 不会删除该 `inode`，因为其引用计数仍然大于零。

另一个风险是死锁。例如，在查找 "." 时，`next` 与 `ip` 指向同一个 `inode`。在释放 `ip` 上的锁之前锁定 `next` 会导致死锁。为了避免这种死锁，`namex` 在获取 `next` 上的锁之前先解锁目录。这里我们再次看到 `iget` 与 `ilock` 分离的重要性。

8.13 文件描述符层

Unix 接口一个很酷的特性是，Unix 中的大多数资源都以文件的形式表示，包括控制台、管道等设备，当然还有真正的文件。文件描述符层正是实现这种统一性的层。正如我们在第 1 章中所看到的，xv6 为每个进程提供了自己的打开文件表，即文件描述符表。每个打开的文件由一个 `struct file` (`kernel/file.h:1`) 表示，它是对 `inode` 或管道的封装，同时包含一个 I/O 偏移量。每次调用 `open` 都会创建一个新的打开文件（一个新的 `struct file`）：如果多个进程独立打开同一个文件，不同的实例将拥有各自不同的 I/O 偏移量。另一方面，单个打开文件（同一个 `struct file`）可以在一个进程的文件表中多次出现，也可以出现在多个进程的文件表中。这种情况会在以下场景中发生：一个进程使用 `open` 打开文件后，通过 `dup` 创建了别名，或者通过 `fork` 将其共享给子进程。引用计数用于追踪某个打开文件的引用数量。文件可以以只读、只写或读写方式打开，`readable` 和 `writable` 字段记录这一信息。系统中所有打开的文件都保存在一个全局文件表 `f_table` 中。文件表提供以下函数：分配文件（`filealloc`）、创建重复引用（`filedup`）、释放引用（`fileclose`），以及读写数据（`fileread` 和 `filewrite`）。前三个函数遵循已经熟悉的模式。`filealloc` (`kernel/file.c:30`) 扫描文件表，查找

未被引用的文件（`f->ref == 0`）并返回一个新引用；`filedup` (`kernel/file.c:48`) 递增

引用计数；`fileclose` (`kernel/file.c:60`) 递减引用计数。当一个文件的引用计数降至零时，`fileclose` 会根据文件类型释放底层的管道或 `inode`。函数 `filestat`、`fileread` 和 `filewrite` 实现了对文件的

`stat`、`read`和`write`操作。`Filestat` (`kernel/file.c:88`) 仅允许作用于 `inode`，并调用 `stati`。`Fileread`和`filewrite` 首先检查打开模式是否允许该操作，然后将调用转发给管道或 `inode` 的具体实现。如果文件表示一个 `inode`，`fileread`和`filewrite` 会将 I/O 偏移量作为操作的偏移量，并在操作后推进该偏移量 (`kernel/file.c:122-123`) (`kernel/file.c:153-154`)。管道没有偏移量的概念。回想一下，`inode` 相关函数要求调用者自行处理加锁 (`kernel/file.c:94-96`) (`kernel/file.c:121-124`) (`kernel/file.c:163-166`)。`inode` 加锁有一个便利的副作用：读写偏移量的更新是原子性的，因此多个进程同时向同一文件写入时不会相互覆盖彼此的数据，尽管它们的写入内容可能会交错出现。

8.14 代码：系统调用

借助底层所提供的各种函数，大多数系统调用的实现都相当简单（参见 (`kernel/sysfile.c`)）。有几个调用值得深入研究。函数 `sys_link`和`sys_unlink` 负责编辑目录，创建或删除对 `inode` 的引用，它们是使用事务机制的又一个典型示例。`sys_link` (`kernel/sysfile.c:120`) 首先获取其参数，即两个字符串 `old`和`new` (`kernel/sysfile.c:125`)。假设

`old` 存在且不是目录 (`kernel/sysfile.c:129-132`)，`sys_link` 会递增其 `ip->nlink`

计数。然后 `sys_link` 调用 `nameiparent` 来查找 `new` 的父目录及最终路径元素 (`kernel/sysfile.c:145`)，并创建一个指向 `old` 的 `inode` 的新目录项 (`kernel/sysfile.c:148`)。新的父目录必须存在，且必须与已有的 `inode` 位于同一设备上：`inode` 编号只在单块磁盘上具有唯一含义。如果发生此类错误，`sys_link`

必须回退并递减 `ip->nlink`。

事务简化了实现过程，因为尽管需要更新多个磁盘块，但我们无需关心操作的顺序。这些操作要么全部成功，要么

全部不执行。例如，若不使用事务，在创建链接之前就更新 `ip->nlink`，会使

文件系统暂时处于不安全状态，期间若发生崩溃则可能造成严重破坏。使用事务后，我们无需担心这一问题。`sys_link` 为已有的 `inode` 创建一个新名称。函数 `create` (`kernel/sysfile.c:242`) 则为新的 `inode` 创建一个新名称。它是对三种文件创建系统调用的统一抽象：带 `O_CREATE` 标志的 `open` 创建普通文件，`mkdir` 创建新目录，`mkdev` 创建新设备文件。与 `sys_link` 类似，`create` 首先调用 `nameiparent` 获取父目录的 `inode`，然后调用 `dirlookup` 检查该名称是否已存在 (`kernel/sysfile.c:252`)。如果名称已存在，`create` 的行为取决于它被哪个系统调用所使用：`open` 的语义与 `mkdir` 和 `mkdev` 不同。如果 `create`

是代表 `open` 被调用 (`type == T_FILE`)，且已存在的名称本身是普通文件，则

`open` 将其视为成功，`create` 也同样处理 (`kernel/sysfile.c:256`)。否则，视为错误 (`kernel/sysfile.c:257-258`)。如果名称尚不存在，`create` 此时通过 `ialloc` 分配一个新的 `inode`

(kernel/sysfile.c:261)。若新 inode 是目录，`create` 会用 `.` 和 `..` 条目对其进行初始化。最后，在数据正确初始化之后，`create` 将其链接到父目录中 (kernel/sysfile.c:274)。与 `sys_link` 一样，`create` 同时持有两个 inode 锁：`ip` 和 `dp`。这里不存在死锁的可能性，因为 inode `ip` 是新分配的：系统中不会有其他进程持有 `ip` 的锁后再尝试获取 `dp` 的锁。借助 `create`，可以很容易地实现 `sys_open`、`sys_mkdir` 和 `sys_mknod`。`sys_open` (kernel/sysfile.c:287) 最为复杂，因为创建新文件只是它功能的一小部分。如果 `open` 传入了 `O_CREATE` 标志，它会调用 `create` (kernel/sysfile.c:301)；否则，调用 `namei` (kernel/sysfile.c:307)。`create` 返回一个已加锁的 inode，而 `namei` 不会，因此 `sys_open` 必须自行对 inode 加锁。这也提供了一个方便的时机，用于检查目录是否仅以只读方式打开，而非写入。无论以哪种方式获得 inode，`sys_open` 都会分配一个文件和一个文件描述符 (kernel/sysfile.c:325)，然后填充该文件的各个字段 (kernel/sysfile.c:337-

342)。注意，没有其他进程能够访问这个尚未完全初始化的文件，因为它只存在于当前进程的表中。第 7 章在我们尚未拥有文件系统之前就已经考察了管道的实现。函数 `sys_pipe` 通过提供一种创建管道对的方式，将该实现与文件系统连接起来。其参数是一个指向存放两个整数空间的指针，函数将在其中记录两个新的文件描述符，随后分配管道并安装这两个文件描述符。

8.15 现实世界

现实世界操作系统中的缓冲区缓存比 xv6

的复杂得多，但其服务于相同的两个目的：缓存以及同步对磁盘的访问。xv6 的缓冲区缓存与 V6 的一样，使用简单的最近最少使用 (LRU) 淘汰策略；还有许多更复杂的策略可以实现，每种策略对某些工作负载表现良好，对其他工作负载则不那么理想。更高效的 LRU

缓存会消除链表，转而使用哈希表进行查找，并使用堆来执行 LRU 淘汰。现代缓冲区缓存通常与虚拟内存系统集成，以支持内存映射文件。

xv6 的日志系统效率低下。提交不能与文件系统系统调用并发执行。即使一个块中只有少数几个字节发生了变化，系统也会记录整个块。它每次一个块地执行同步日志写入，每次写入都可能需要整整一个磁盘旋转时间。真实的日志系统会解决所有这些问题。

日志并非提供崩溃恢复的唯一方式。早期文件系统在重启时使用清道夫程序（例如 UNIX 的 `fsck` 程序）来检查每个文件和目录以及块和 inode 空闲列表，查找并解决不一致问题。对于大型文件系统，清道夫扫描可能需要数小时，而且在某些情况下，无法以使原始系统调用具有原子性的方式来解决不一致问题。从日志中恢复要快得多，并且能够在面对崩溃时保证系统调用的原子性。

xv6 使用与早期 UNIX 相同的基本磁盘上 inode

和目录布局；这一方案在多年来表现出了显著的持久性。BSD 的 UFS/FFS 和 Linux 的 ext2/ext3 使用的本质上是相同的数据结构。文件系统布局中效率最低的部分是目录，每次查找时都需要对所有磁盘块进行线性扫描。当目录只有几个磁盘块时，这是合理的，但对于包含大量文件的目录来说代价高昂。Microsoft Windows 的 NTFS、Mac OS X 的 HFS 以及 Solaris 的

ZFS（仅举几例）将目录实现为磁盘上块的平衡树。这很复杂，但能保证目录查找的对数时间复杂度。

xv6 对磁盘故障处理得很天真：如果磁盘操作失败，xv6 会 panic。这是否合理取决于硬件：如果操作系统构建于使用冗余来屏蔽磁盘故障的专用硬件之上，也许操作系统遇到故障的频率极低，发生 panic 是可以接受的。另一方面，使用普通磁盘的操作系统应该预期故障并更优雅地处理它们，以便一个文件中某个块的丢失不会影响文件系统其余部分的使用。

xv6 要求文件系统适配单一磁盘设备，且大小不可改变。随着大型数据库和多媒体文件不断推高存储需求，操作系统正在开发消除“每个文件系统一块磁盘”瓶颈的方法。基本方法是将多个磁盘合并为单一逻辑磁盘。RAID 等硬件解决方案仍然最为流行，但当前趋势是尽可能多地在软件中实现这些逻辑。这些软件实现通常允许丰富的功能，例如通过动态添加或移除磁盘来扩展或收缩逻辑设备。当然，能够动态扩展或收缩的存储层需要文件系统也能做到同样的事：xv6 使用的固定大小 inode 块数组在这样的环境中表现不佳。将磁盘管理与文件系统分离可能是最简洁的设计，但两者之间复杂的接口促使一些系统（如 Sun 的 ZFS）将它们合并在一起。

xv6 的文件系统还缺乏现代文件系统的许多其他特性；例如，它不支持快照和增量备份。现代 Unix 系统允许通过与访问磁盘存储相同的系统调用来访问多种资源：命名管道、网络连接、远程访问的网络文件系统，以及诸如 `/proc` 之类的监控和控制接口。这些系统通常为每个打开的文件提供一张函数指针表（每种操作对应一个函数指针），并通过调用函数指针来调用该 inode 对应操作的实现，而不是像 xv6 那样在 `fileread` 和 `filewrite` 中使用 `if` 语句。网络文件系统和用户级文件系统提供的函数会将这些调用转换为网络 RPC，并在返回前等待响应。

8.16 练习

1. 为什么 `balloc` 中会触发 panic？xv6 能够恢复吗？
2. 为什么 `ialloc` 中会触发 panic？xv6 能够恢复吗？
3. 为什么 `filealloc` 在文件耗尽时不触发 panic？为什么这种情况更为常见，因而值得专门处理？
4. 假设在 `sys_link` 调用 `iunlock(ip)` 和 `dirlink` 之间，另一个进程将 `ip` 对应的文件取消链接（`unlink`）。链接是否会被正确创建？为什么？
5. `create` 发起了四次函数调用（一次调用 `ialloc`，三次调用 `dirlink`），并要求这四次调用全部成功。如果其中任何一次失败，`create` 就会调用 panic。为什么这样做是可以接受的？为什么这四次调用都不会失败？

6. `sys_chdir` calls `iunlock(ip)` before `iput(cp->cwd)`, which might try to lock `cp->cwd`,

但如果将 `iunlock(ip)` 推迟到 `iput` 之后执行，也不会导致死锁。为什么？

7. 实现 `lseek` 系统调用。支持 `lseek` 还需要修改

```
filewrite to fill holes in the file with zero if lseek sets off beyond f->ip->size.
```

8. 在 `open` 中添加 `O_TRUNC` 和 `O_APPEND` 支持，使得 shell 中的 `>` 和 `>>` 运算符能够正常工作。

9. 修改文件系统以支持符号链接（symbolic links）。

10. 修改文件系统以支持命名管道（named pipes）。

11. 修改文件与虚拟内存系统以支持内存映射文件（memory-mapped files）。

第9章：并发回顾

在内核设计中，同时实现良好的并行性能、并发下的正确性以及可读性强的代码，是一项巨大的挑战。直接使用锁是实现正确性的最佳途径，但并非总是可行。本章重点介绍 xv6 被迫以复杂方式使用锁的示例，以及 xv6 使用类锁技术但并非真正使用锁的示例。

9.1 锁的使用模式

缓存项通常是加锁的难点。例如，文件系统的块缓存（kernel/bio.c:26）存储了最多 NBUF 个磁盘块的副本。至关重要的一点是，给定的磁盘块在缓存中最多只能有一个副本；否则，不同进程可能会对同一块的不同副本进行相互冲突的修改。每个缓存块存储在一个 `struct buf`（kernel/buf.h:1）中。`struct buf` 有一个锁字段，用于确保同一时刻只有一个进程使用给定的磁盘块。然而，这个锁还不够：如果某个块根本不在缓存中，而两个进程同时想使用它，该怎么办？此时不存在 `struct buf`（因为该块尚未被缓存），因此也没有任何东西可以加锁。xv6 通过将一个额外的锁（`bcache.lock`）与已缓存块的标识集合相关联来处理这种情况。需要检查某个块是否已被缓存（例如 `bget`（kernel/bio.c:59））或修改缓存块集合的代码，必须持有 `bcache.lock`；在该代码找到所需的块和 `struct buf` 之后，就可以释放 `bcache.lock`，并仅对该特定块加锁。这是一种常见模式：对项目集合使用一把锁，加上每个项目各自一把锁。

通常情况下，获取锁的函数也会释放它。但更精确的理解方式是：锁在一个必须以原子方式呈现的操作序列开始时被获取，在该序列结束时被释放。如果序列在不同函数、不同线程或不同 CPU 上开始和结束，那么锁的获取和释放也必须如此。锁的作用是迫使其他使用者等待，而不是将某块数据绑定到特定的执行实体上。一个例子是 `yield`（kernel/proc.c:515）中的 `acquire`，它由调度器线程而非获取锁的进程来释放。另一个例子是 `ilock`（kernel/fs.c:289）中的 `acquiresleep`；这段代码在读取磁盘时经常会睡眠；它可能在不同的 CPU 上被唤醒，这意味着锁可能在不同的 CPU 上被获取和释放。

释放一个由嵌入其中的锁所保护的對象是一件微妙的事情，因为持有该锁并不足以保证释放操作是正确的。问题出现在另一个线程正在 `acquire` 中等待使用该对象的情况下；释放该对象会隐式地释放嵌入的锁，从而导致等待的线程出现异常。一种解决方案是追踪该对象有多少个引用存在，只有在最后一个

```
reference disappears. See pipeclose (kernel/pipe.c:59) for an example; pi->readopen and pi->writeopen track whether the pipe has file descriptors referring to it.
```

9.2 类锁模式

在许多地方，xv6 使用引用计数或标志作为一种软锁，用于表示对象已被分配且不应被释放或重用。进程的 `p->state` 就以这种方式运作，`file`、`inode` 和 `buf` 结构中的引用计数也是如此。虽然在每种情况下都由锁来保护

标志或引用计数，但正是后者防止了对象被过早释放。文件系统将 `struct inode` 引用计数用作一种可由多个进程持有的共享锁，以避免在代码使用普通锁时可能发生的死锁。例如，`namex` (`kernel/fs.c:626`) 中的循环依次锁定每个路径名分量所命名的目录。然而，`namex` 必须在循环结束时释放每个锁，因为如果持有多个锁，当路径名包含点号（例如 `a/./b`）时，它可能与自身发生死锁。它也可能与涉及该目录和 `..` 的并发查找操作发生死锁。正如第 8 章所解释的，解决方案是让循环在进入下一次迭代时携带该目录的 `inode`，并将其引用计数递增，但不加锁。

某些数据项在不同时刻受到不同机制的保护，有时甚至不依赖显式锁，而是隐式地通过 `xv6` 代码的结构来防止并发访问。例如，当一个物理页面空闲时，它受 `kmem.lock` (`kernel/kalloc.c:24`) 保护。如果该页面随后被分配为管道 (`kernel/pipe.c:23`)，则由另一个锁（内嵌的 `pi->lock`）保护。如果该页面被重新分配给新进程的用户内存，则根本不受任何锁的保护。相反，分配器不会将该页面交给其他任何进程（直到它被释放），这一事实保护了它免受并发访问。新进程内存的所有权是复杂的：首先由父进程在 `fork` 中分配并操作，然后由子进程使用，之后（在子进程退出后）父进程再次拥有该内存并将其传递给 `kfree`。这里有两点启示：一个数据对象在其生命周期的不同阶段可以通过不同方式得到并发保护，且这种保护可以采用隐式结构的形式，而非显式锁。

最后一个类锁示例是在调用 `mycpu()` (`kernel/proc.c:68`) 时需要禁用中断。禁用中断使调用代码相对于可能强制发生上下文切换的时钟中断具有原子性，从而避免进程被迁移到不同的 CPU 上。

9.3 完全不使用锁

`xv6` 中有少数几个地方在完全没有锁的情况下共享可变数据。其一是自旋锁的实现，尽管也可以将 RISC-V 的原子指令视为依赖于硬件实现的锁。其二是 `main.c` (`kernel/main.c:7`) 中的 `started` 变量，用于防止其他 CPU 在 CPU 零号完成 `xv6` 初始化之前运行；`volatile` 关键字确保编译器实际生成加载和存储指令。其三是 `proc.c` 中对

`p->parent` 的一些使用 (`kernel/proc.c:398`) (`kernel/proc.c:306`)，在这些地方使用正确的锁可能会导致死锁，但显然没有其他进程会同时修改 `p->parent`。

第四个例子是 `p->killed`，它在持有 `p->lock` 时被设置 (`kernel/proc.c:611`)，但

在没有持有锁的情况下被检查 (`kernel/trap.c:56`)。`xv6` 中存在这样的情况：一个 CPU 或线程写入某些数据，而另一个 CPU 或线程读取该数据，但没有专门用于保护该数据的锁。例如，在 `fork` 中，父进程写入子进程的用户内存页，而子进程（一个不同的线程，可能运行在不同的 CPU 上）读取这些页；没有锁显式地保护这些页。这严格来说并非锁的问题，因为子进程直到父进程完成写入之后才开始执行。这是一个潜在的内存排序问题（见第 6 章），因为如果没有内存屏障，便没有理由期望一个 CPU 能看到另一个 CPU 的写入。然而，由于父进程会释放锁，而子进程在启动时会获取锁，`acquire` 和 `release` 中的内存屏障确保了子进程的 CPU 能看到父进程的写入。

9.4 并行性

锁的主要作用是为了保证正确性而抑制并行性。由于性能同样重要，内核设计者通常需要考虑如何使用锁，以同时实现正确性与良好的并行性。虽然 xv6 并非系统性地针对高性能而设计，但仍值得考虑哪些 xv6 操作可以并行执行，哪些操作可能在锁上产生冲突。xv6

中的管道是并行性较好的一个例子。每个管道都有自己的锁，因此不同的进程可以在不同的 CPU 上并行地读写不同的管道。然而，对于同一个管道，写者和读者必须等待对方释放锁，它们不能同时读写同一个管道。对空管道的读操作（或对满管道的写操作）也必须阻塞，但这并非由锁机制造成。上下文切换是一个更复杂的例子。两个内核线程，各自在自己的 CPU 上执行，可以同时调用 `yield`、`sched` 和 `swtch`，这些调用将并行执行。每个线程各自持有一把锁，但它们是不同的锁，因此不必互相等待。然而，一旦进入调度器，两个 CPU 在搜索进程表以寻找处于 `RUNNABLE`

状态的进程时，可能会在锁上产生冲突。也就是说，xv6 在上下文切换期间很可能从多个 CPU 中获得性能收益，但或许不如理论上的最大收益。另一个例子是不同 CPU 上的不同进程并发调用 `fork`。这些调用可能需要在 `pid_lock` 和 `kmem.lock` 上互相等待，以及为了在进程表中搜索 `UNUSED` 进程而需要的各进程锁。另一方面，两个正在执行 `fork` 的进程可以完全并行地复制用户内存页和构建页表页。上述每个例子中的锁方案在某些情况下都牺牲了并行性能。在每种情况下，都可以通过更精细的设计来获得更高的并行度。是否值得这样做取决于具体细节：相关操作被调用的频率、代码持有竞争锁的时长、可能同时运行冲突操作的 CPU 数量，以及代码中其他部分是否存在更严重的瓶颈。要判断某种锁方案是否会导致性能问题，或新设计是否有显著改善，往往并不容易，因此通常需要在真实工作负载下进行实际测量。

9.5 练习

1. 修改 xv6 的管道实现，允许对同一管道的读操作和写操作在不同核心上并行进行。
2. 修改 xv6 的 `scheduler()`，以减少不同核心同时查找可运行进程时的锁竞争。

```
for runnable processes at the same time.
```

3. 消除 xv6 的 `fork()` 中部分串行化操作。

第 10 章：总结

本书通过逐行研究一个操作系统——xv6，介绍了操作系统中的核心思想。有些代码行体现了核心思想的精髓（例如上下文切换、用户/内核边界、锁等），每一行都至关重要；另一些代码行则展示了如何实现某个特定的操作系统思想，这些实现完全可以用不同的方式来完成（例如更好的调度算法、更好的磁盘上数据结构来表示文件、更好的日志机制以支持并发事务等）。所有这些思想都在一个特定的、非常成功的系统调用接口——Unix 接口——的背景下加以阐述，但这些思想同样适用于其他操作系统的设计。

参考文献

- [1] The RISC-V instruction set manual: privileged architecture. <https://riscv.org/specifications/privileged-isa/>, 2019.
- [2] The RISC-V instruction set manual: user-level ISA. <https://riscv.org/specifications/isa-spec-pdf/>, 2019.
- [3] Hans-J Boehm. Threads cannot be implemented as a library. ACM PLDI Conference, 2005.
- [4] Edsger Dijkstra. Cooperating sequential processes. <https://www.cs.utexas.edu/users/EWD/transcriptions/EWD01xx/EWD123.html>, 1965.
- [5] Maurice Herlihy and Nir Shavit. The Art of Multiprocessor Programming, Revised Reprint. 2012.
- [6] Brian W. Kernighan. The C Programming Language. Prentice Hall Professional Technical Reference, 2nd edition, 1988.
- [7] Donald Knuth. Fundamental Algorithms. The Art of Computer Programming. (Second ed.), volume 1. 1997.
- [8] L Lamport. A new solution of dijkstra's concurrent programming problem. Communications of the ACM, 1974.
- [9] John Lions. Commentary on UNIX 6th Edition. Peer to Peer Communications, 2000.
- [10] Paul E. Mckenney, Silas Boyd-wickizer, and Jonathan Walpole. RCU usage in the linux kernel: One decade later, 2013.
- [11] Martin Michael and Daniel Durich. The NS16550A: UART design and application considerations. http://bitsavers.trailing-edge.com/components/national/_appNotes/AN-0491.pdf, 1987.
- [12] David Patterson and Andrew Waterman. The RISC-V Reader: an open architecture Atlas. Strawberry Canyon, 2017.

[13] Dave Presotto, Rob Pike, Ken Thompson, and Howard Trickey. Plan 9, a distributed system. In In Proceedings of the Spring 1991 EurOpen Conference, pages 43 – 50, 1991.

[14] Dennis M. Ritchie and Ken Thompson. The UNIX time-sharing system. Commun. ACM, 17(7):365 – 375, July 1974.

索引

., 92, 94 conditional synchronization (条件同步) , 71 .., 92, 94 conflict (冲突) , 58 /init, 28, 37
contention (竞争) , 58 _entry, 27 contexts (上下文) , 68 convoys (护航现象) , 78 absorption (吸收) , 86
copy-on-write (COW) fork (写时复制 fork) , 46 acquire, 59, 62 copyinstr, 45 address space (地址空间) , 25
copyout, 37 argc, 37 coroutines (协程) , 69 argv, 37 CPU, 9

atomic (原子) , 59

cpu->scheduler, 68, 69

crash recovery (崩溃恢复) , 81 balloc, 87, 89 create, 94 batching (批处理) , 85 critical section (临界区) , 58
bcache.head, 83 current directory (当前目录) , 17 begin_op, 86 bfree, 87 deadlock (死锁) , 60 bget, 83 direct
blocks (直接块) , 91 binit, 83 direct memory access (DMA) (直接内存访问) , 52 bmap, 91 dirlink, 92 bottom
half (下半部) , 49 dirlookup, 91, 92, 94 bread, 82, 84 DIRSIZ, 91 brelse, 82, 84 disk (磁盘) , 83 BSIZE, 91
driver (驱动程序) , 49 buf, 82 dup, 93 busy waiting (忙等待) , 72 ecall, 23, 26 bwrite, 82, 84, 86 ELF format (ELF
格式) , 37 ELF_MAGIC, 37 chan, 72, 74 end_op, 86 child process (子进程) , 10 exception (异常) , 41 commit,
84 exec, 12, 14, 27, 37, 44 concurrency (并发) , 55 exit, 11, 70, 76 concurrency control (并发控制) , 55 condition
lock (条件锁) , 73 file descriptor (文件描述符) , 13

filealloc, 93 kvmmap, 33 fileclose, 93 filedup, 93 lazy allocation (惰性分配) , 47 fileread, 93, 96 links (链接) , 18
filestat, 93 loadseg, 37 filewrite, 87, 93, 96 lock (锁) , 55 fork, 10, 12, 14, 93 log (日志) , 84 forkret, 69 log_write, 86
freerange, 35 lost wake-up (丢失唤醒) , 72 fsck, 95 machine mode (机器模式) , 23 fsinit, 86 main, 33 – 35, 83
ftable, 93 malloc, 13 getcmd, 12 mappages, 33 group commit (组提交) , 85 memory barrier (内存屏障) , 63 guard
page (保护页) , 33 memory model (内存模型) , 63 memory-mapped (内存映射) , 31, 49 hartid, 70
metadata (元数据) , 18 microkernel (微内核) , 24 I/O, 13 mkdev, 94 I/O concurrency (I/O 并发) , 51 mkdir, 94
I/O redirection (I/O 重定向) , 14 mkfs, 82 ialloc, 89, 94 monolithic kernel (宏内核) , 21, 23 iget, 88, 89, 92
multi-core (多核) , 21 ilock, 88, 89, 92 multiplexing (多路复用) , 67 indirect block (间接块) , 91
multiprocessor (多处理器) , 21 initcode.S, 27, 44 mutual exclusion (互斥) , 57 initlog, 86 mycpu, 70
inode (索引节点) , 18, 82, 87 myproc, 71 install_trans, 86 interface design (接口设计) , 9 namei, 37, 94
interrupt (中断) , 41 nameiparent, 92, 94 iput, 88, 89 namex, 92 isolation (隔离) , 21 NBUF, 83 itrunc, 89, 91
NDIRECT, 90, 91 iunlock, 89 NINDIRECT, 91

kalloc, 35 O_CREATE, 94 kernel (内核) , 9, 23 open, 93, 94 kernel space (内核空间) , 9, 23

kfree, 35
kinit, 35

p->context, 70
p->killed, 77, 101

kvminit, 33	p->kstack, 26
kvminithart, 34	p->lock, 69, 70, 74
p->pagetable, 26, 27	sbrk, 13
p->state, 27	scause, 42
p->xxx, 26	sched, 68, 69, 74

page (页), 29 scheduler (调度器), 69, 70 page table entries (PTEs) (页表项), 29 semaphore (信号量), 71
 page-fault exception (缺页异常), 30, 47 sepc, 42 paging from disk (从磁盘换页), 47 sequence
 coordination (序列协调), 71 parent process (父进程), 10 serializing (串行化), 58 path (路径), 17
 sfence.vma, 34 persistence (持久性), 81 shell (Shell), 10 PGROUNDUP, 35 signal (信号), 78 physical
 address (物理地址), 25 skipelem, 92 PHYSTOP, 33, 34 sleep, 72 – 74 PID, 10 sleep-locks (睡眠锁), 63
 pipe (管道), 15 SLEEPING, 74 piperead, 75 sret, 27 pipewrite, 75 sscratch, 42 polling (轮询), 52, 72 sstatus, 42
 pop_off, 62 stat, 91, 93 printf, 12 stati, 91, 93 priority inversion (优先级反转), 78 struct context, 68 privileged
 instructions (特权指令), 23 struct cpu, 70 proc_pagetable, 37 struct dinode, 87, 90 process (进程), 9, 24 struct
 dirent, 91 procinit, 34 struct elfhdr, 37 programmed I/O (程序控制 I/O), 52 struct file, 93 PTE_R, 30

struct inode, 88

PTE_U, 30

struct pipe, 75

PTE_V, 30

struct proc, 26

PTE_W, 30

struct run, 34

PTE_X, 30

struct spinlock, 58

push_off, 62 stval, 47 race condition (竞态条件), 57 stvec, 42 read, 93 superblock (超级块), 82 readi, 37, 91
 supervisor mode (监督者模式), 23 recover_from_log, 86 swtch, 68 – 70 release, 59, 62 SYS_exec, 45 root (根), 17
 sys_link, 94 round robin (轮转调度), 77 sys_mkdir, 94 RUNNABLE, 70, 74, 76 sys_mknod, 94 sys_open, 94 satp,
 30 sys_pipe, 95

sys_sleep, 62 unlink, 85 sys_unlink, 94 user memory (用户内存), 25 syscall, 45 user mode (用户模式), 23 system
 call (系统调用), 9 user space (用户空间), 9, 23 usertrap, 68 T_DEV, 91 ustack, 37 T_DIR, 91 uvmmalloc, 37
 T_FILE, 94 thread (线程), 26 valid (有效), 83 thundering herd (惊群效应), 78 virtio_disk_rw, 83, 84 ticks, 62
 virtual address (虚拟地址), 25 tickslock, 62 time-share (分时), 10, 21 wait, 11, 12, 70, 76 top half (上半部), 49

wait channel (等待通道) , 72 TRAMPOLINE, 43 wakeup, 61, 72, 74 trampoline, 26, 43 walk, 33
transaction (事务) , 81 walkaddr, 37 Translation Look-aside Buffer (TLB) (转址旁路缓冲) , 34 write, 85, 93
transmit complete (传输完成) , 50 write-through (直写) , 88 trap (陷阱) , 41 writei, 87, 91 trapframe, 26 type
cast (类型转换) , 35 yield, 68 – 70

UART, 49 ZOMBIE (僵尸) , 76